

On-Device AI

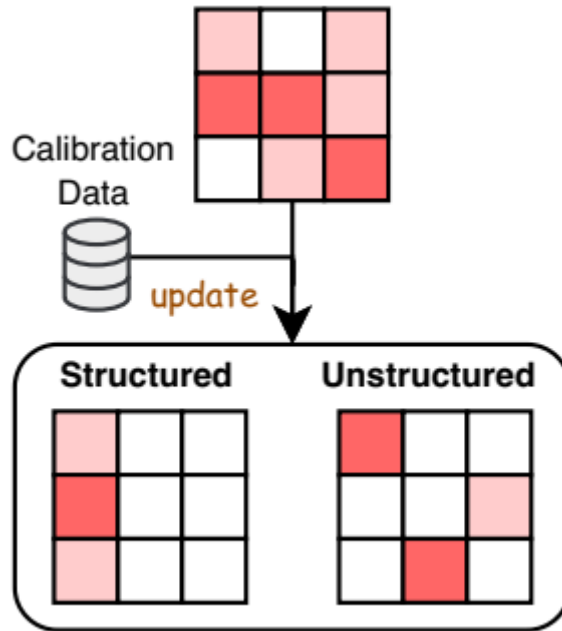
Part 2

성균관대 신동균 교수

Outline

- Pruning & Sparsity
- Knowledge Distillation
- Efficient LLM Inference
- HW-aware Optimization

Pruning



Network Pruning

- Network Redundancy
 - For easy training, DNNs are usually over-parameterized
 - Weights can be removed (i.e., set to zero)
- Optimal Brain Damage (LeCun, 1989)
 - Compute the impact of each weight on the training loss (**weight saliency**)
 - Low saliency weights were removed and the remaining weights were fine tuned
- Magnitude-based pruning (2015)
 - Too difficult to compute saliency for the large-scaled DNNs
 - Simply based on the **magnitude** of the weights
 - Small weights were pruned and the model was fine tuned to restore the accuracy
 - Use L1/L2 regularization to penalize non-zero parameters resulting in more parameters near zero

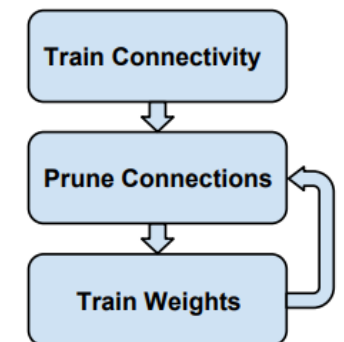
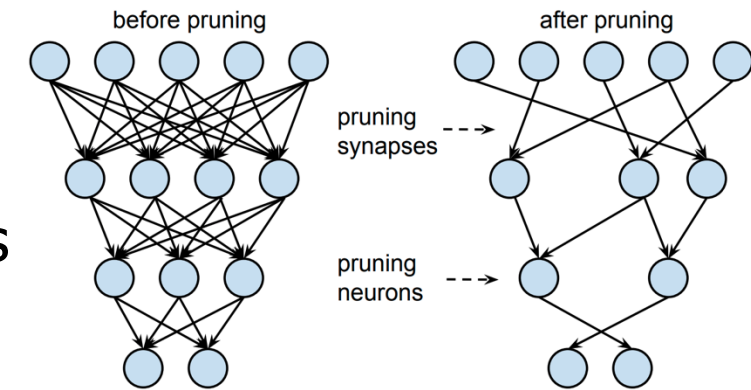
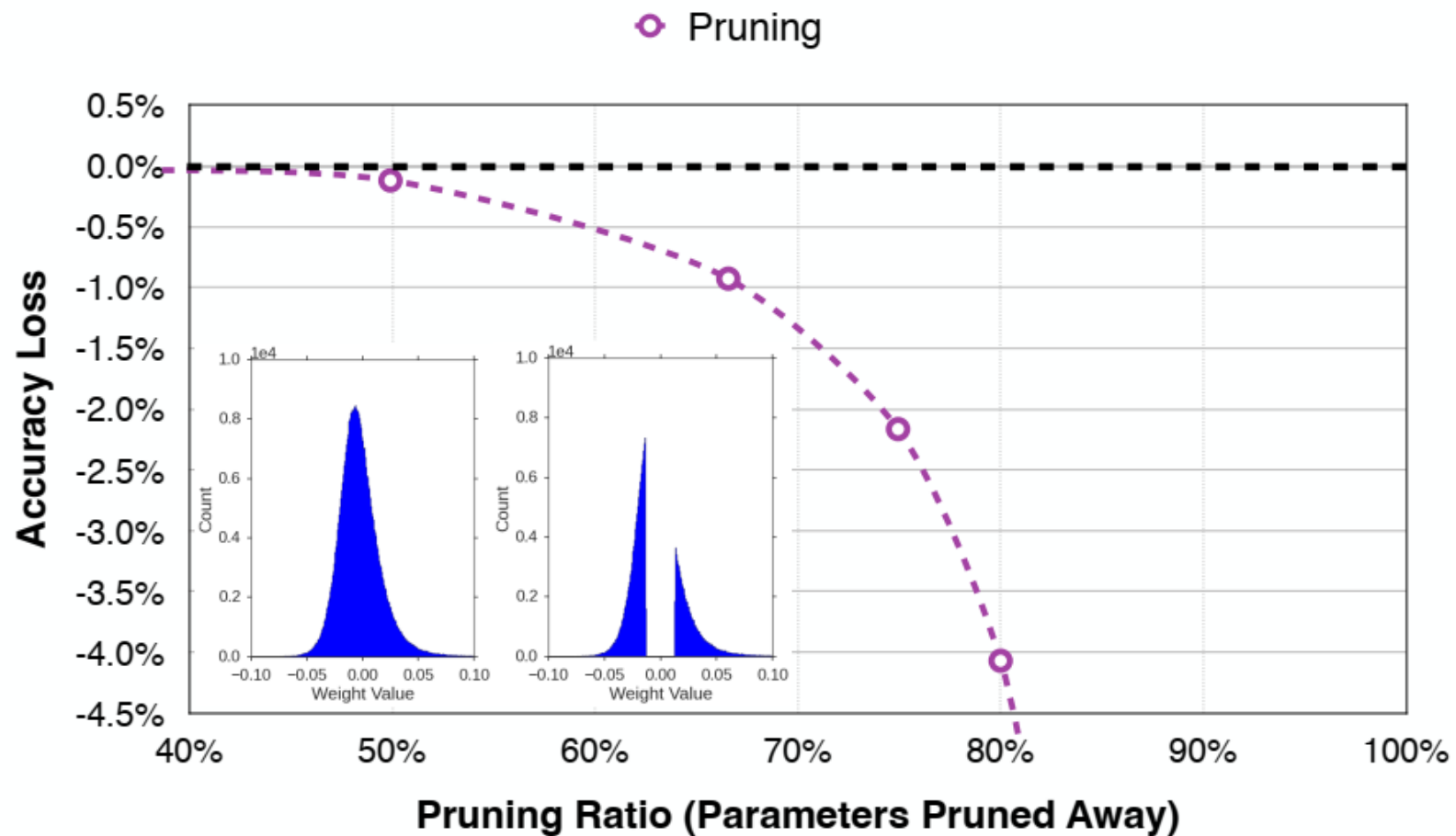
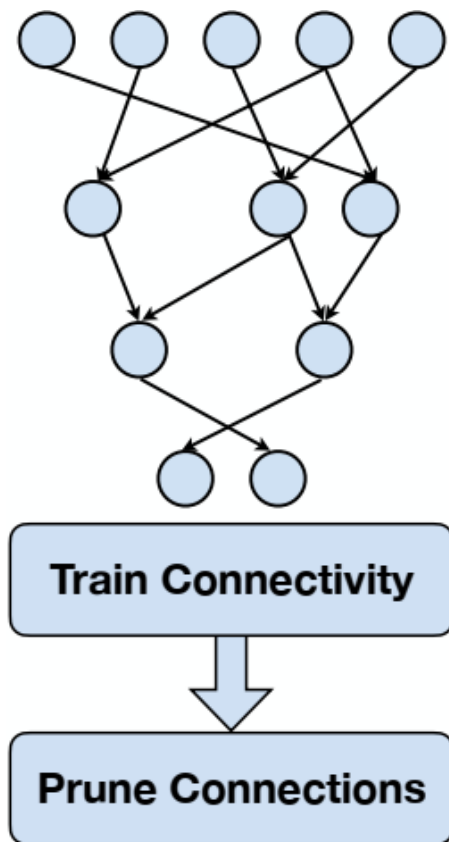


Figure 2: Three-Step Training Pipeline.

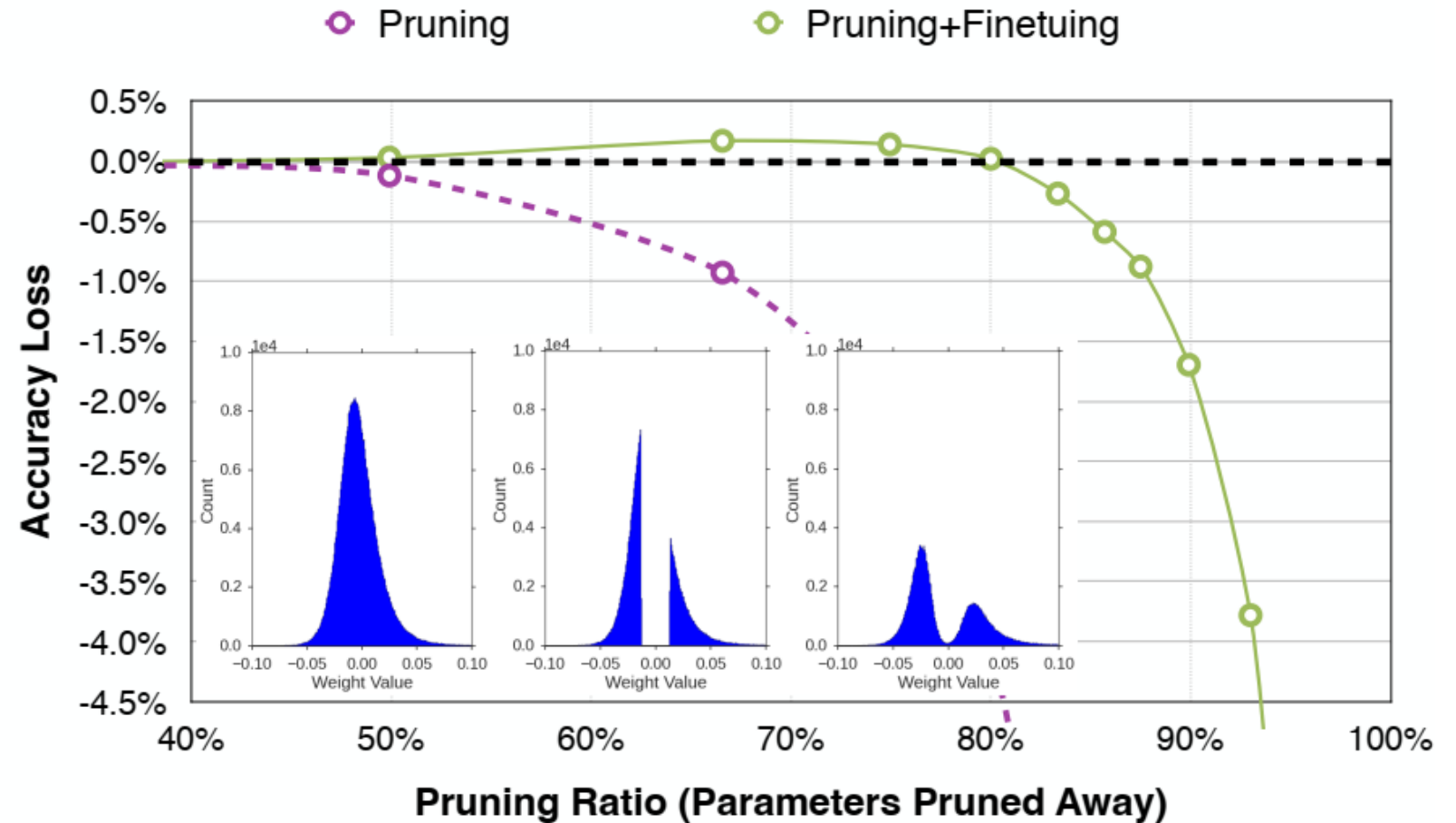
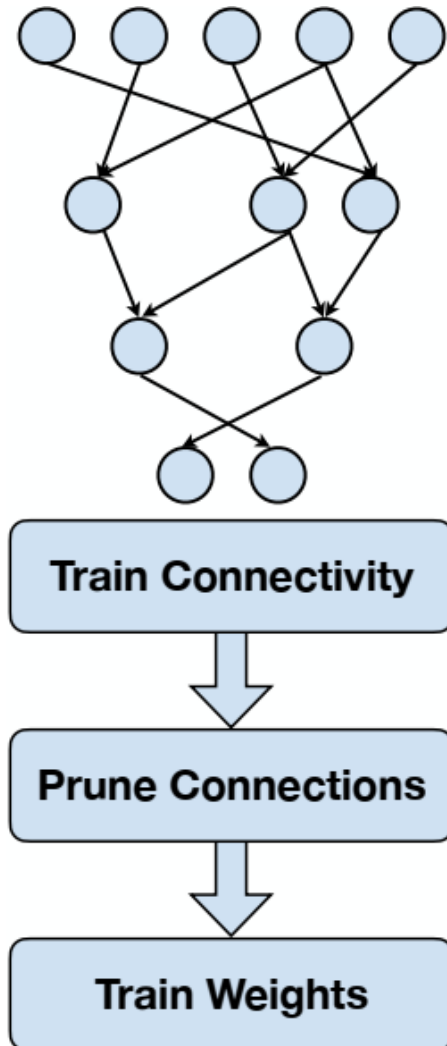
Network Pruning

From <https://efficientml.ai>



Network Pruning

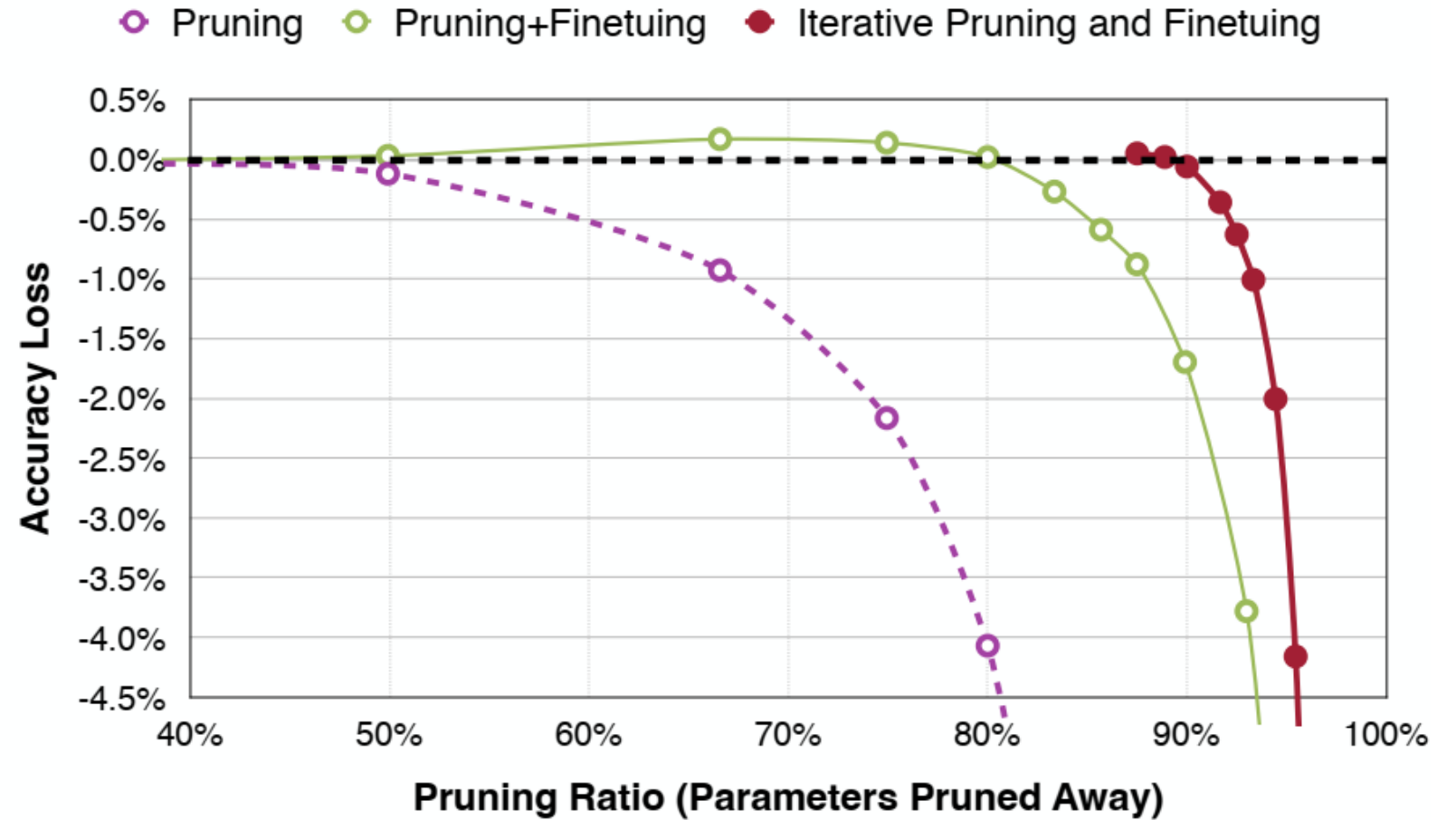
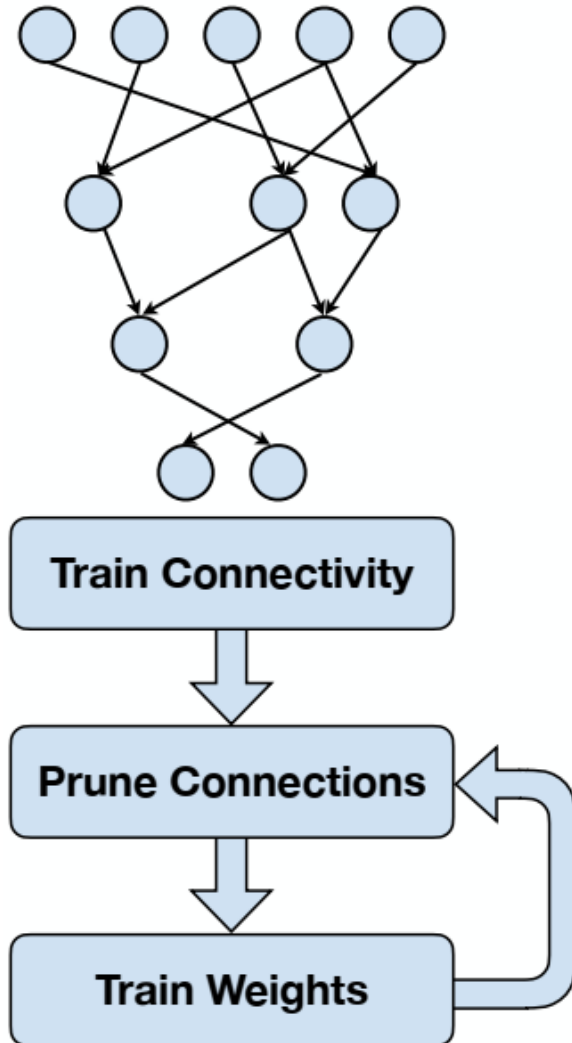
From <https://efficientml.ai>



- After pruning, the model may decrease, especially for larger pruning ratio.
- Fine-tuning can recover the accuracy and push the pruning ratio higher.
 - Learning rate for fine-tuning is usually 1/100 or 1/10 of the original LR.

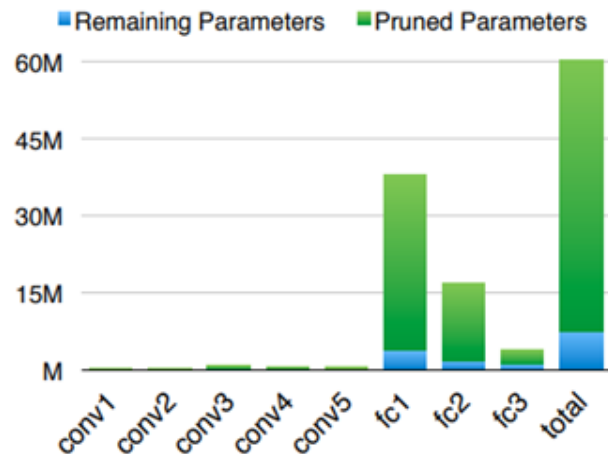
Network Pruning

From <https://efficientml.ai>



Iterative pruning gradually increases the target sparsity in each iteration.

Reduced OPs and Model Size



Most of the weight reduction comes from FC layers

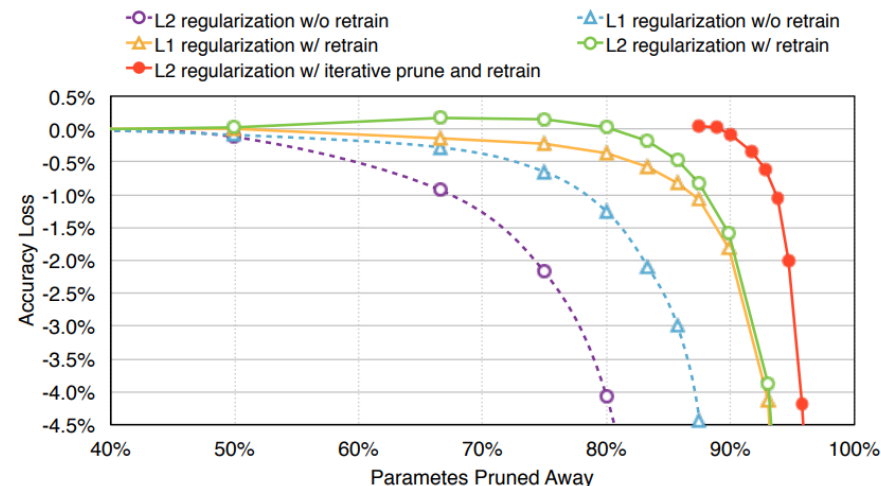
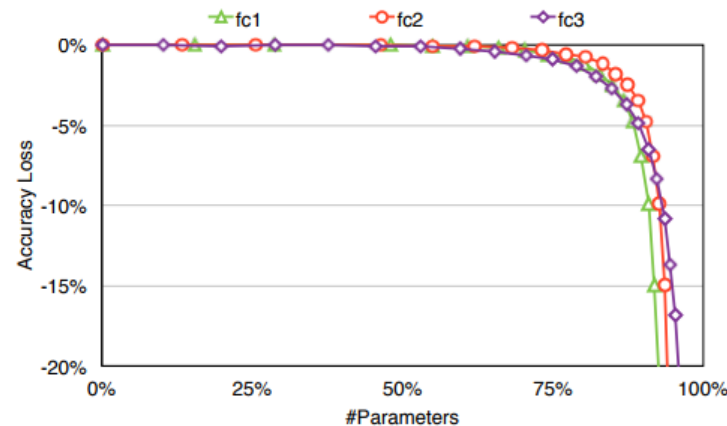
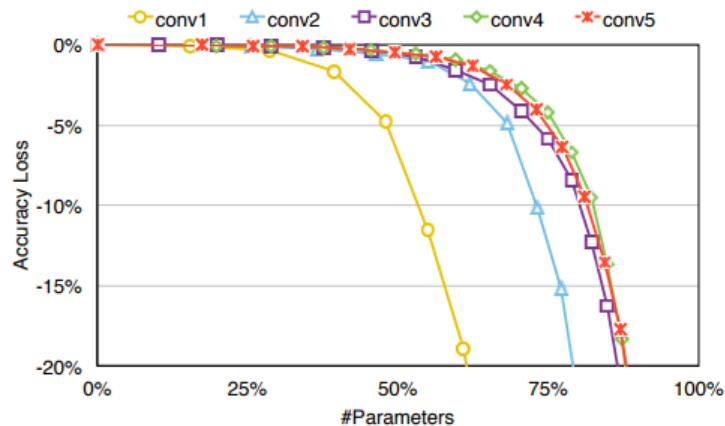


Figure 5: Trade-off curve for parameter reduction and loss in top-5 accuracy. L1 regularization performs better than L2 at learning the connections without retraining, while L2 regularization performs better than L1 at retraining. Iterative pruning gives the best result.

Conv layers are more sensitive to pruning.



FC layers have more redundancy.

Figure 6: Pruning sensitivity for CONV layer (left) and FC layer (right) of AlexNet.

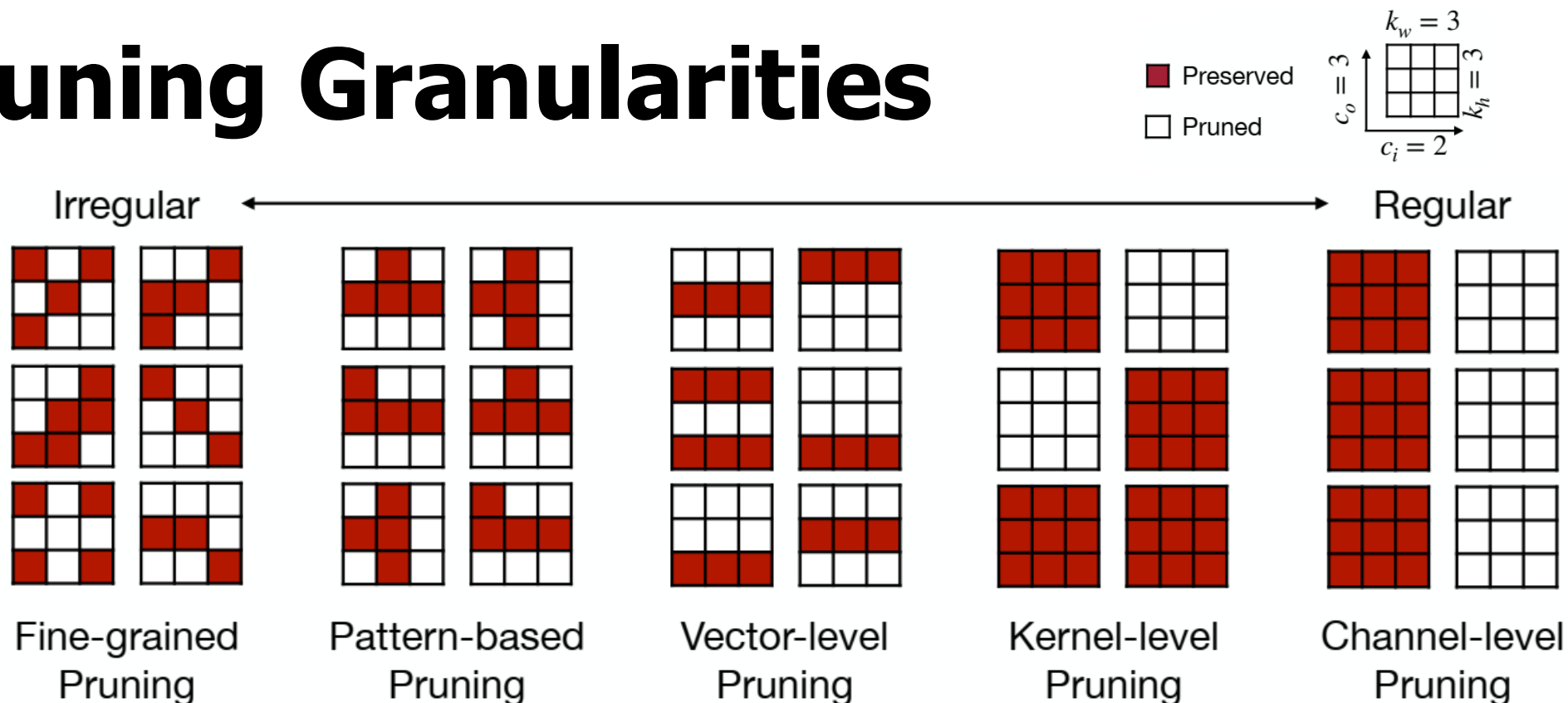
Reduced OPs and Model Size

Network	Top-1 Error	Top-5 Error	Parameters	Compression Rate
LeNet-300-100 Ref	1.64%	-	267K	12×
LeNet-300-100 Pruned	1.59%	-	22K	
LeNet-5 Ref	0.80%	-	431K	12×
LeNet-5 Pruned	0.77%	-	36K	
AlexNet Ref	42.78%	19.73%	61M	9×
AlexNet Pruned	42.77%	19.67%	6.7M	
VGG-16 Ref	31.50%	11.32%	138M	13×
VGG-16 Pruned	31.34%	10.88%	10.3M	

Design Choice of Pruning

- Pruning Granularity/Pattern
- Pruning Criterion
 - What synapses/neurons should we prune?
- Pruning Ratio
- Fine-tuning after Pruning or Training Pruned Neural Network

Pruning Granularities



Fine-grained/Unstructured

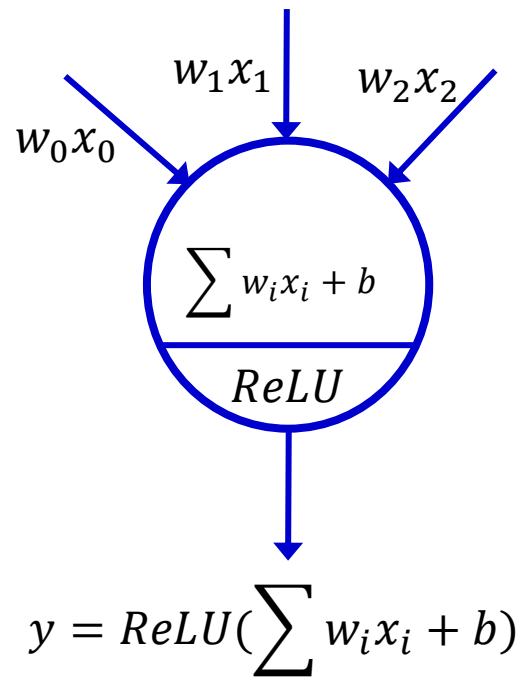
- More flexible pruning index choice
 - Higher Accuracy
- Hard to accelerate (irregular data expression)

Coarse-grained/Structured

- Less flexible pruning index choice
- Easy to accelerate (just a smaller matrix!)

Parameter Selection

- When removing parameters from a neural network model,
 - the less important the parameters being removed are,
 - the better the performance of pruned neural network is

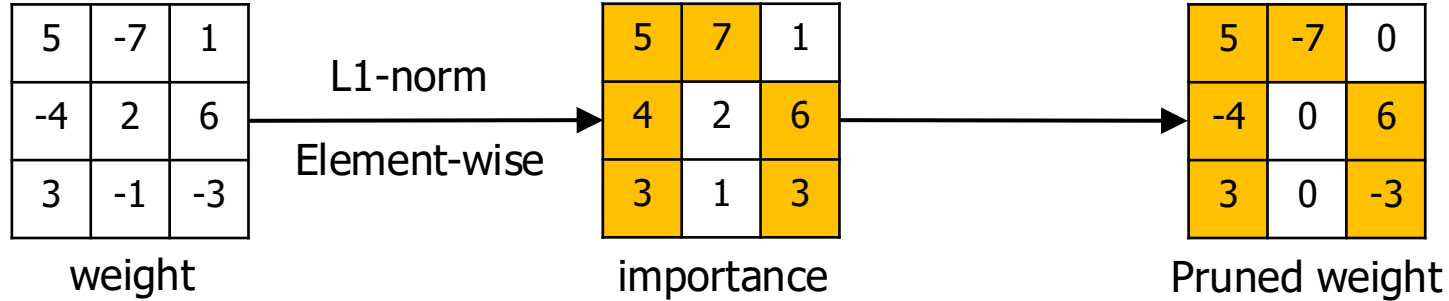


If $W = [15, 0.2, -9]$, $y = ReLU(15x_0 + 0.2x_1 - 9x_2)$

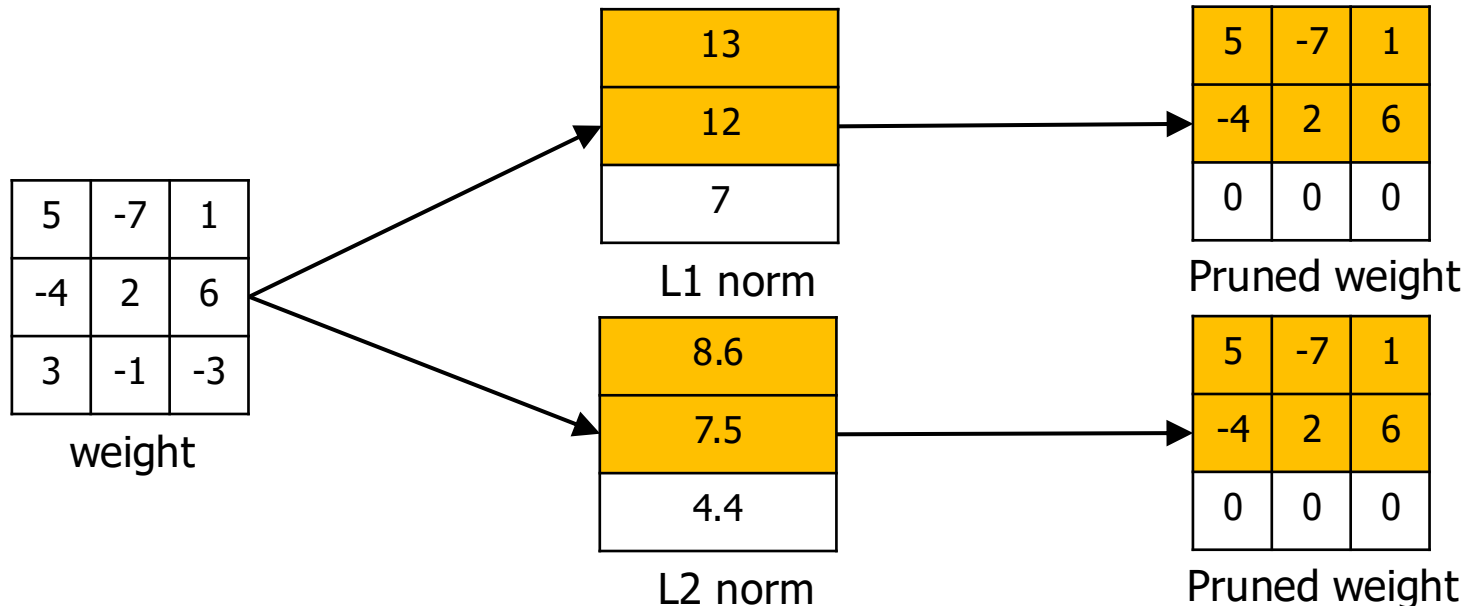
If you want remove one weight parameter, which one?
To remove a weight parameter is to change it to zero.

Magnitude-based Weight Pruning

- Element-wise pruning: $\text{importance}(W_{i,j}) = |W_{i,j}|$



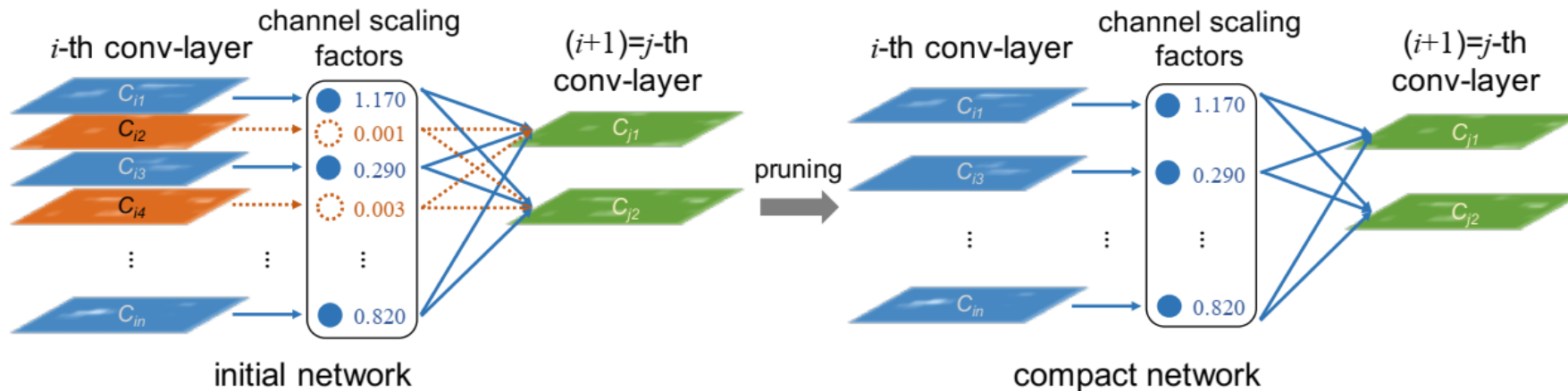
- Row vector pruning and L1/L2-norm magnitude



$$\|x\|_2 = \sqrt{\sum_{i=1}^n |x_i|^2}$$

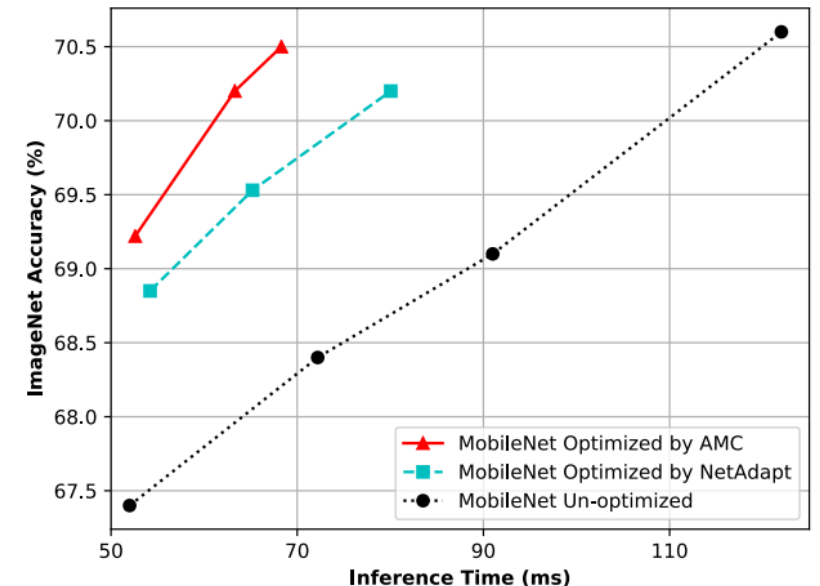
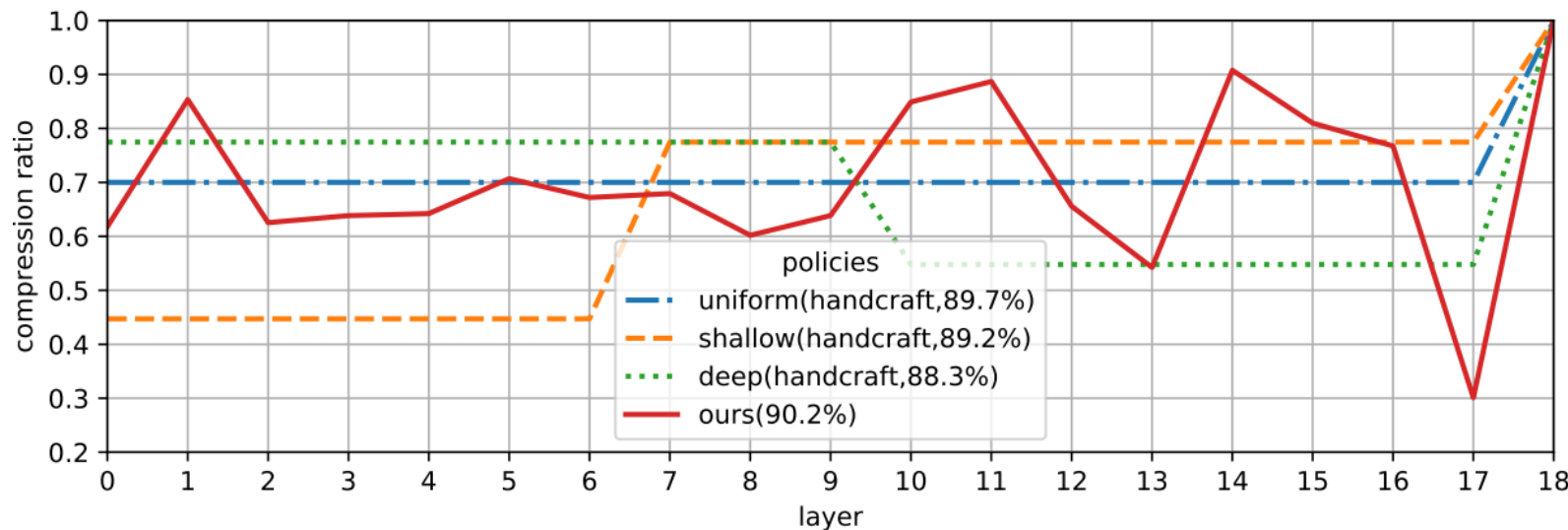
Scaling-based Channel Pruning

- BN transformation $\hat{z} = \frac{z_{in} - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$; $z_{out} = \gamma \hat{z} + \beta$
- γ and β are trainable affine transformation parameters (scale and shift).
- Add an ℓ_1 -norm regularization onto the scaling factors of BN layers.
- During training, γ regularization as $L = \sum_{(x,y)} l(f(x, W), y) + \lambda \sum_{\gamma \in \Gamma} g(\gamma)$ $g(s) = |s|$ (L1-norm)
- After training, the output FMs with small γ (resulting small z_{out}) can be pruned, as well as the connected weights.



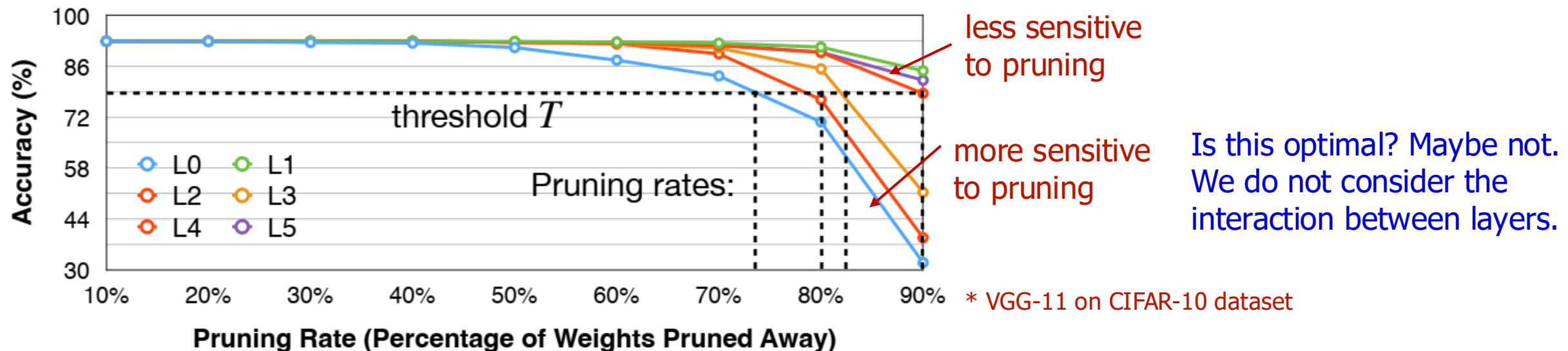
Layer-wise Channel Pruning

- Different pruning ratios for each layer since different layers have different sensitivity
 - Some layers are more sensitive (e.g., first layer), some layers are more redundant
- Need to analyze the sensitivity of each layer



Finding Pruning Ratios

- Sensitivity Analysis
 - Pick a layer L_i in the model and Prune with pruning ratio $r \in \{0, 0.1, 0.2, \dots, 0.9\}$
 - Observe the accuracy degrade $\Delta \text{acc}_{i,r}$ for each pruning ratio
 - Repeat the process for all layers
 - Pick a degradation threshold T such that the overall pruning rate is desired



CSR Format for Sparse Matrices

		A	B		
C	D				
		E		F	
			G		
	H	I			J
K			L	M	

0	2	4	6	7	10	13
---	---	---	---	---	----	----

Row Pointer

2	3	0	1	2	4	3	...
---	---	---	---	---	---	---	-----

Column Indices

A	B	C	D	E	F	G	...
---	---	---	---	---	---	---	-----

Values

CSR Format for Sparse Matrices

		A	B		
C	D				
		E		F	
			G		
	H	I			J
K			L	M	

Because there is no element before the current row.

0	2	4	6	7	10	13
---	---	---	---	---	----	----

Row Pointer

2	3	0	1	2	4	3	...
---	---	---	---	---	---	---	-----

$A[0].\text{nonzeros} = [2, 3]$ Column Indices

A	B	C	D	E	F	G	...
---	---	---	---	---	---	---	-----

Nonzero values for row 0

Values

CSR Format for Sparse Matrices

Because there is two element before the current row.

		A	B		
C	D				
		E		F	
			G		
	H	I			J
K			L	M	

0	2	4	6	7	10	13
---	---	---	---	---	----	----

Row Pointer

2	3	0	1	2	4	3	...
---	---	---	---	---	---	---	-----

$A[1].\text{nonzeros} = [0, 1]$

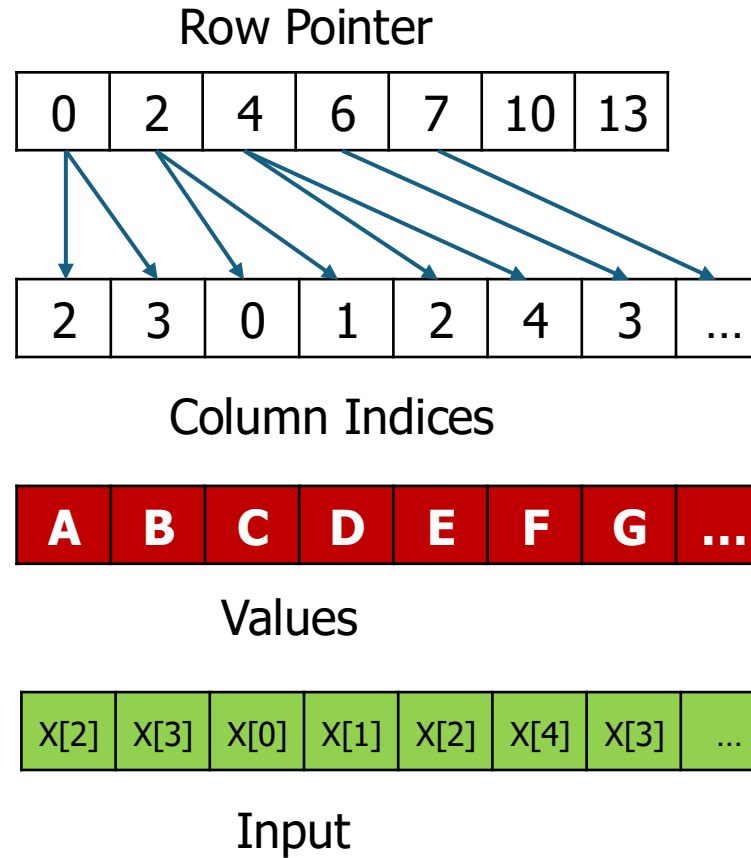
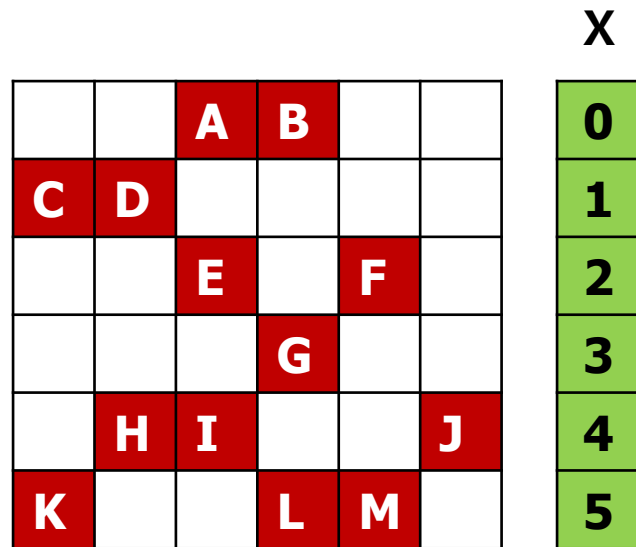
Column Indices

A	B	C	D	E	F	G	...
---	---	---	---	---	---	---	-----

Nonzero values for row 1

Values

CSR Format for Sparse Matrices

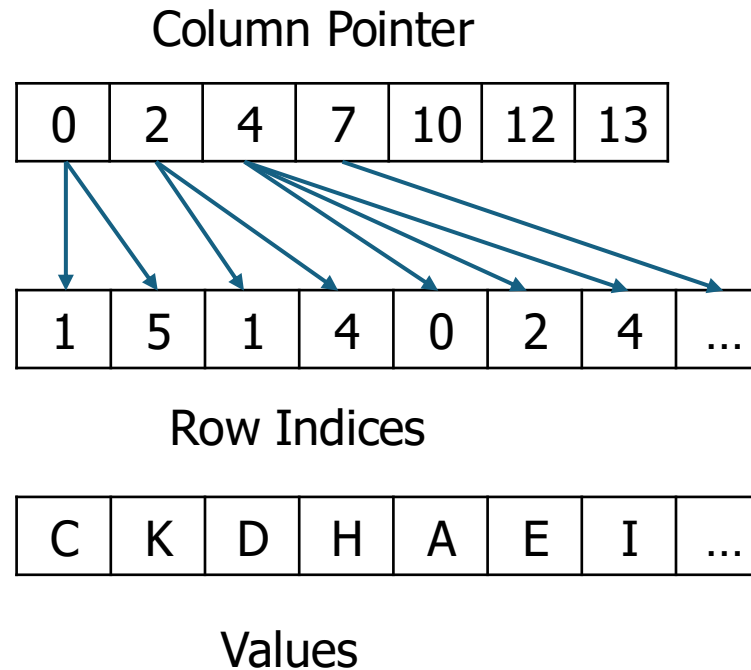


Algorithm 1: Standard CSR SpMV

```
1 for i = 0 to nRows-1 do // allow parallelization
2   sum = 0
3   for j = row_pointers[i] to row_pointers[i+1]
4     do
5       sum += values[j] * x[col_indices[j]]
6   y[i] = sum
```

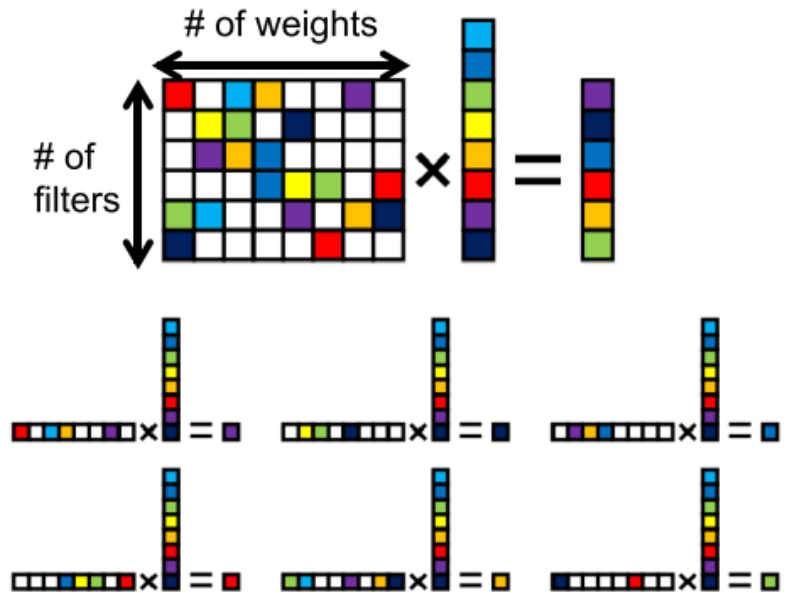
CSC Format for Sparse Matrices

		A	B		
C	D				
		E		F	
			G		
	H	I			J
K			L	M	

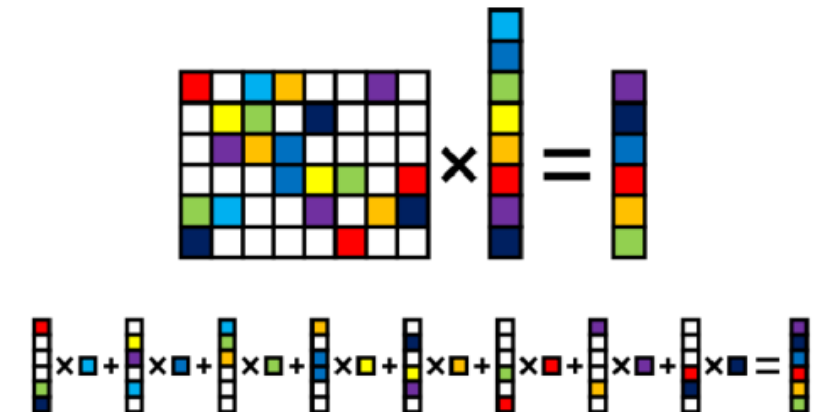


Sparse matrix compressed format

- Compressed sparse row (CSR)
 - Input vector needs to be read in multiple times
 - Only a subset of input vector is used since each row of the matrix is sparse.
- Compressed sparse column (CSC)
 - Output is updated several times
 - Only one element of input vector is read at a time



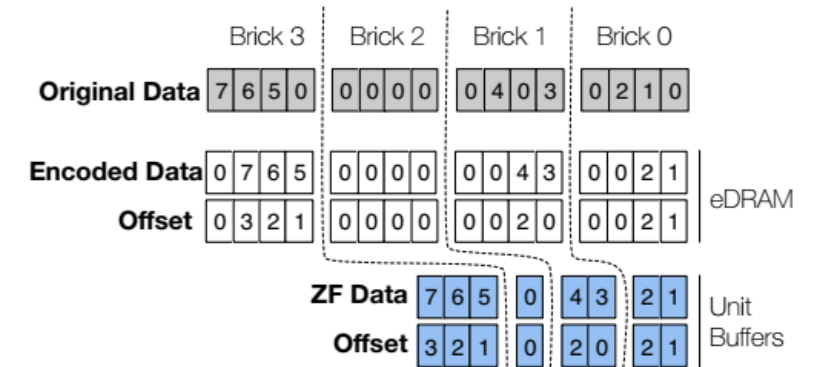
(a) CSR



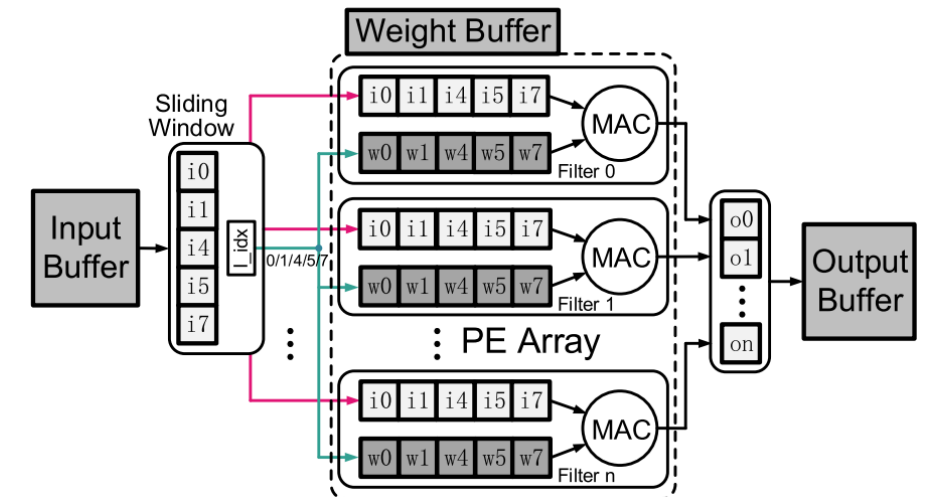
CSC

Sparse Architecture – Input Sparsity

- ReLU produces abundant zeros
- Simple run-length activation compression
- Zero input detection
 - can be aggressively extended to small inputs
- PE data gating
- Structured activation sparsity such as FM-wise pruning can be used (e.g., Scalpel)



Zero-Free Neuron Array format (ZFNAf)
[Cnvlutin: Ineffectual-Neuron-Free Deep Neural Network Computing \[ISCA'16\]](#)



Sparse Architecture – Irregular Weight Sparsity

- Leverage weight sparsity by naively skipping the MACs with zero weights.
- Weights are stored in a sparse format with addressing indexes.
- Each PE accesses the required activations according to the weight index and then performs the residual MACs.
- Reduced OPs, low latency and energy.
- The distribution of nonzero weights is usually very irregular
 - large indexing overhead, PE imbalance, and memory access inefficiency

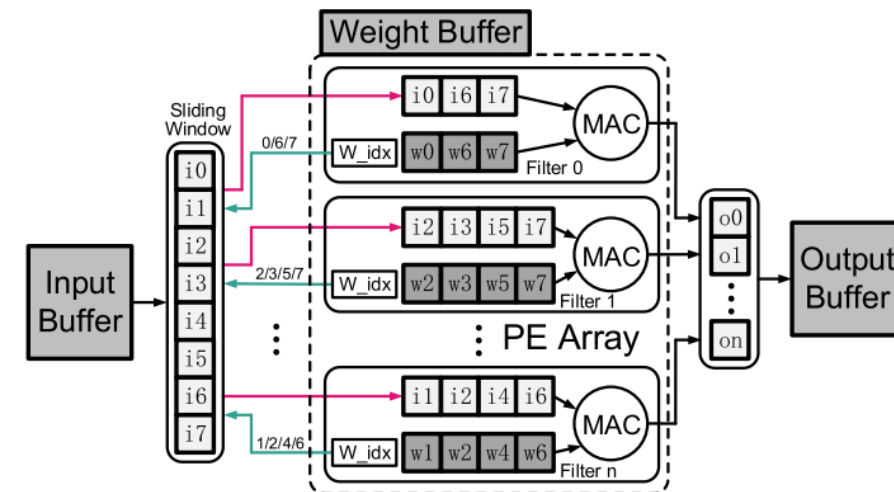
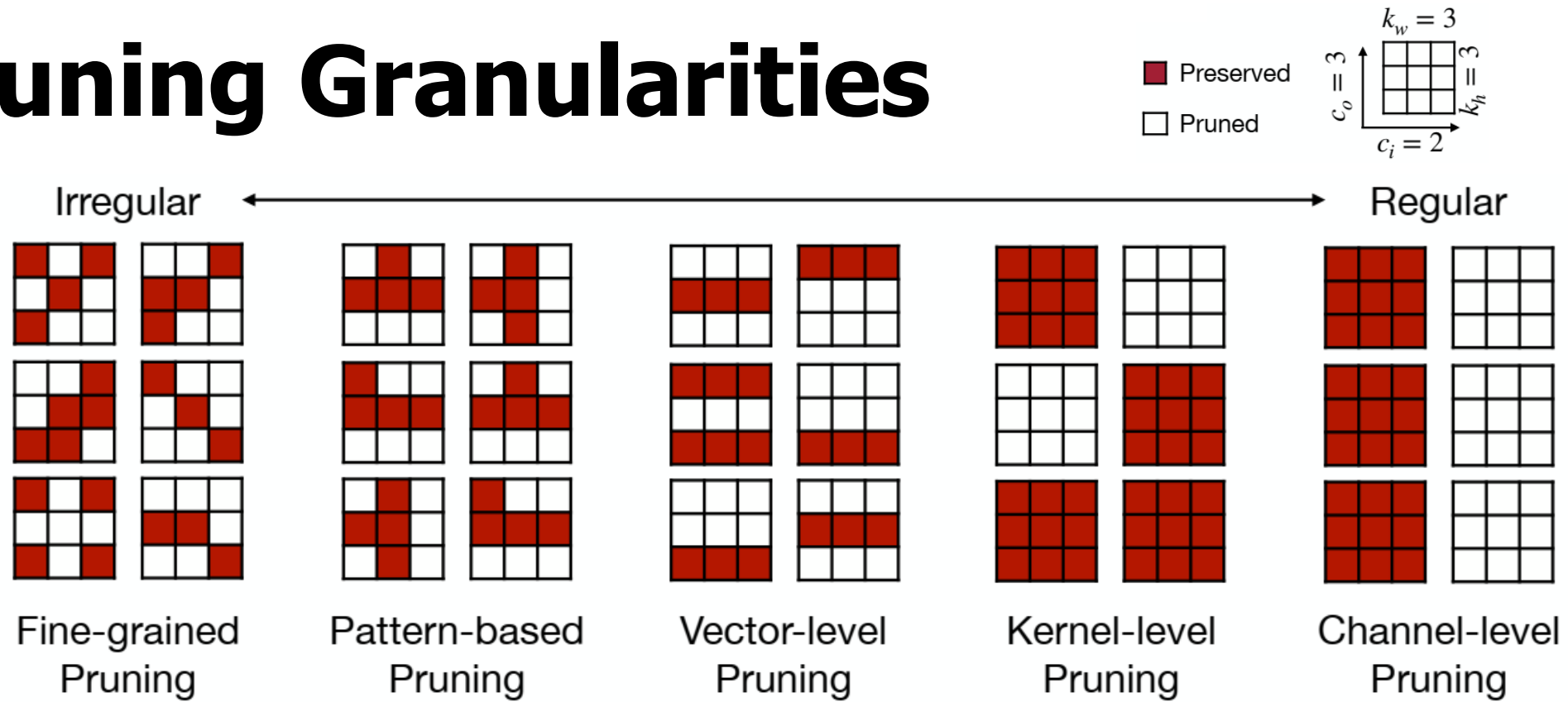


Image from "Model Compression and Hardware Acceleration for Neural Networks: A Comprehensive Survey [Deng et al. 2020]

Pruning Granularities



Fine-grained/Unstructured

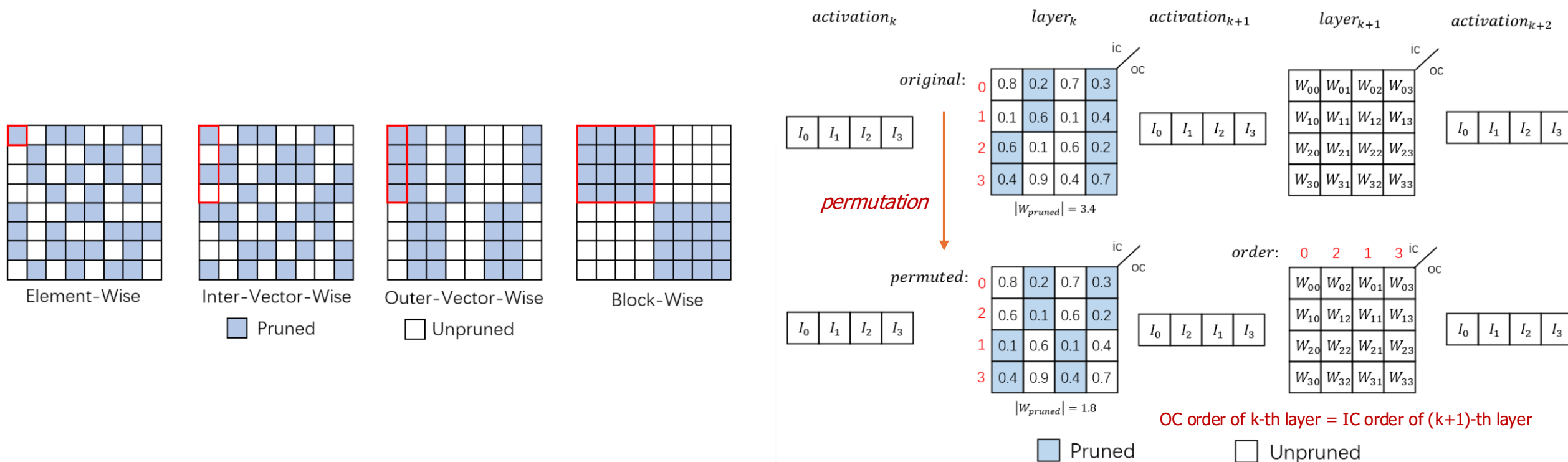
- More flexible pruning index choice
 - Higher Accuracy
- Hard to accelerate (irregular data expression)

Coarse-grained/Structured

- Less flexible pruning index choice
- Easy to accelerate (just a smaller matrix!)

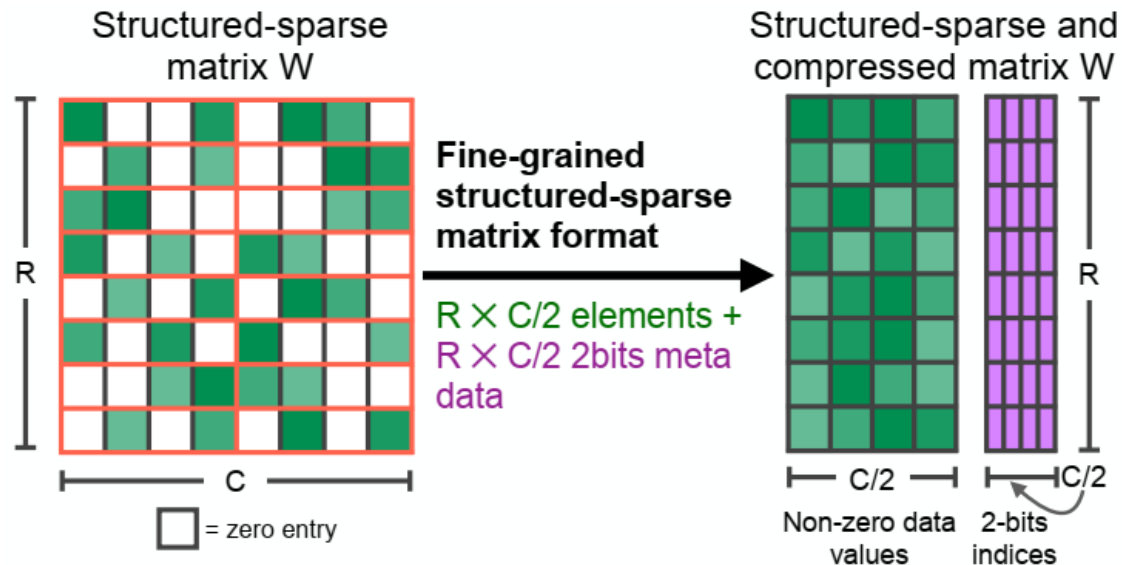
Vector-Wise Sparsity

- IVW prunes a certain proportion of weights inside each vector
- OVW prunes the entire vector of weights
- **Channel Permutation** (Reordering) will be beneficial



N:M sparsity

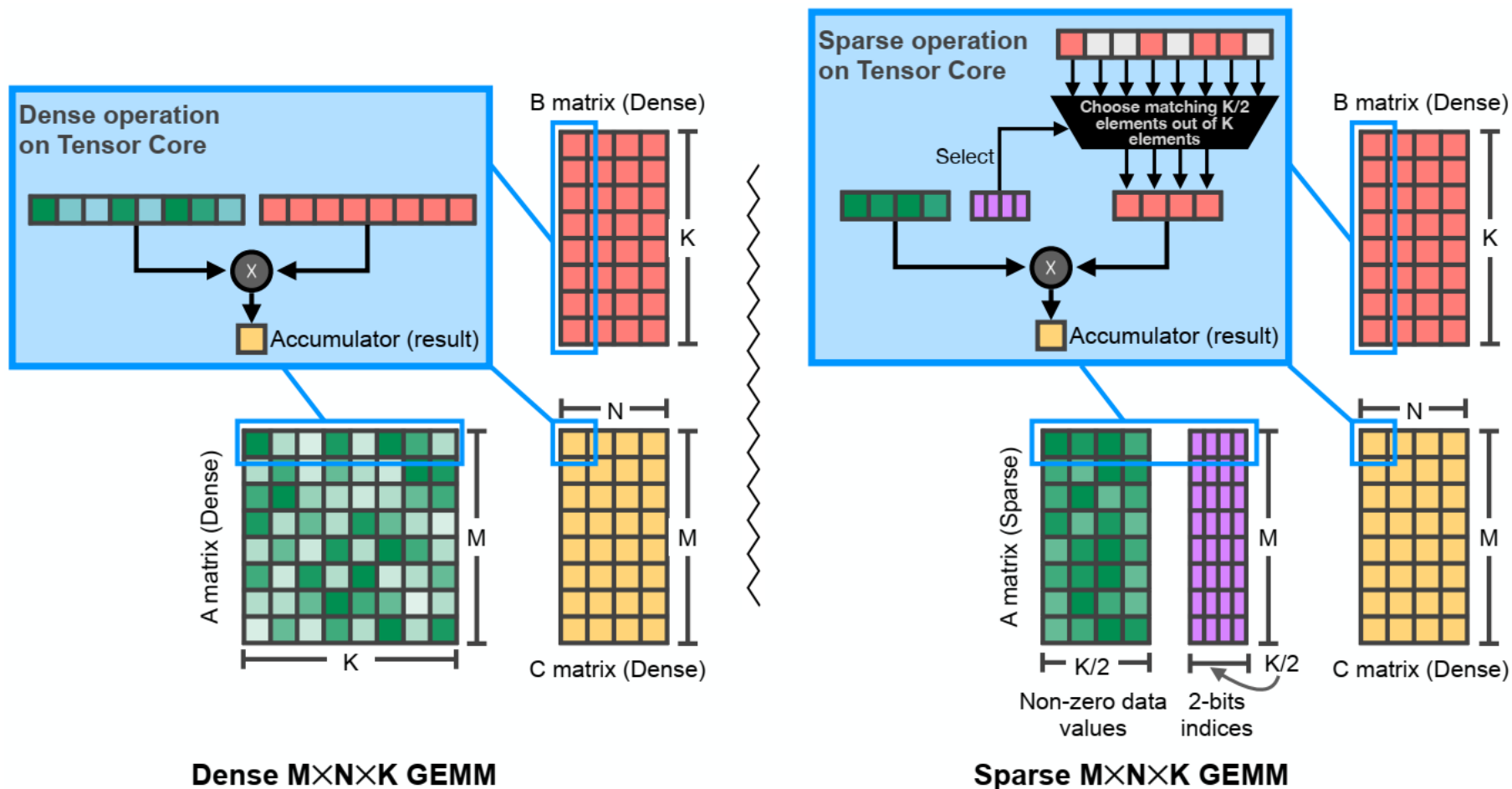
- In each contiguous M elements, N of them is pruned
- A classic case is 2:4 sparsity (50% sparsity)
- Supported by NVIDIA's Ampere GPU Architecture (Tensor Core)
- Usually maintains accuracy



Network	Data Set	Metric	Dense FP16	Sparse FP16
ResNet-50	ImageNet	Top-1	76.1	76.2
ResNeXt-101_32x8d	ImageNet	Top-1	79.3	79.3
Xception	ImageNet	Top-1	79.2	79.2
SSD-RN50	COCO2017	bbAP	24.8	24.8
MaskRCNN-RN50	COCO2017	bbAP	37.9	37.9
FairSeq Transformer	EN-DE WMT'14	BLEU	28.2	28.5
BERT-Large	SQuAD v1.1	F1	91.9	91.9

N:M sparsity

- Mapping M:N sparse matrices onto NVIDIA tensor cores



LLM Pruning

- The enormous size of LLMs (100+ Billion Parameters) requires special considerations
- Magnitude Pruning preserves accuracy only until 10% sparsity.
- Retraining to recover model performance after pruning is prohibitively expensive.

Taylor Expansion Analysis on Pruning Error

- Evaluate pruning error induced by pruning synapses
 - The task of training neural network is to minimize the objective function.
 - The importance of a parameter can be quantified by the error induced by removing it (saliency 현저함).
 - The induced error (perturbation) can be approximated by a Taylor series

$$\delta L = L(\mathbf{x}; \mathbf{W}) - L(\mathbf{x}; \mathbf{W}_P = \mathbf{W} - \delta \mathbf{W}) = \sum_i g_i \delta w_i + \frac{1}{2} \sum_{i,j} h_{ij} \delta w_i \delta w_j + O(\|\delta \mathbf{W}\|^3)$$

$$g_i = \frac{\partial L}{\partial w_i} \quad h_{ij} = \frac{\partial^2 L}{\partial w_i \partial w_j}$$

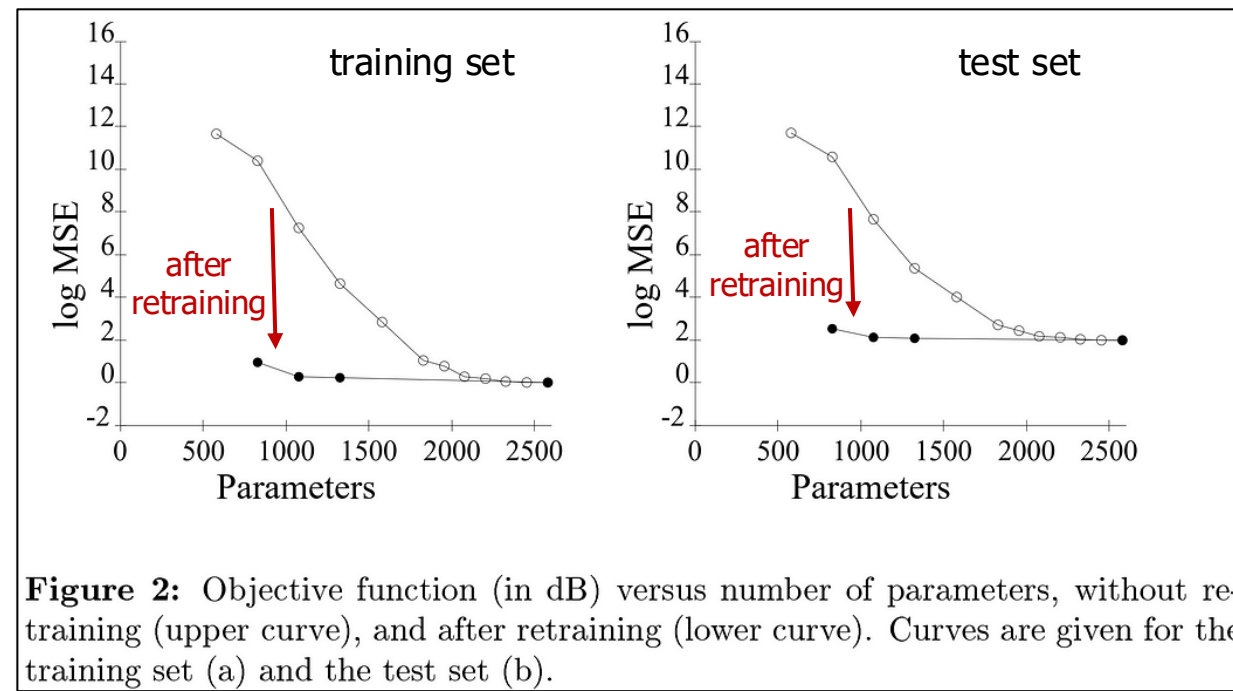
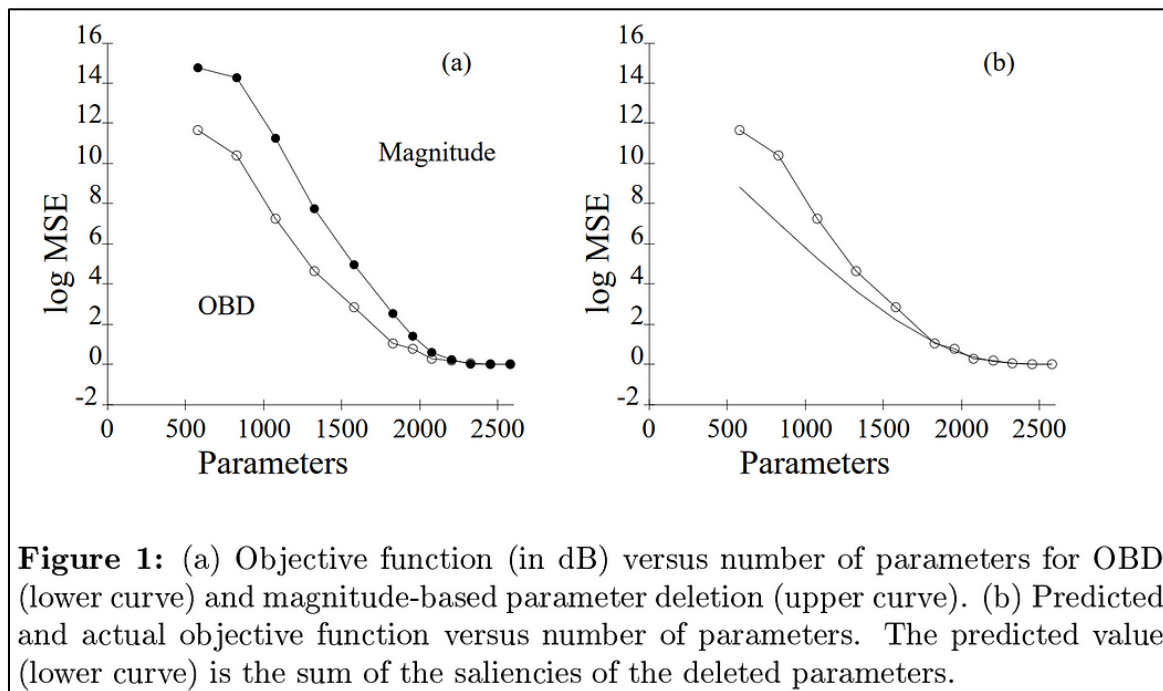
OBD: Second-Order-based Pruning

$$\begin{aligned}\delta L &= \sum_i g_i \delta w_i + \frac{1}{2} \sum_{i,j} h_{ij} \delta w_i \delta w_j + O(\|\delta \mathbf{W}\|^3) \\ &= \sum_i g_i \delta w_i + \frac{1}{2} \sum_i h_{ii} \delta w_i^2 + \frac{1}{2} \sum_{i \neq j} h_{ij} \delta w_i \delta w_j + O(\|\delta \mathbf{W}\|^3)\end{aligned}$$

- **Optimal Brain Damage** assumes that
 - The neural network training has converged
 - first-order terms are neglected ($g_i = 0$)
 - The error caused by deleting each parameter is **independent**
 - cross terms are neglected ($h_{ij} = 0$)
 - The objective function is nearly quadratic: the last term is neglected
- The synapses with smaller induced error $|\delta L_i|$ will be removed

$$\delta L_i = L(\mathbf{x}; \mathbf{W}) - L(\mathbf{x}; \mathbf{W}_P | w_i = 0) \approx \frac{1}{2} h_{ii} w_i^2 \quad \text{Hessian } h_{ii} : \text{curvature}$$

OBD: Second-Order-based Pruning



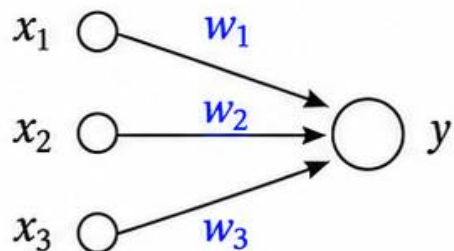
Optimal Brain Surgeon (OBS)

- Consider the **cross-terms** within the Hessian matrix (remove the diagonality assumption of OBD).
- Determine optimal updates to the remaining weights based on the removal of a given parameter
- Simultaneously pruning and optimizing the model, **no need for retraining**.
- Optimal weight change and resulting change in error:
 - $\delta w = -\frac{w_q}{[H^{-1}]_{qq}} \cdot H_{:,q}^{-1}$ and $L_q = \frac{1}{2} \frac{w_q^2}{[H^{-1}]_{qq}}$ $[H^{-1}]_{qq}$: the q th diagonal entry of the inverse Hessian, and $H_{:,q}^{-1}$ is its q th column.
- repeat 
 - Compute H^{-1}
 - Find the q that gives the smallest L_q
 - Update all weights δw
- Many follow-up methods for Hessian approximation (empirical Fisher)
 - WoodFisher (NeurIPS'20), M-FAC (NeurIPS'21), oBERT (EMNLP'22)

Optimal Brain Surgeon (OBS): Example

A pruning method that removes weights while minimizing the increase in loss using a second-order approximation

1. Trained Weights

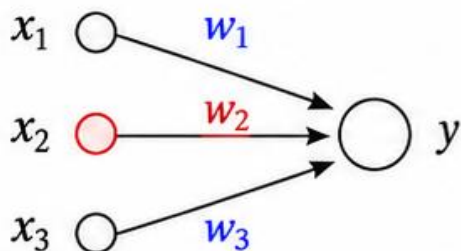


$$w = [0.8, -0.3, 0.5]$$

$$H^{-1} = \begin{bmatrix} 2.0 & 0.4 & -0.2 \\ 0.4 & 1.0 & 0.1 \\ -0.2 & 0.1 & 1.5 \end{bmatrix}$$

H^{-1} : inverse Hessian,
captures weight
interdependence

2. Choose the Weight to Remove



$$\Delta E_q = \frac{w_q^2}{2 [H^{-1}]_{qq}}$$

Example: $q = 2$ (remove w_2)

$$\Delta E_2 = \frac{(-0.3)^2}{2 \times 1.0} = 0.045$$

Choose the weight that
causes the smallest
increase in loss

3. Adjust the Remaining Weights

$$\Delta w = -\frac{w_q}{[H^{-1}]_{qq}} H^{-1} e_q$$

For $q = 2$ (removing w_2)

$$\Delta w = -\frac{-0.3}{1.0} \begin{bmatrix} 0.4 \\ 1.0 \\ 0.1 \end{bmatrix} = \begin{bmatrix} 0.12 \\ 0.30 \\ 0.03 \end{bmatrix}$$

Force w_2 to 0,
and adjust the other weights by Δw

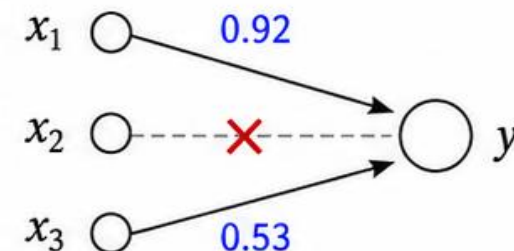
Before $[0.8, -0.3, 0.5]$

$+0.12 \quad \rightarrow 0 \quad +0.03$

After correction $[0.92, 0, 0.53]$

Update w_1 and w_3

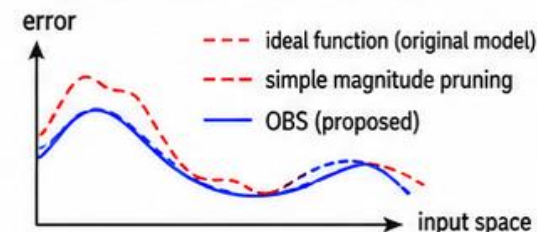
4. Model After Pruning



$$w = [0.92, 0, 0.53]$$

Minimize output change and loss increase

Change in output function
(difference from the ideal function)

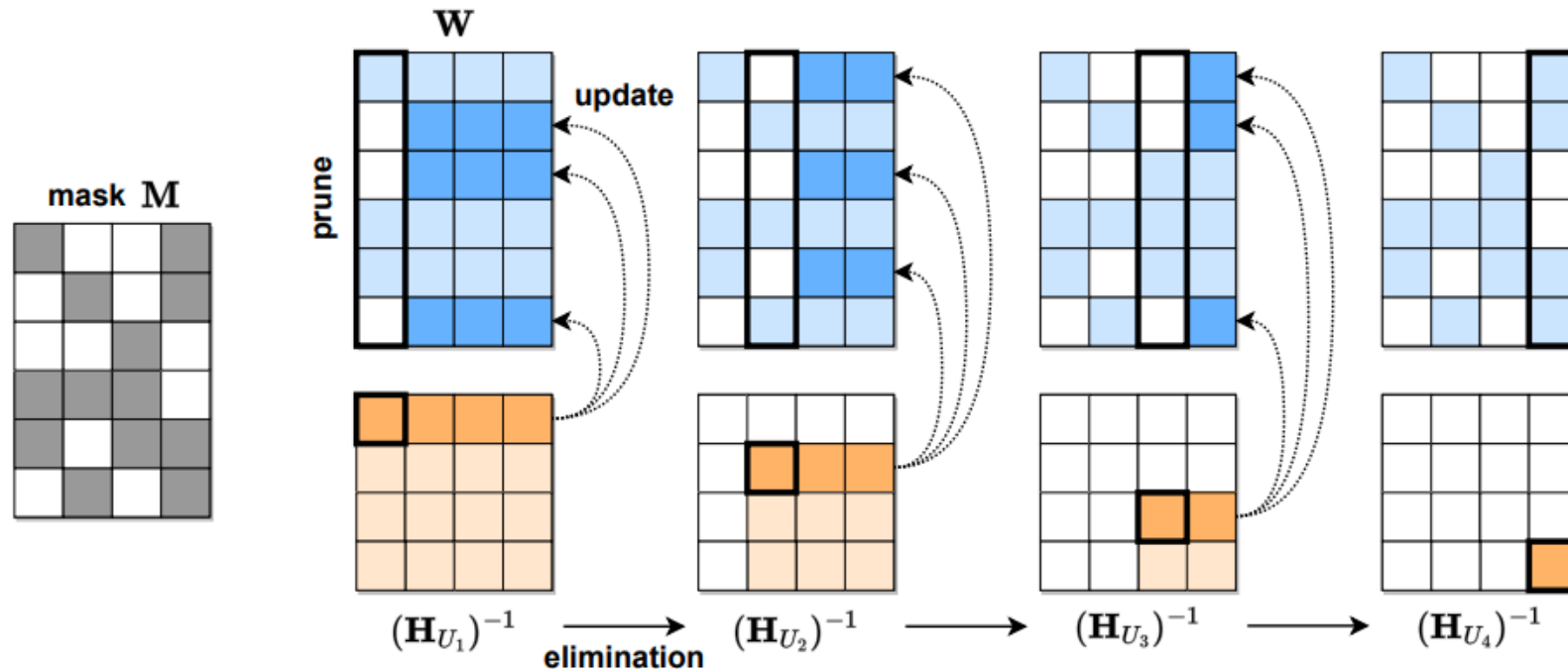


Simple magnitude pruning
Larger function change
(larger loss increase)

OBS
Closer to the original
function (smaller loss
increase)

SparseGPT

- OBS takes hundreds of hours for 100+ billion parameters.
- Need an approximated and efficient algorithm (GTPQ)
 - **Layer-wise** approach
 - Prune column by column, update only right-side, freeze left
 - **Mask selection and weight reconstruction are separated**



SparseGPT

Initial Weight

0.8	-0.3	0.5	0.1
-0.2	0.4	-0.6	0.3
0.7	-0.1	0.2	-0.5
0.1	0.6	-0.4	0.9

Stage 1

0.8	-0.3	0.5	0.1
0	0.44	-0.58	0.30
0.7	-0.1	0.2	-0.5
0	0.58	-0.4	0.9

Stage 2

0.8	0	0.53	0.14
0	0.44	-0.58	0.30
0.7	0	0.21	-0.49
0	0.58	-0.4	0.9

Stage 3

0.8	0	0.53	0.14
0	0.44	-0.58	0.30
0.7	0	0	-0.52
0	0.58	0	0.97

Pruning mask

1	0	1	0
0	1	1	0
1	0	0	1
0	1	0	1

$$e_2 = \frac{W_{2,1}}{(H_{U_1}^{-1})_{1,1}} = \frac{-0.2}{1.0} = -0.2$$

$$W_{2,j} \leftarrow W_{2,j} - e_2(H_{U_1}^{-1})_{1,j}$$

$$W_{2,2} \leftarrow 0.4 - (-0.2)(0.2) = 0.44$$

$$e_4 = \frac{W_{4,1}}{(H_{U_1}^{-1})_{1,1}} = \frac{0.1}{1.0} = 0.1$$

$$W_{4,2} \leftarrow 0.6 - (0.1)(0.2) = 0.58$$

$$e_1 = \frac{W_{1,2}}{(H_{U_2}^{-1})_{1,1}} = \frac{-0.3}{0.76} = -0.395$$

$$W_{1,j} \leftarrow W_{1,j} - e_1(H_{U_2}^{-1})_{1,j}$$

$$W_{1,3} \leftarrow 0.5 - (-0.395)(0.08) = 0.53$$

$$e_3 = \frac{W_{3,2}}{(H_{U_2}^{-1})_{1,1}} = \frac{-0.1}{0.76} = -0.13$$

$$W_{3,3} \leftarrow 0.2 - (-0.13)(0.08) = 0.21$$

Stage 4

0.8	0	0.53	0
0	0.44	-0.58	0
0.7	0	0	-0.52
0	0.58	0	0.97

$H_{U_1}^{-1}$

1.0	0.2	0.1	0
0.2	0.8	0.1	0.1
0.1	0.1	1.2	0.2
0	0.1	0.2	0.9

$$H_{U_1}^{-1} = \begin{bmatrix} d & h^T \\ h & B \end{bmatrix}$$

$$H_{U_2}^{-1} = B - \frac{hh^T}{d} = \begin{bmatrix} 0.8 & 0.1 & 0.1 \\ 0.1 & 1.2 & 0.2 \\ 0.1 & 0.2 & 0.9 \end{bmatrix} - \begin{bmatrix} 0.04 & 0.02 & 0 \\ 0.02 & 0.01 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

$H_{U_2}^{-1}$

0.76	0.08	0.10
0.08	1.19	0.20
0.10	0.20	0.90

$H_{U_3}^{-1}$

1.18	0.19
0.19	0.89

$H_{U_4}^{-1}$

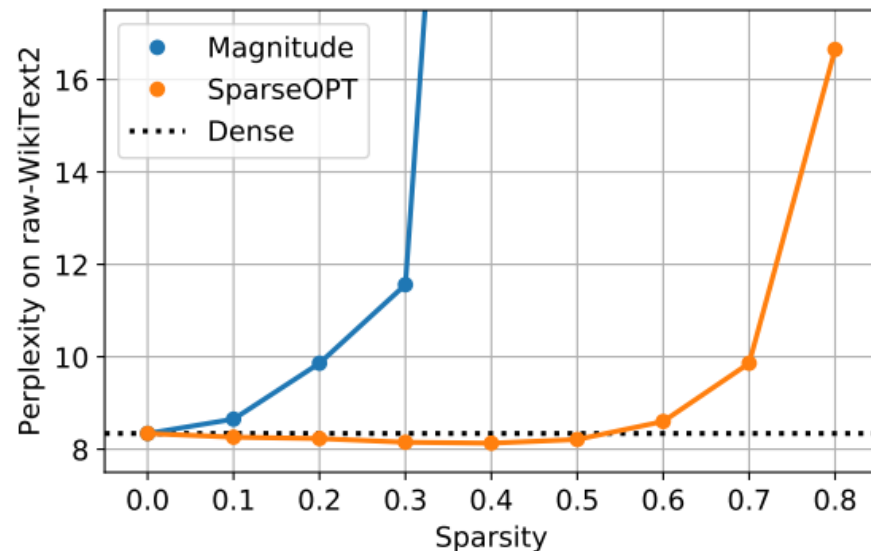
0.86

SparseGPT

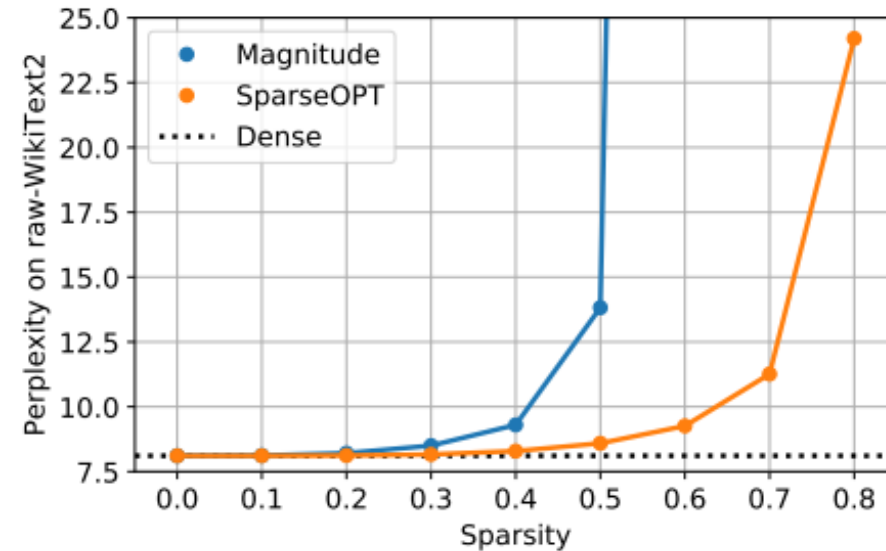
- Achieve significantly higher accuracy compared to the Magnitude Pruning.
- OPT-175B and BLOOM-176B can reach 60% unstructured sparsity with negligible increase in **perplexity** in under 4.5 hours.

<https://www.lakera.ai/blog/large-language-model-evaluation>

OPT-175B Uniform Unstructured Sparsity

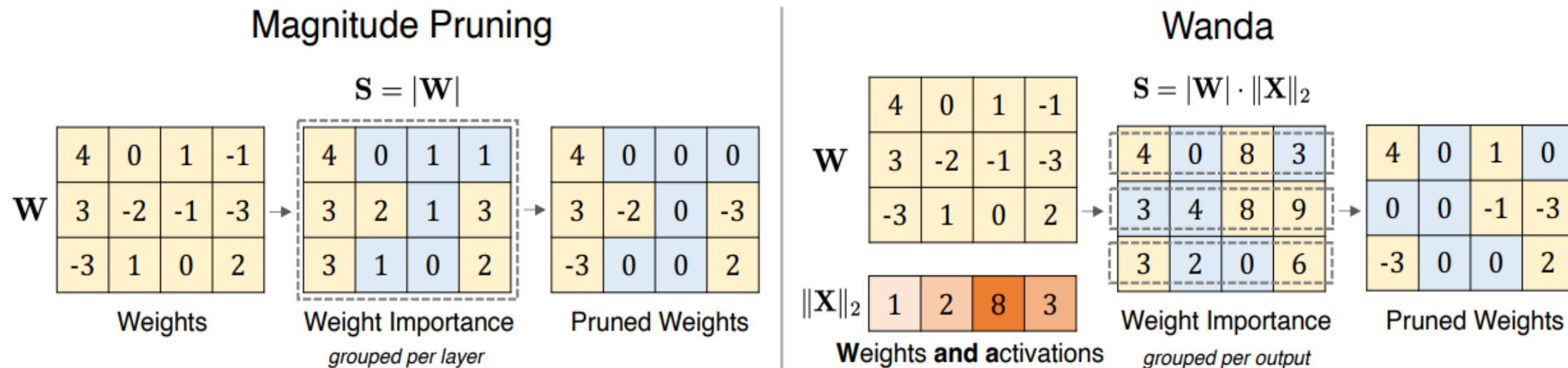


BLOOM-176B Uniform Unstructured Sparsity



Wanda

- Simple magnitude-based unstructured pruning
- Multiply the weight magnitudes by the norm of their associated input activations



Wanda

- Exceed the results of SparseGPT while reducing complexity

Method	Weight Update	Calibration Data	Pruning Metric S_{ij}	Complexity
Magnitude	✗	✗	$ \mathbf{W}_{ij} $	$O(1)$
SparseGPT	✓	✓	$[\mathbf{W} ^2 / \text{diag}[(\mathbf{X}\mathbf{X}^T + \lambda\mathbf{I})^{-1}]]_{ij}$	$O(d_{\text{hidden}}^3)$
Wanda	✗	✓	$ \mathbf{W}_{ij} \cdot \ \mathbf{X}_j\ _2$	$O(d_{\text{hidden}}^2)$

Zero-shot Accuracy			LLaMA				LLaMA-2		
Method	Weight Update	Sparsity	7B	13B	30B	65B	7B	13B	70B
Dense	-	0%	59.99	62.59	65.38	66.97	59.71	63.03	67.08
Magnitude	✗	50%	46.94	47.61	53.83	62.74	51.14	52.85	60.93
SparseGPT	✓	50%	54.94	58.61	63.09	66.30	56.24	60.72	67.28
Wanda	✗	50%	54.21	59.33	63.60	66.67	56.24	60.83	67.03

Knowledge Distillation

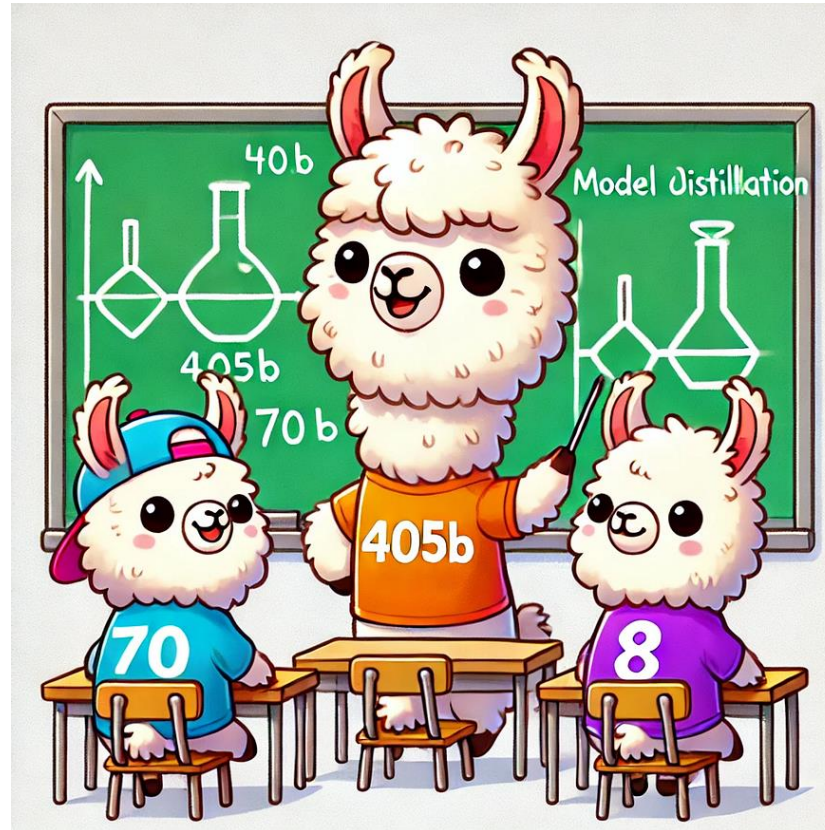
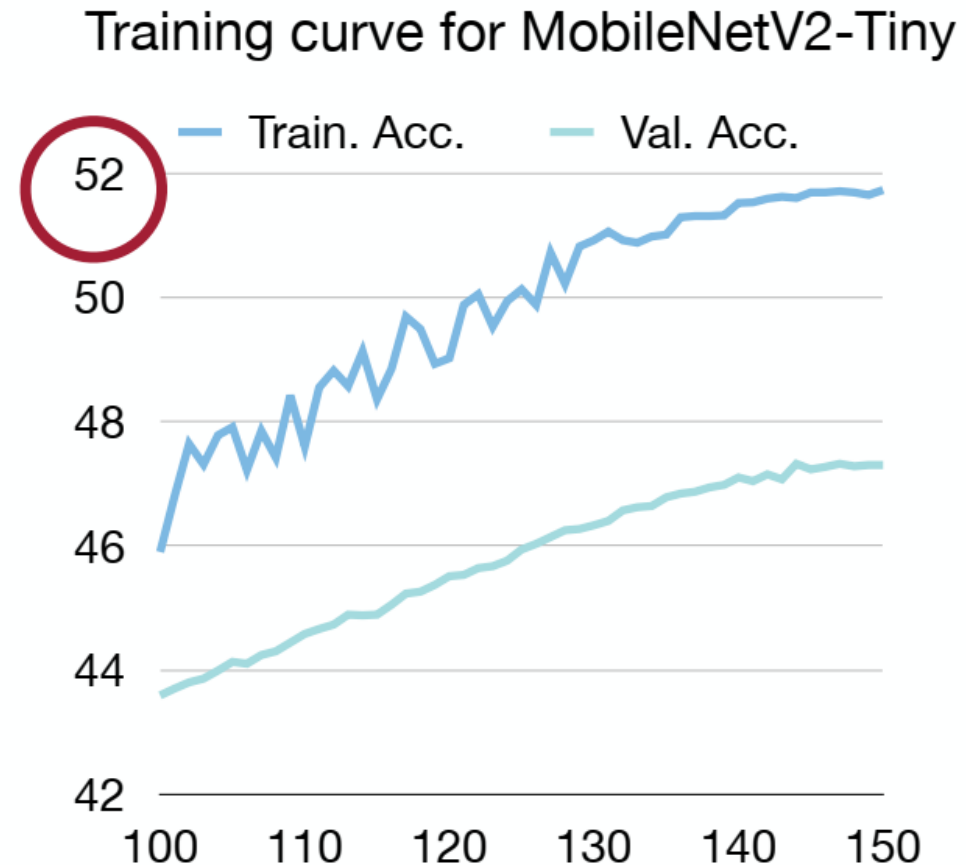
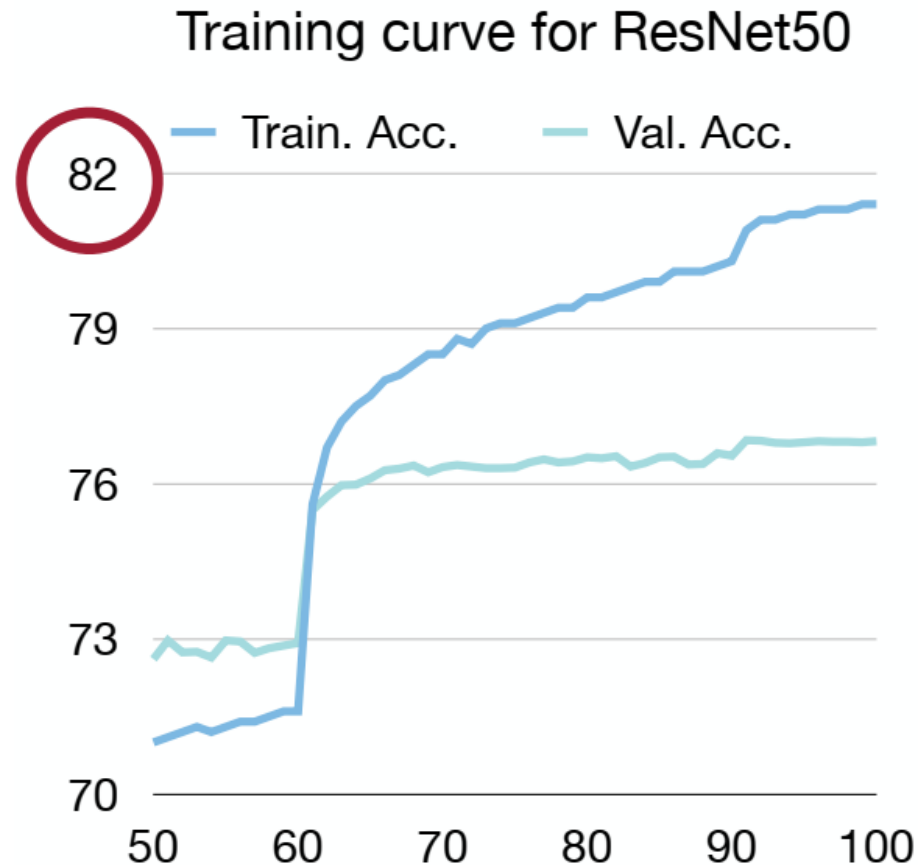


Image from <https://pub.towardsai.net/smaller-faster-smarter-the-power-of-model-distillation-d002662308c7>

Tiny models are hard to train

- Tiny models underfit large datasets



Question: Can we help the training of tiny models with large models?

Knowledge Distillation

- Using a **deep network** or averaging the predictions of different models (ensemble) gives a better accuracy than using a single **shallower network**.
- knowledge distillation transfers the knowledge learned by the complex model (teacher) to the simpler model (student).
- Student network can achieve an accuracy that would be unachievable if it was directly trained with the same data set.

Distilling the Knowledge in a Neural Network

Geoffrey Hinton*[†]
Google Inc.
Mountain View
geoffhinton@google.com

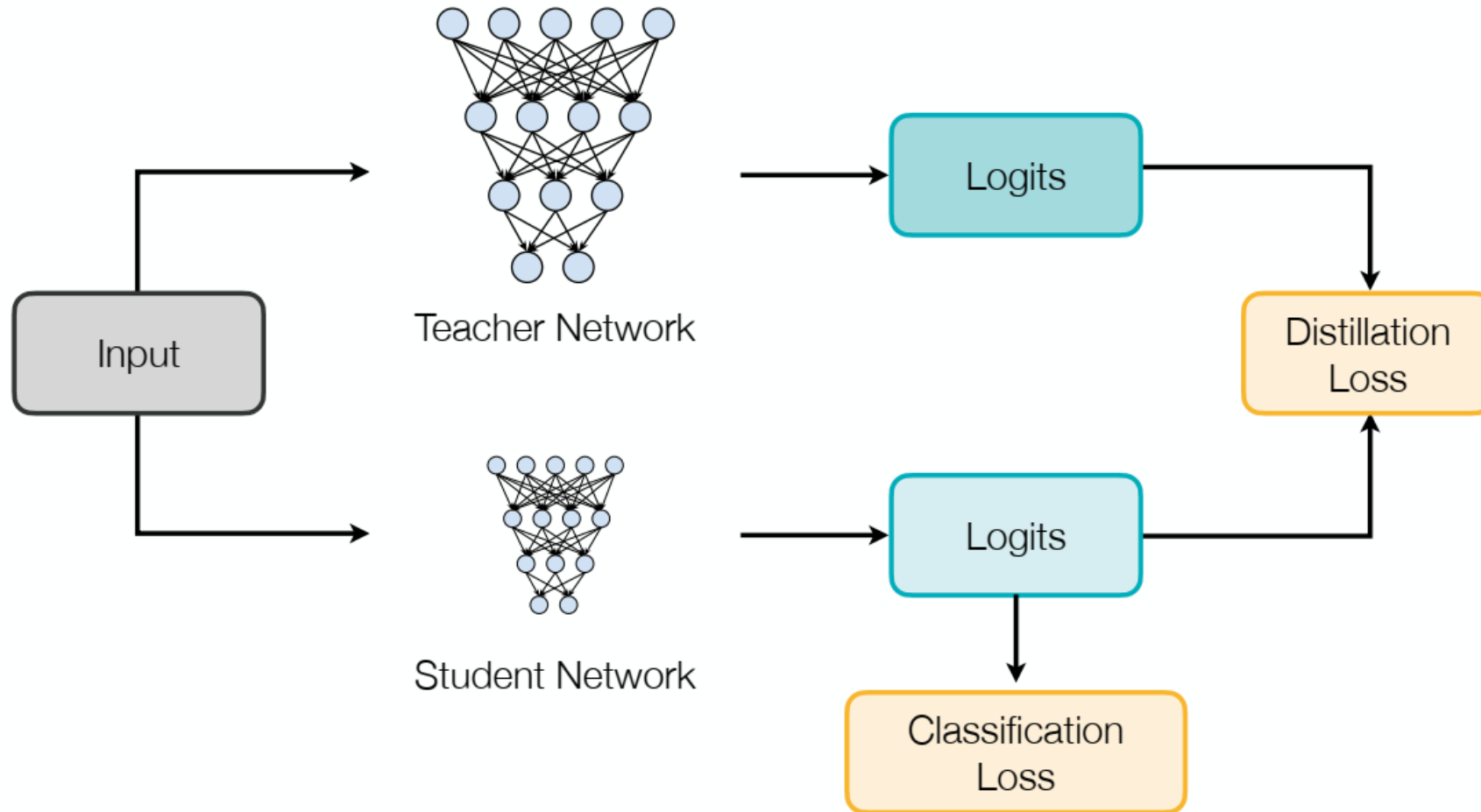
Oriol Vinyals[†]
Google Inc.
Mountain View
vinyals@google.com

Jeff Dean
Google Inc.
Mountain View
jeff@google.com

Abstract

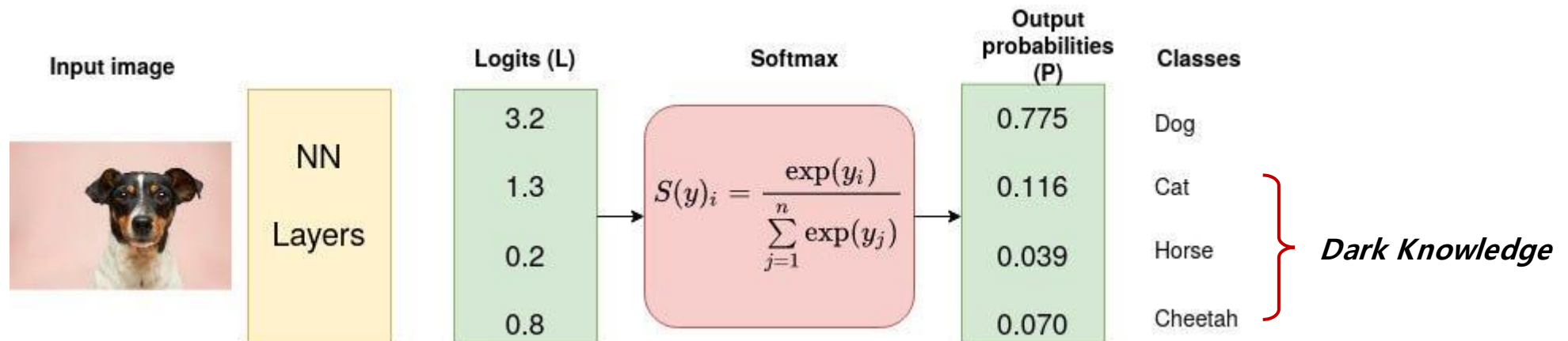
A very simple way to improve the performance of almost any machine learning algorithm is to train many different models on the same data and then to average their predictions [3]. Unfortunately, making predictions using a whole ensemble of models is cumbersome and may be too computationally expensive to allow deployment to a large number of users, especially if the individual models are large neural nets. Caruana and his collaborators [1] have shown that it is possible to compress the knowledge in an ensemble into a single model which is much easier to deploy and we develop this approach further using a different compression technique. We achieve some surprising results on MNIST and we show that we can significantly improve the acoustic model of a heavily used commercial system by distilling the knowledge in an ensemble of models into a single model. We also introduce a new type of ensemble composed of one or more full models and many specialist models which learn to distinguish fine-grained classes that the full models confuse. Unlike a mixture of experts, these specialist models can be trained rapidly and in parallel.

Illustration of knowledge distillation



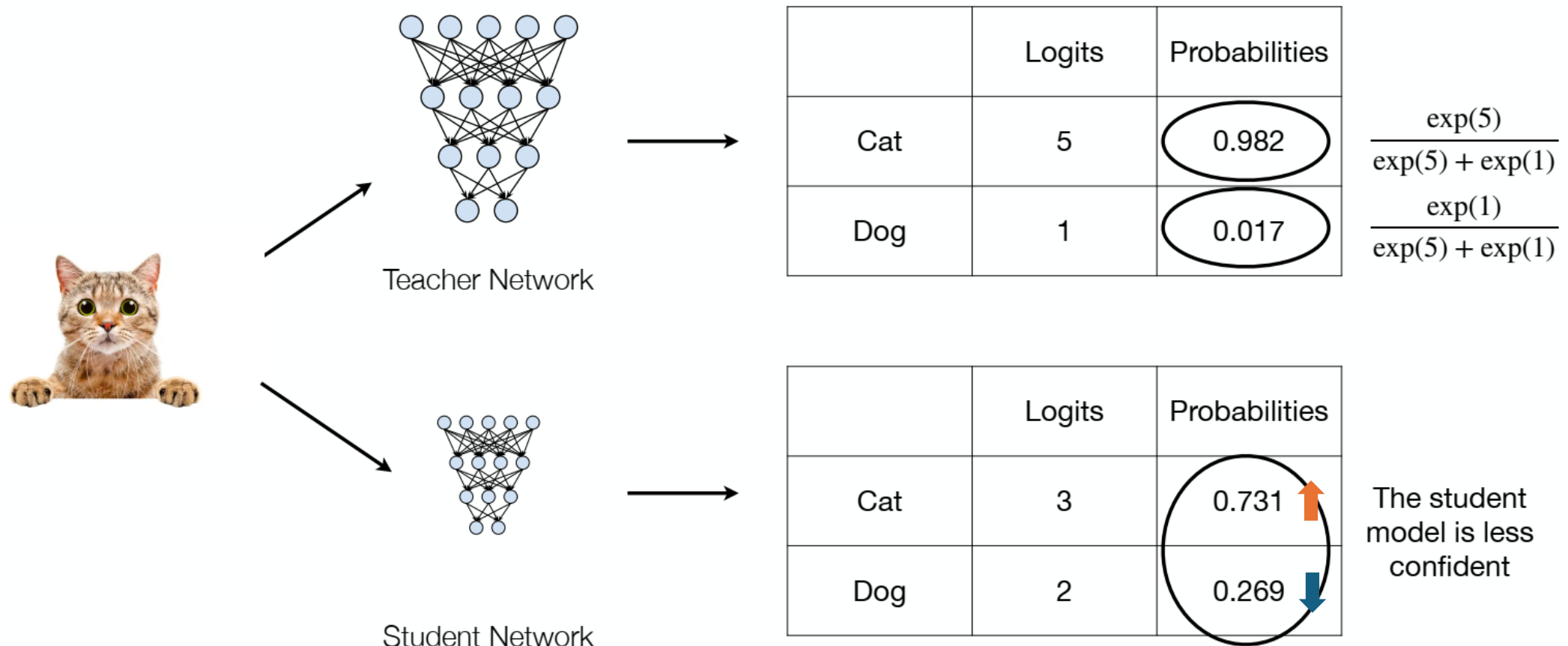
Formal Definition of KD

- Neural networks typically use a softmax function to generate the **logits** z_i to class **probabilities** $p(z_i, T) = \frac{\exp(\frac{z_i}{T})}{\sum_j \exp(\frac{z_j}{T})}$
 - $i, j = 0, 1, 2, \dots, C-1$, where C is the number of classes.
 - T is the **temperature**, which is normally set to 1.
- The goal of knowledge distillation is to align the class probability distributions from teacher and student networks.



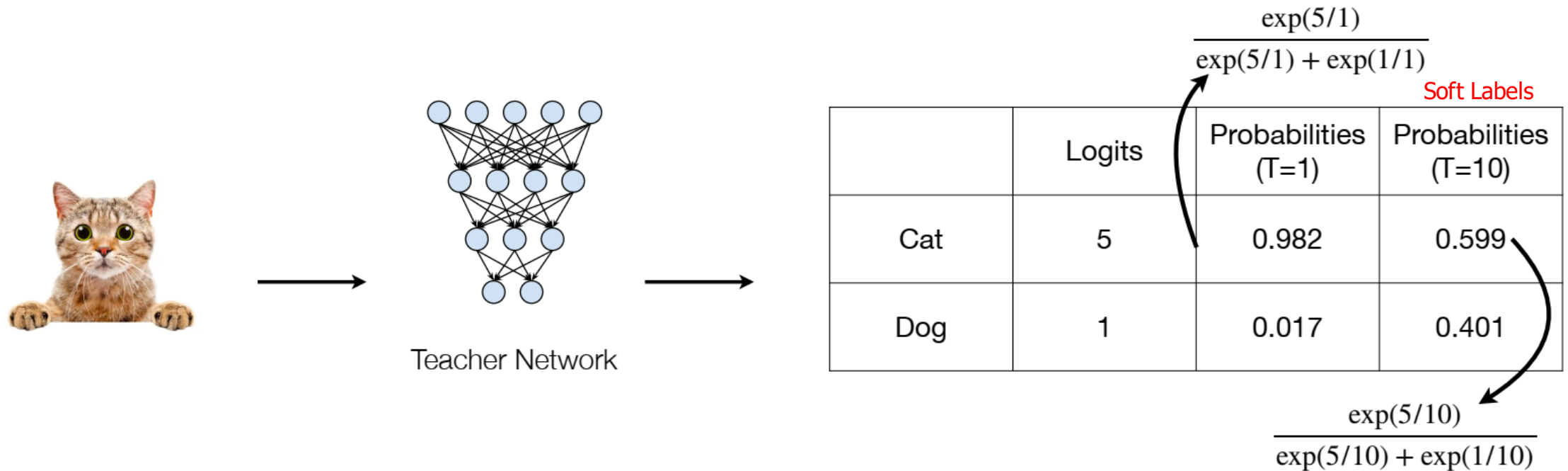
Intuition of knowledge distillation

- Matching prediction probabilities between teacher and student



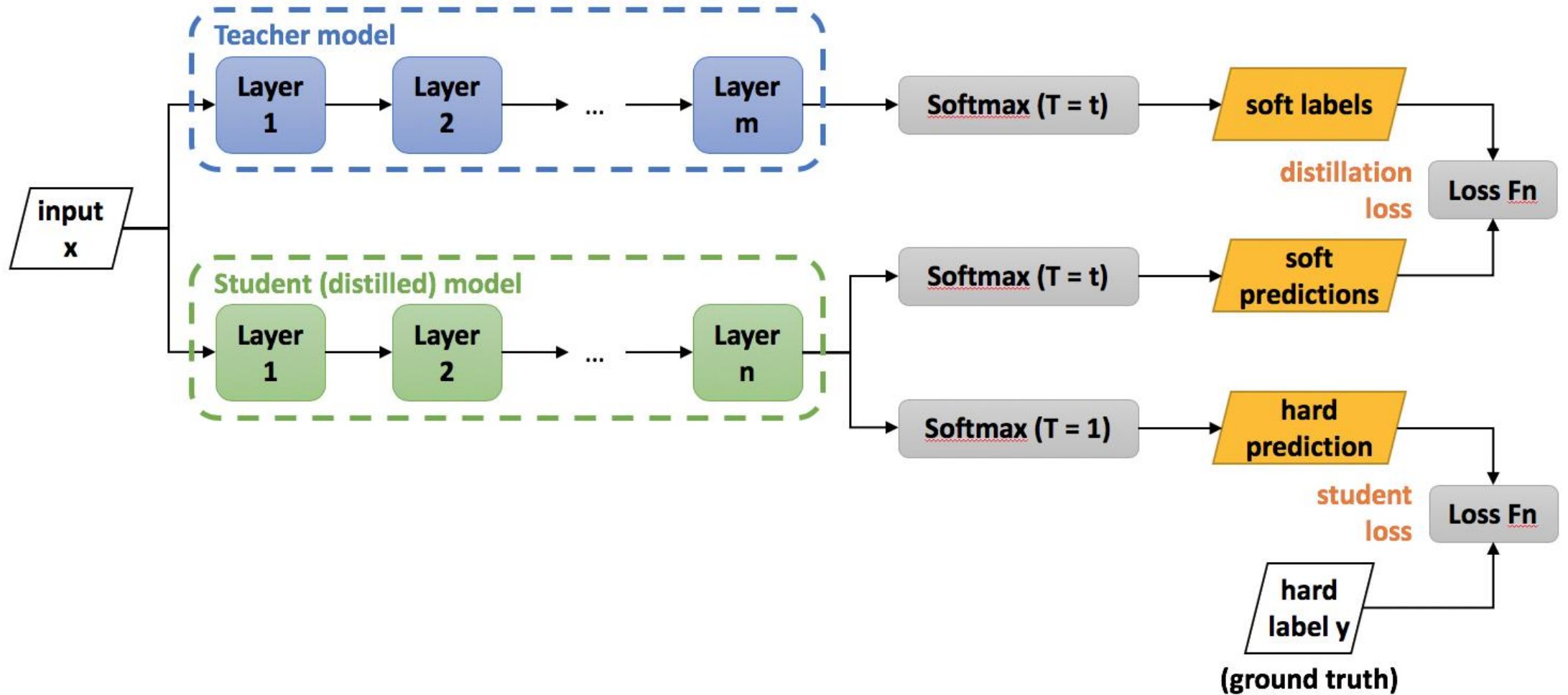
Intuition of knowledge distillation

- Concept of temperature

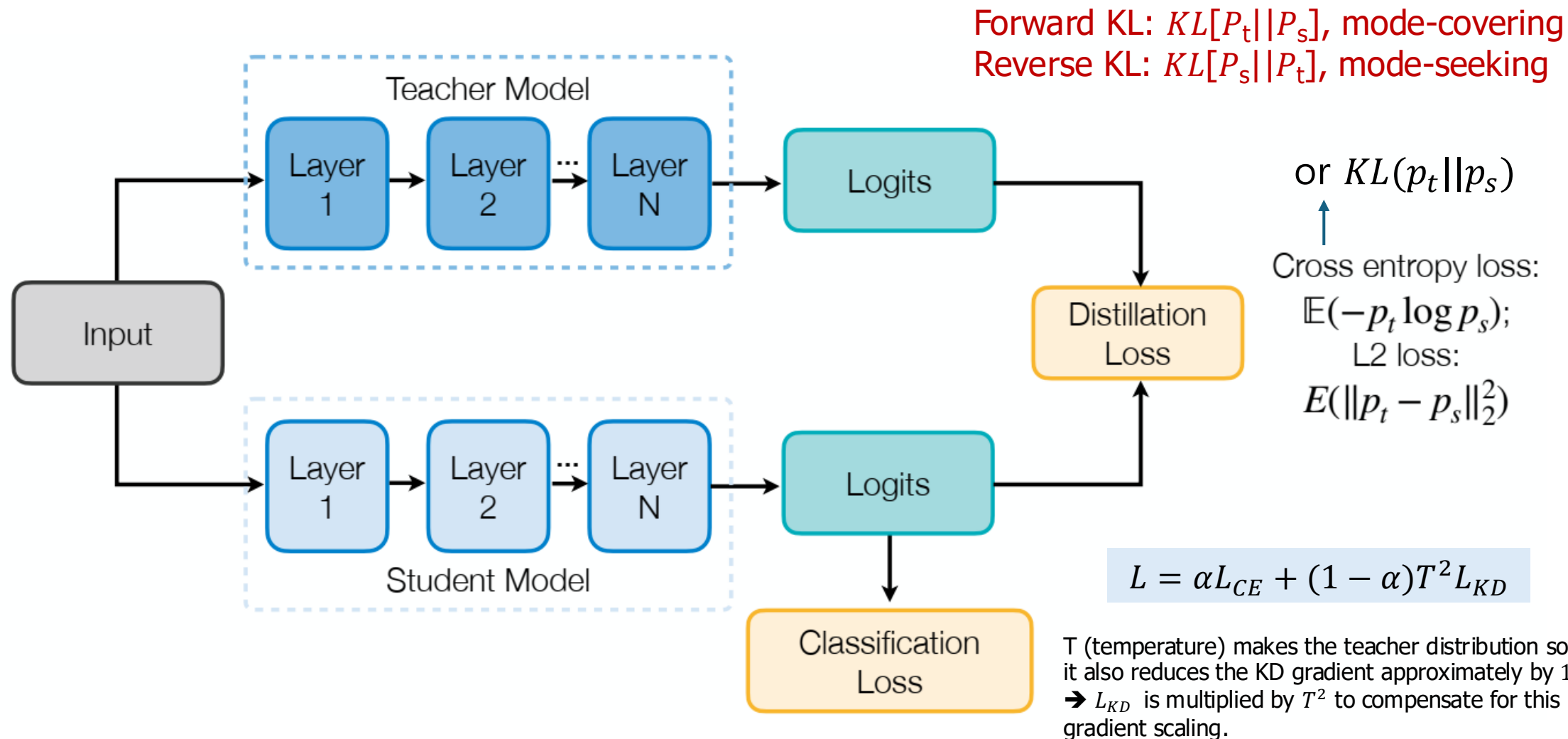


A larger temperature smooths the output probability distribution.

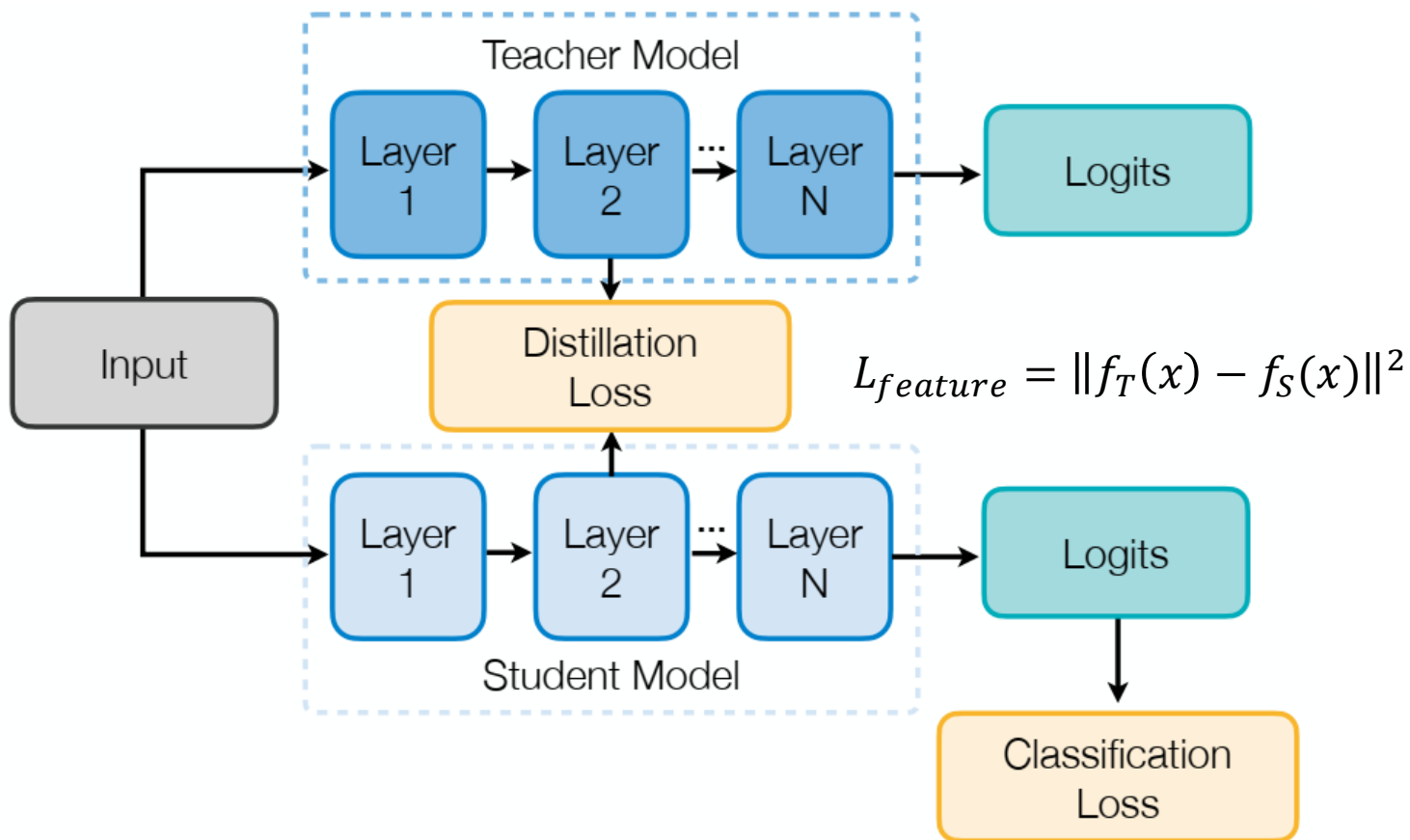
Soft Label vs Hard Label



Response-based KD

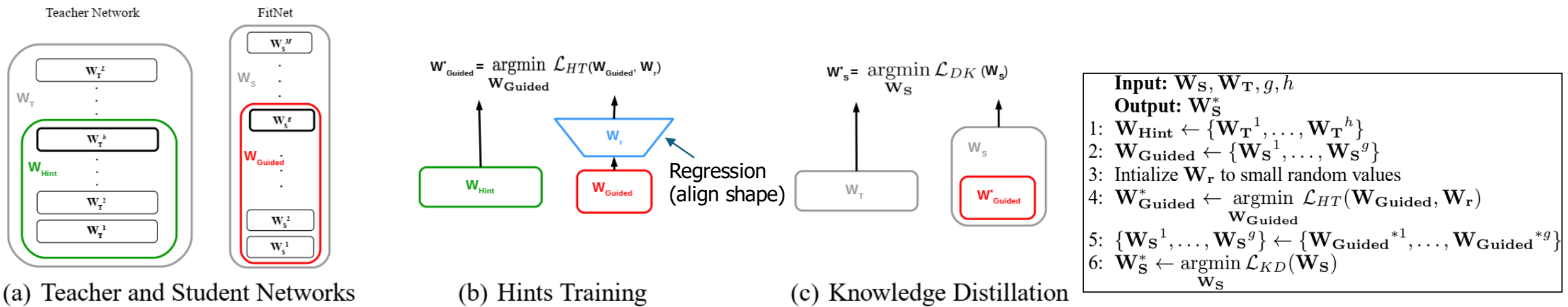


Feature-based KD



Matching intermediate weights

- Hint layer: teacher's hidden layer responsible for guiding the student's learning process.
- Guided layer: student's hidden layer to learn from the teacher's hint layer.
 - The guided layer should be trained to predict the output of the hint layer.



$$\mathcal{L}_{HT}(\mathbf{W}_{\text{Guided}}, \mathbf{W}_r) = \frac{1}{2} \|u_h(\mathbf{x}; \mathbf{W}_{\text{Hint}}) - r(v_g(\mathbf{x}; \mathbf{W}_{\text{Guided}}); \mathbf{W}_r)\|^2$$

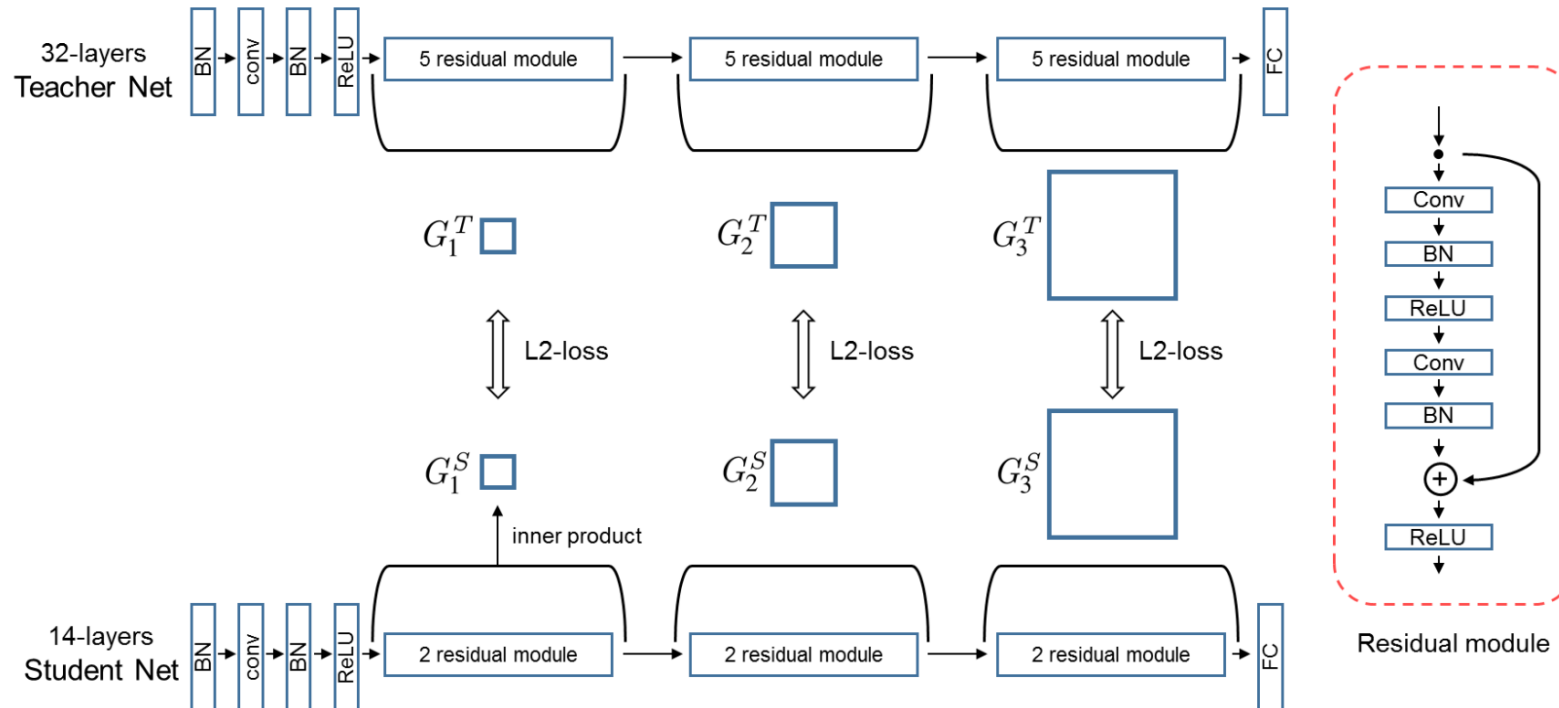
regression

$$\mathcal{L}_{KD}(\mathbf{W}_S) = \mathcal{H}(\mathbf{y}_{\text{true}}, \mathbf{P}_S) + \lambda \mathcal{H}(\mathbf{P}_T^\tau, \mathbf{P}_S^\tau)$$

Cross entropy

Relation-based KD

- Relations between different layers



$$G_{i,j}(x; W) = \sum_{s=1}^h \sum_{t=1}^w \frac{F_{s,t,i}^1(x; W) \times F_{s,t,j}^2(x; W)}{h \times w}$$

- FSP (flow of solution procedure) matrices of teacher and student networks (G_1^T , G_1^S).
- Inner product of the features from two layers.

Transformer KD

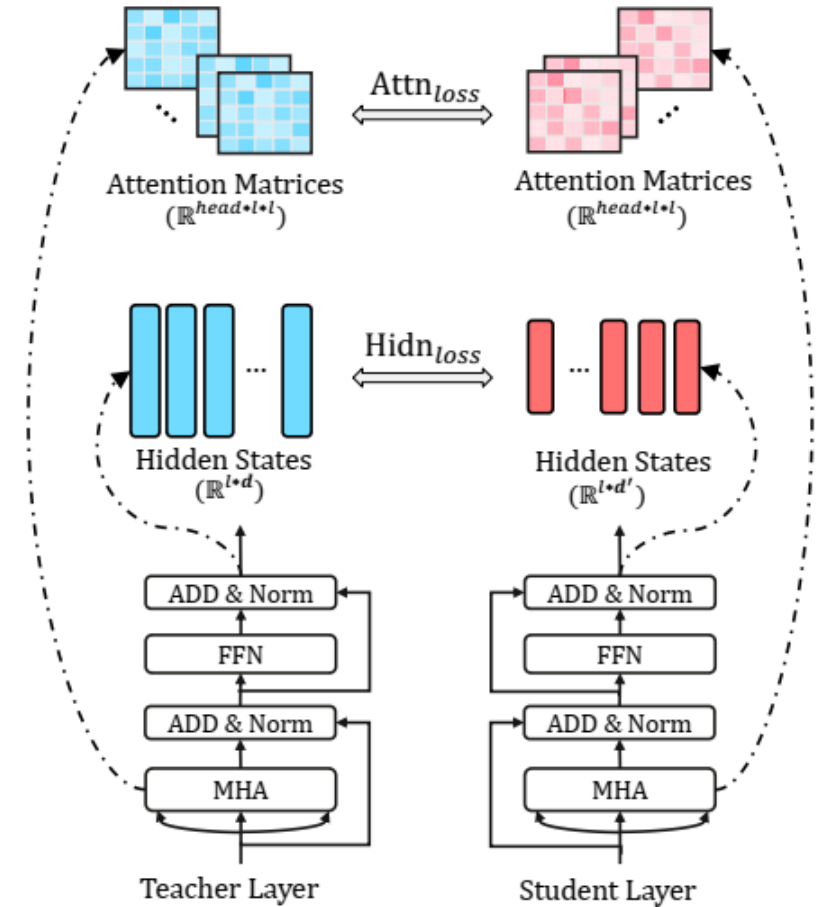
- $L = \lambda_1 L_{logit} + \lambda_2 L_{hidden} + \lambda_3 L_{attn} + \lambda_4 L_{embd}$

- $L_{logit} = CE(Z^T, Z^S)$

- $L_{attn} = \frac{1}{h} \sum_{i=1}^h MSE(A_i^S, A_i^T)$
 - h : # of attention heads

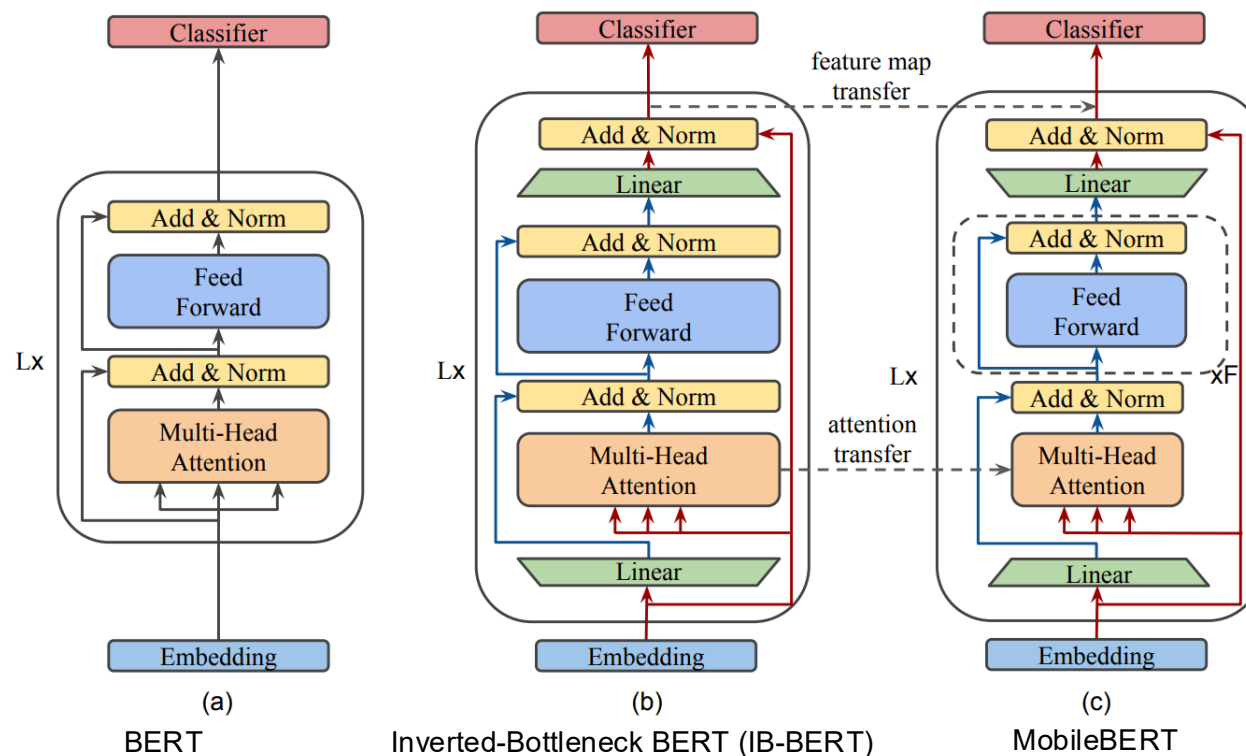
- $L_{hidden} = MSE(H^S W_h, H^T)$

- $L_{embd} = MSE(E^S W_e, E^T)$
 - W_h, W_e : learnable linear transformation



Transformer KD

- IB-BERT and MobileBERT have the same feature map size
- Feature map transfer, Attention transfer

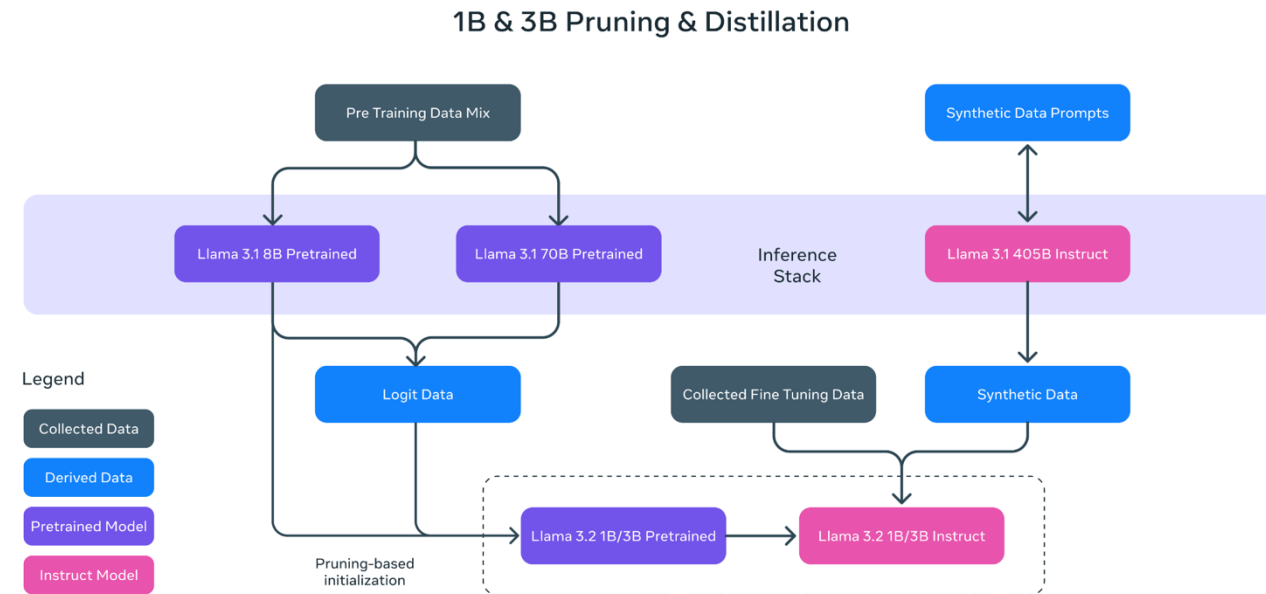
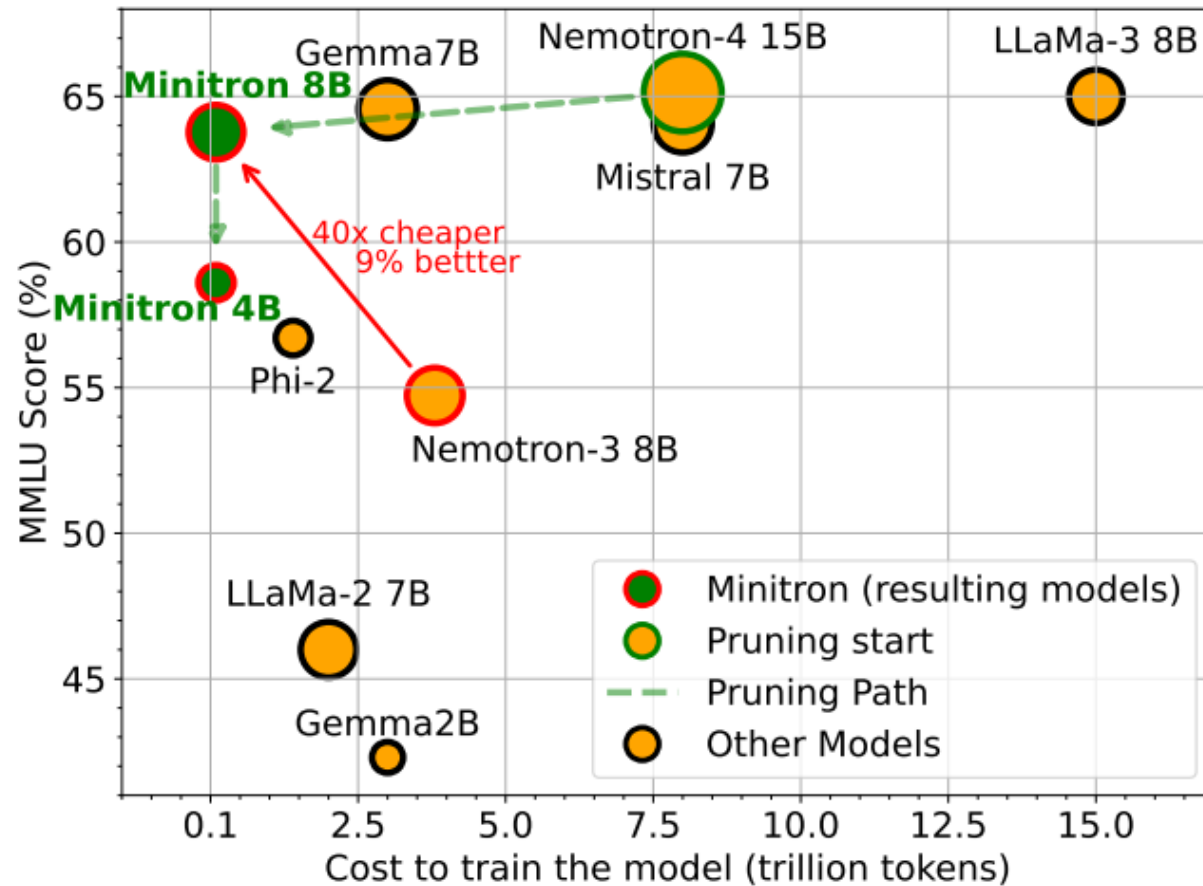


LLM Distillation

- Token-level KD
 - $L_{token-KD} = \sum_t KL(p_T(y_t|y_{<t}, x) || p_S(y_t|y_{<t}, x))$ match vocab distribution
- Sequence/Response Distillation [Sequence-Level Knowledge Distillation \[EMNLP'16\]](#)
 - SFT (super-vised fine-tuning) based on Teacher-generated sequence
 - Useful for black-box teacher
 - $L_{seq} = -\sum_t \log p_S(\hat{y}_t | \hat{y}_{<t}, x), \hat{y}_T = (\hat{y}_1, \hat{y}_2, \dots, \hat{y}_m)$
 - Exposure Bias: mismatch between training distribution and inference distribution
- On-Policy Distillation / Generalized KD
 - Student generates its own sequence, teacher evaluates and gives feedback [On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes \[ICLR'24\]](#)
- Reasoning Distillation
 - Distill reasoning trajectory

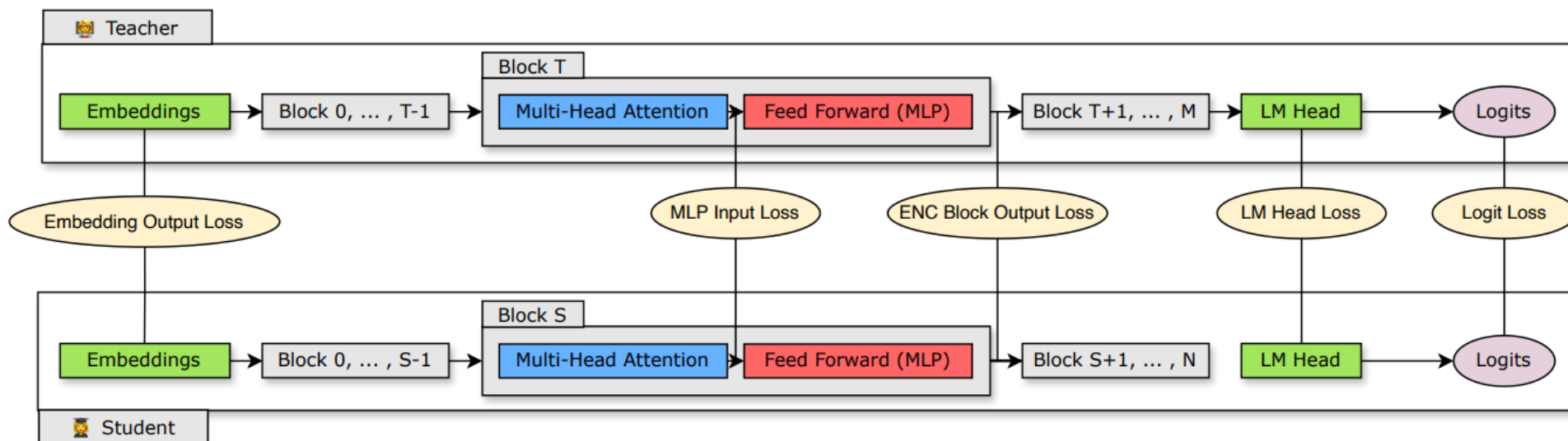
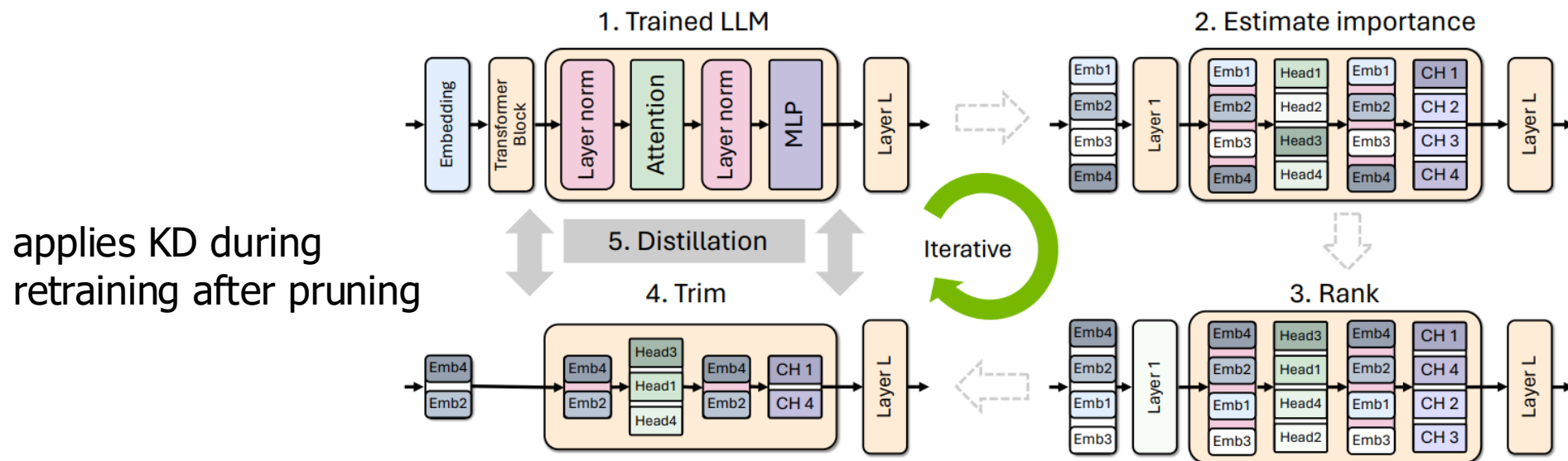
[Distilling Step-by-Step! Outperforming Larger Language Models with Less Training Data and Smaller Model Sizes \[ACL Findings 2023\]](#)

LLM Distillation

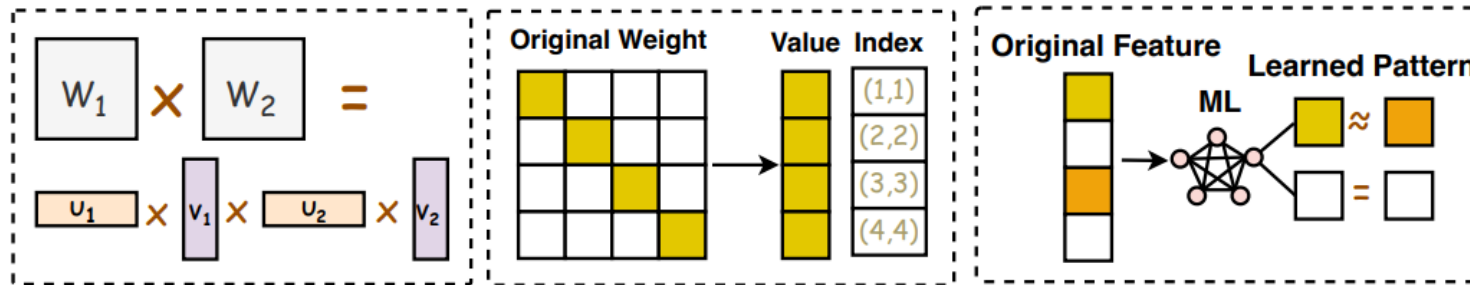


Llama 3.2: Revolutionizing edge AI and vision with open, customizable models

LLM Distillation



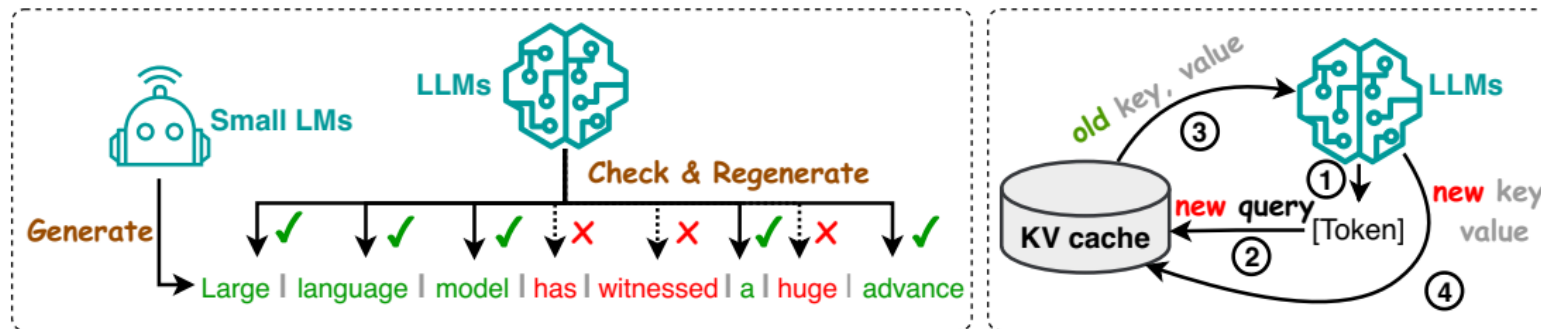
Efficient Transformer



(b) Kernelization or Low-Rank

(c) Fixed Pattern Strategies

(d) Learnable Pattern Strategies



(a) Speculative Decoding

(b) KV-Cache Optimization

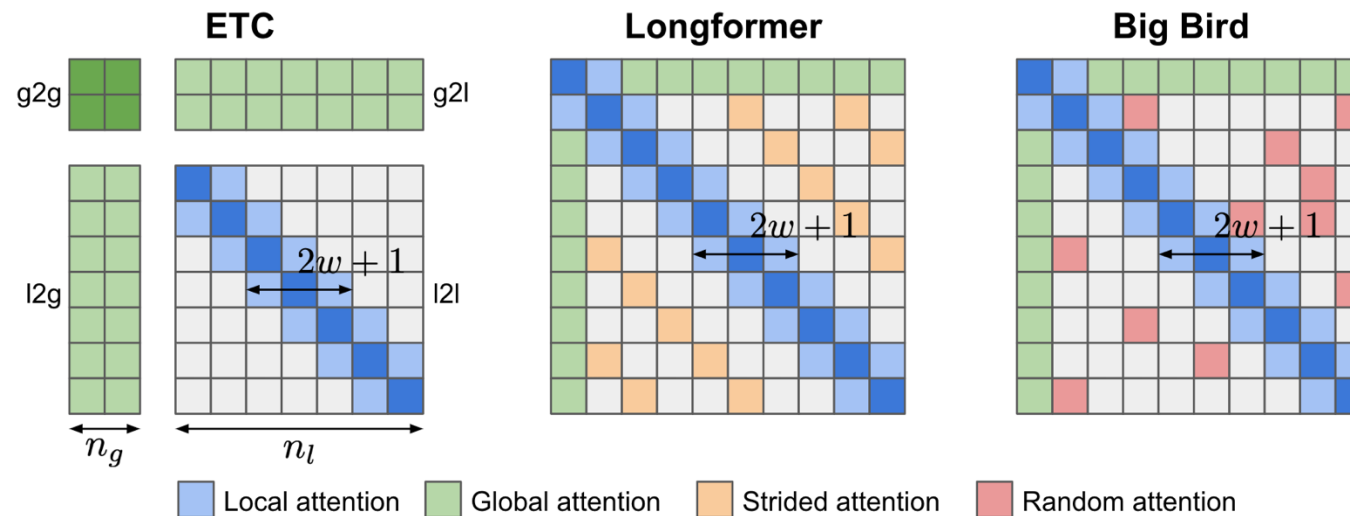
Fixed Sparse Attention

- **Longformer**

- Local Attention
- Global Attention: global memory tokens that have access to all input sequences.
- Dilated Sliding Windows: Smaller window size at lower layers and larger window sizes at higher layers

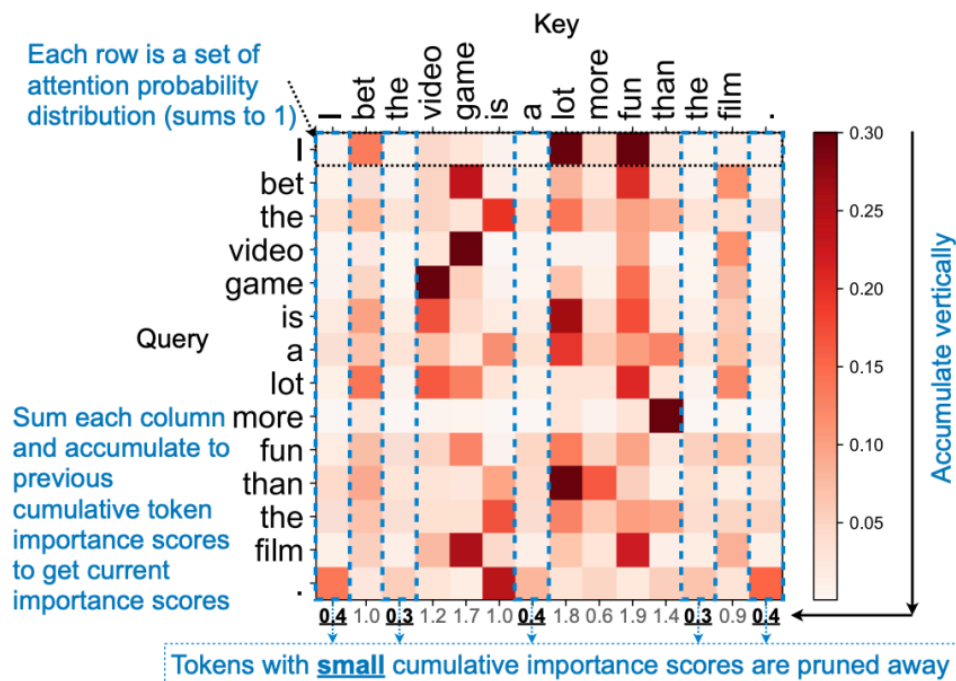
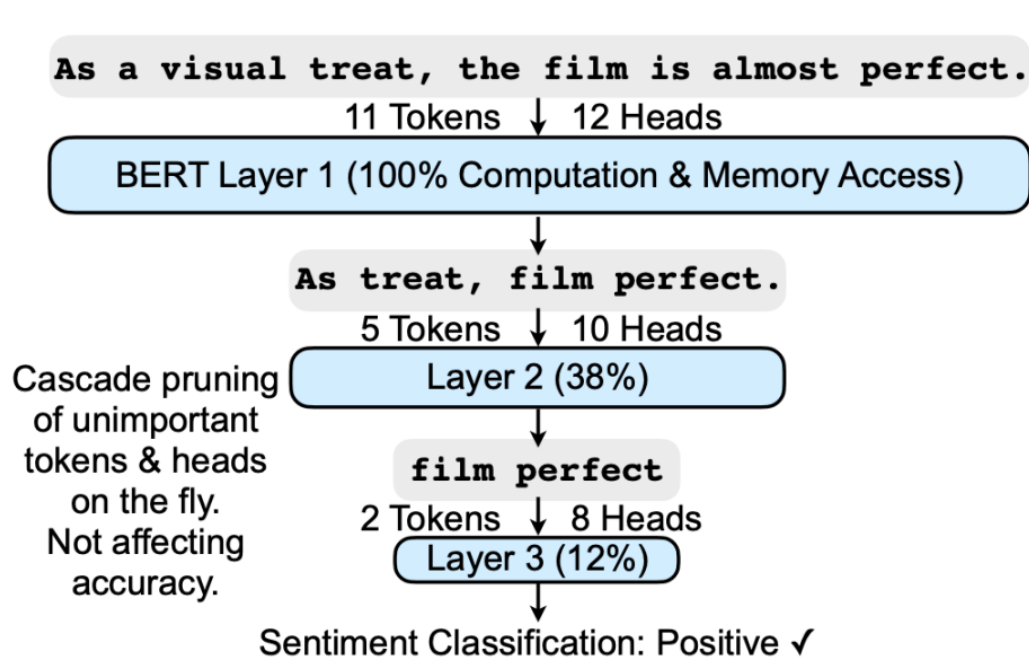
- **Big Bird**

- replaces dilated attention with **random attention**



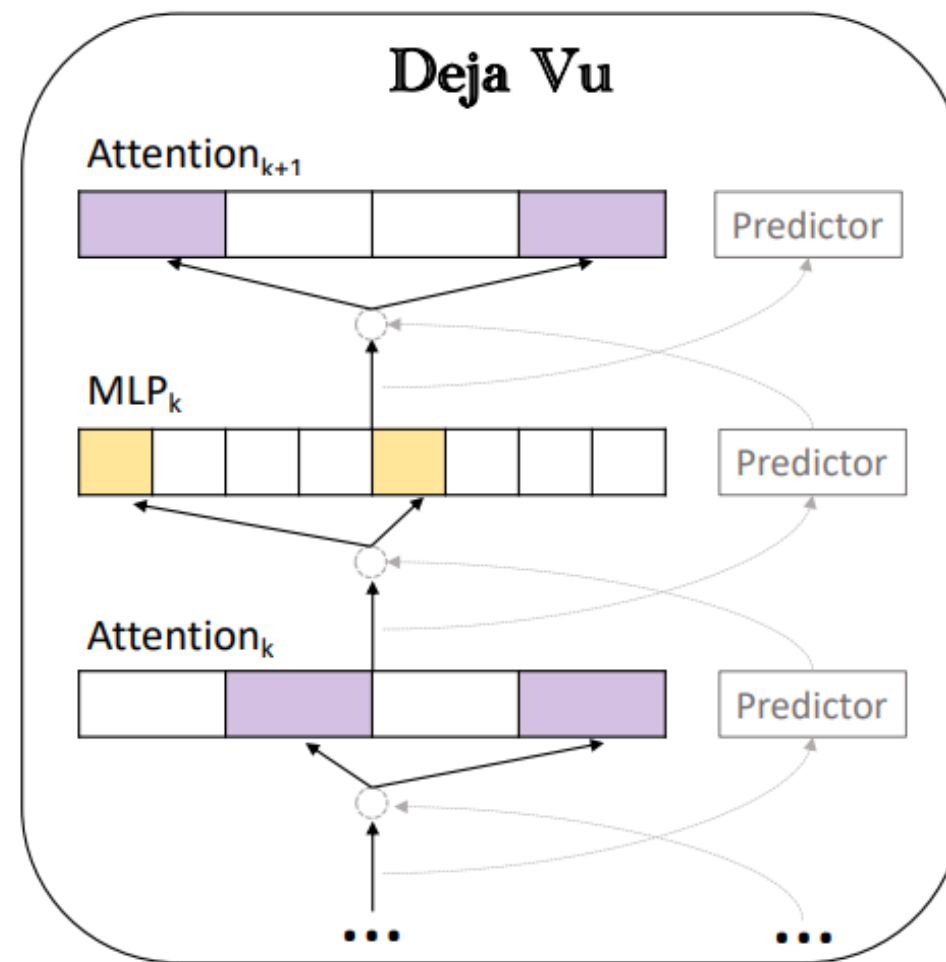
Dynamic Sparsity - SpAtten

- Token pruning & head pruning
- Cascade pruning of unimportant tokens and heads.
- Tokens with a small cumulative attention are pruned away.



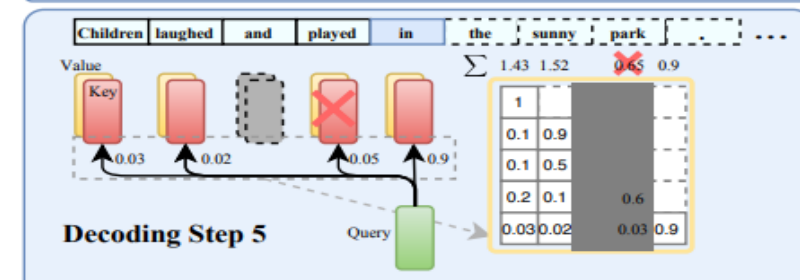
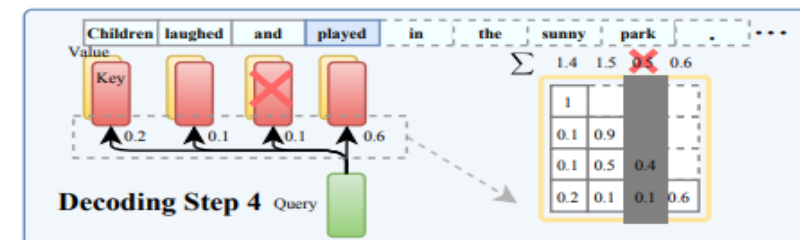
Deja Vu

- Contextual Sparsity Prediction
 - Training additional **sparsity predictor** for MLP and Attention.
 - Predict the sparsity of MLP inner dimensions and Attention-heads
- Asynchronous Execution
 - Leverage slowly evolving embedding phenomenon
 - **Look-ahead** sparse prediction
 - Parallel execution of the sparse prediction with the computation by using previous block's outputs.

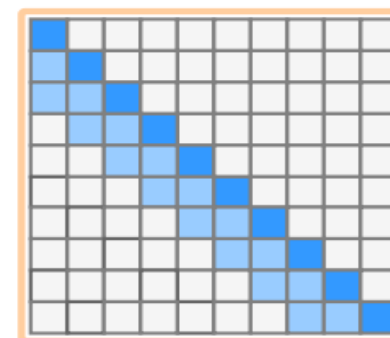


KV Pruning (Eviction)

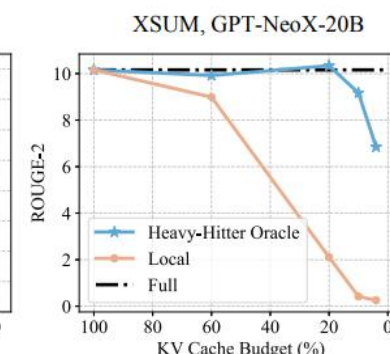
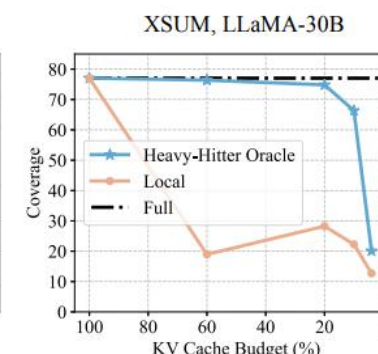
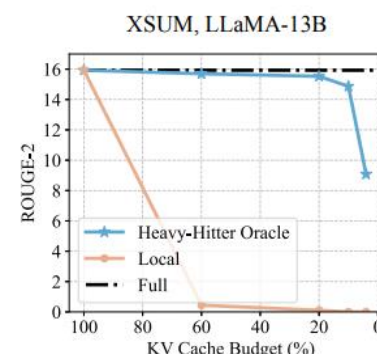
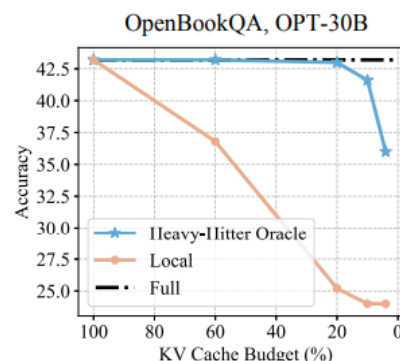
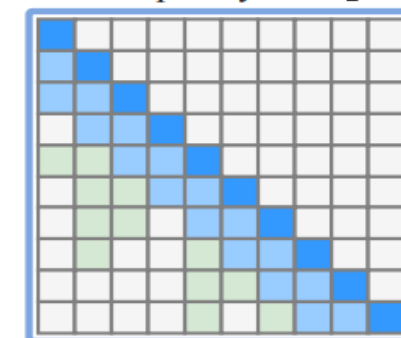
- Only 5% of the KV cache is sufficient for decoding the same output token at each generation step
- Only the **H2** and the **recent KV**
- How to select necessary tokens?
 - token's importance $\approx$ attention scores (softmax($Q \cdot K$)).
- KV cache entries with low accumulative attention scores are removed.



Static Sparsity (Local)

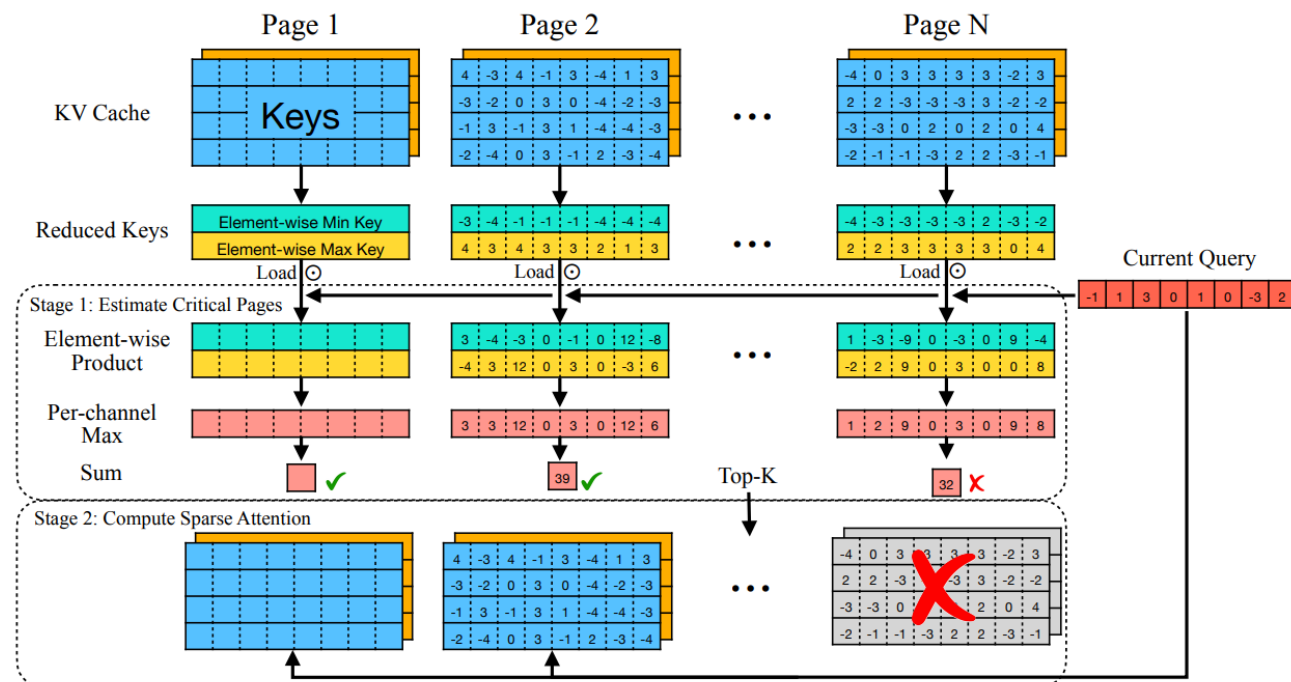
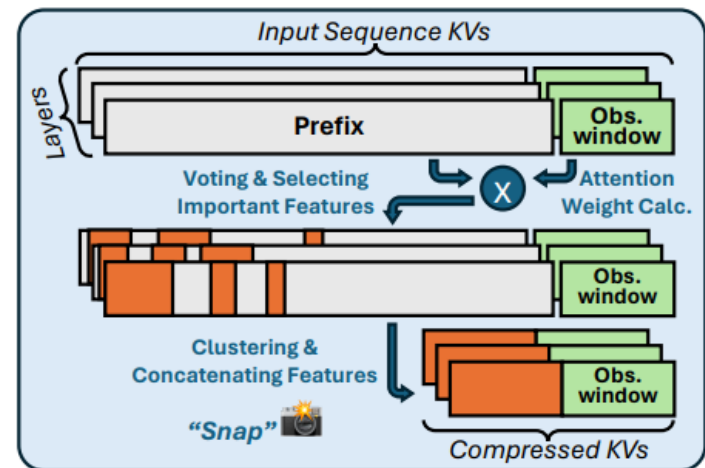


Static Sparsity w. H_2O



KV Pruning (Eviction)

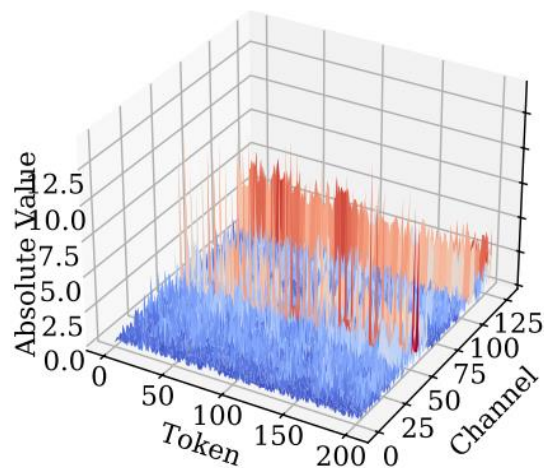
- **SnapKV**: *Static*, Compress prompt KV before generation
 - Observation-window attention pattern
- **Quest**: *Dynamic*, Query-aware
 - Dynamically select KV pages during decoding
 - For each KV cache page, it stores lightweight metadata: the min/max Key values for each feature dimension
 - Evicted tokens can still be attended by future tokens.



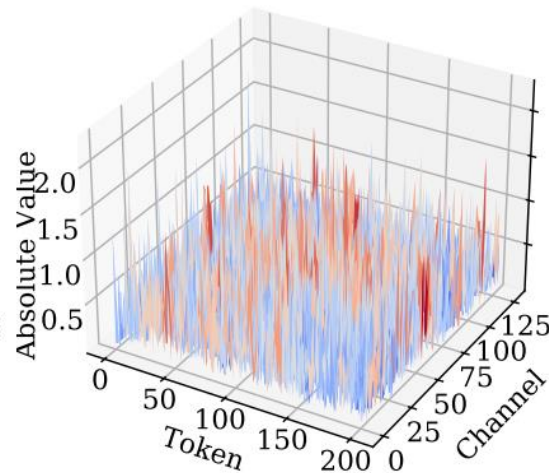
KV Quantization - KIVI

- Key: per-channel, Value: per-token

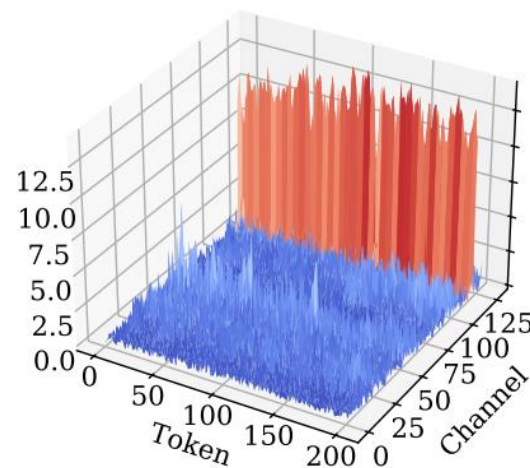
Llama-2-13B Layer 16
Head 0 Key Cache



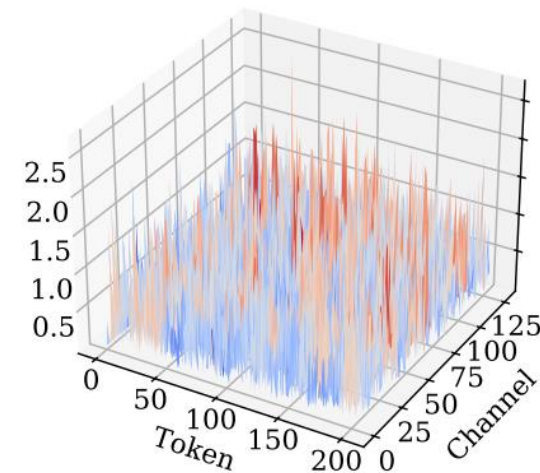
Llama-2-13B Layer 16
Head 0 Value Cache



Llama-2-13B Layer 31
Head 0 Key Cache

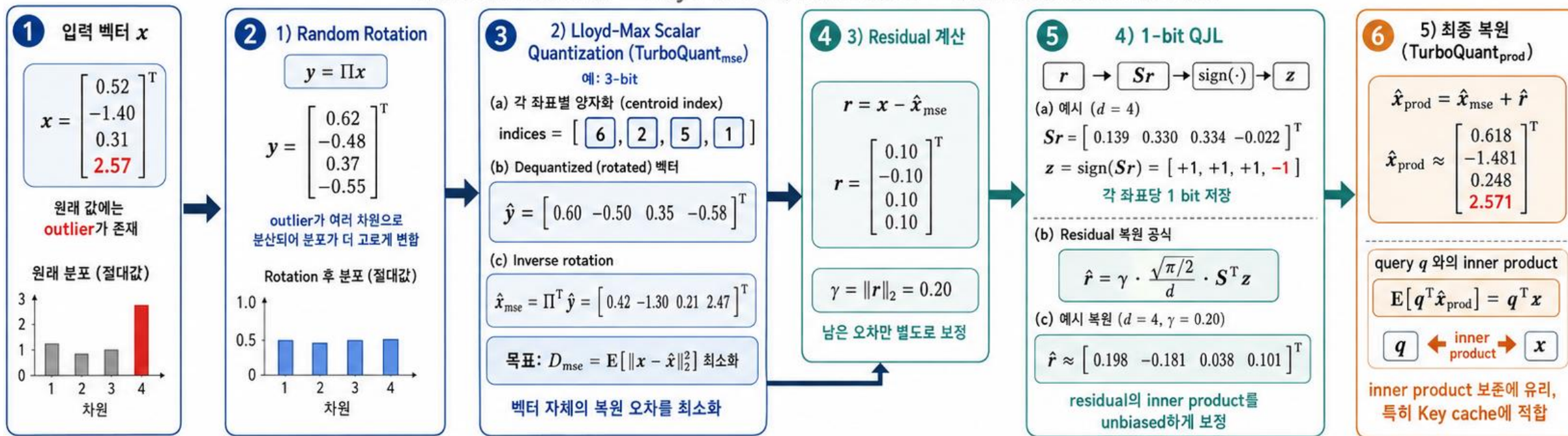


Llama-2-13B Layer 31
Head 0 Value Cache



KV Quantization - TurboQuant

Random Rotation + Lloyd-Max Quantization + QJL Residual Correction



기호 정의

Π : 직교 행렬 (랜덤 회전)
 Π^T : 역회전 행렬
 $S \in \mathbb{R}^{d \times d}$: JL 행렬 (예: Rademacher)
 $\text{sign}(\cdot)$: 원소별 부호 함수
 d : 차원 (예시에서는 4)
 b : 전체 bit-width (예: 3-bit)
 $\gamma = \|r\|_2$: residual 크기

TurboQuant_{mse} vs TurboQuant_{prod}

TurboQuant _{mse}	TurboQuant _{prod}
✓ 전체 b -bit를 MSE 최소화 사용	✓ $(b-1)$ -bit MSE + 1-bit QJL
✓ 벡터 복원 정확도 중심	✓ inner product unbiased correction
✓ 낮은 bit에서 inner product bias 가능	✓ Attention score 보존에 유리
✓ Value cache에 더 자연스러운 선택	✓ Key cache에 더 자연스러운 선택

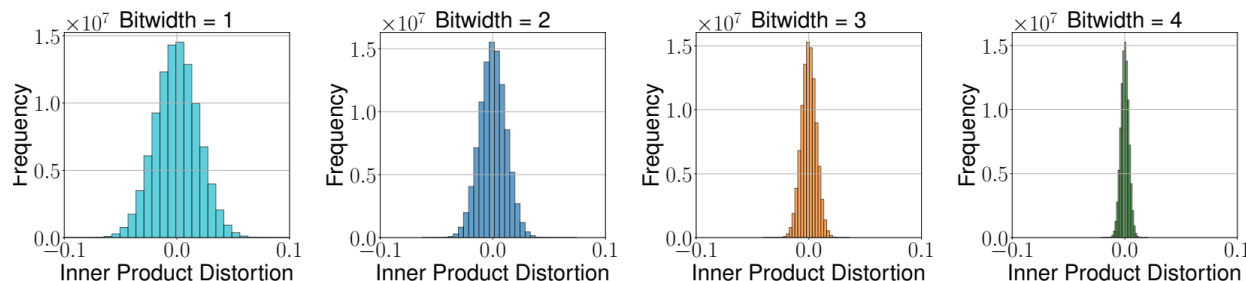
비트 사용 예시 ($b = 3$)

TurboQuant _{mse}	TurboQuant _{prod}
1 1 1	1 1 1
3-bit 전부 MSE	$(b-1)$ -bit MSE + 1-bit QJL

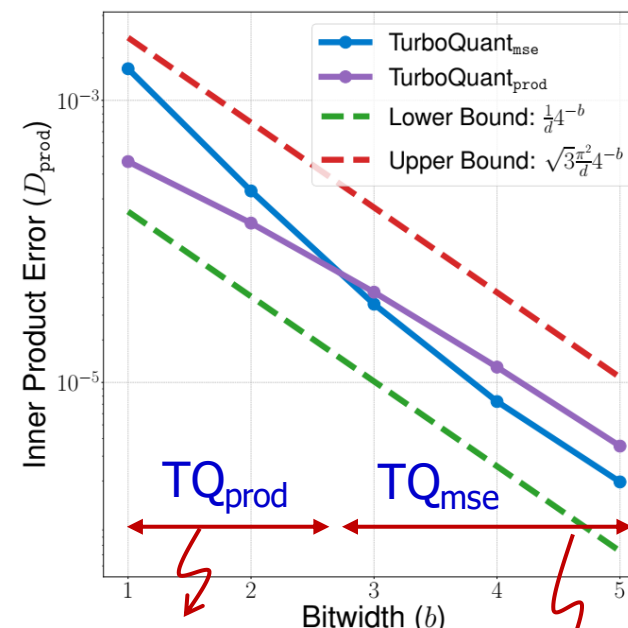
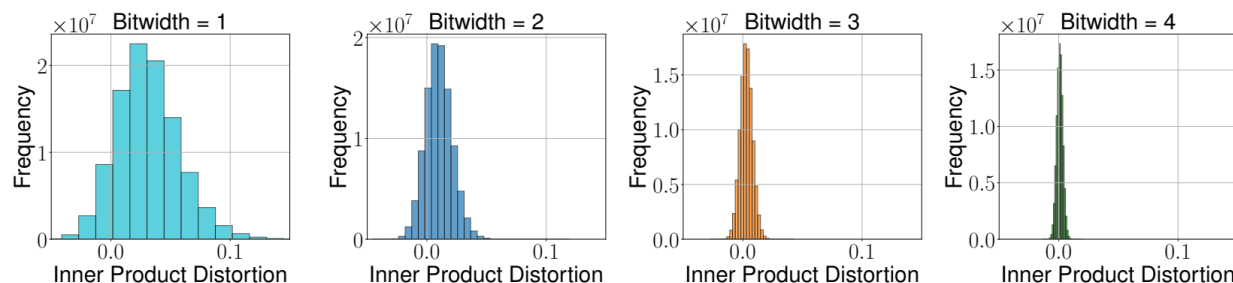
KV Quantization - TurboQuant

- Random rotation $\rightarrow$ Beta distribution on coordinates
- Distribution-aware scalar quantization (Lloyd-Max): data-oblivious
- 1-bit residual correction (QJL) $\rightarrow$ Bias=0
- $\text{TQ}_{\text{mse}}(1)+(2)$: $\min \|K - \hat{K}\|^2$, $\text{TQ}_{\text{prod}}(1)+(2)+(3)$: $\min E[(q^T K - q^T \hat{K})^2]$
- 2.5, 3.5bit mixed precision

(a) TURBOQUANT_{prod}



(b) TURBOQUANT_{mse}

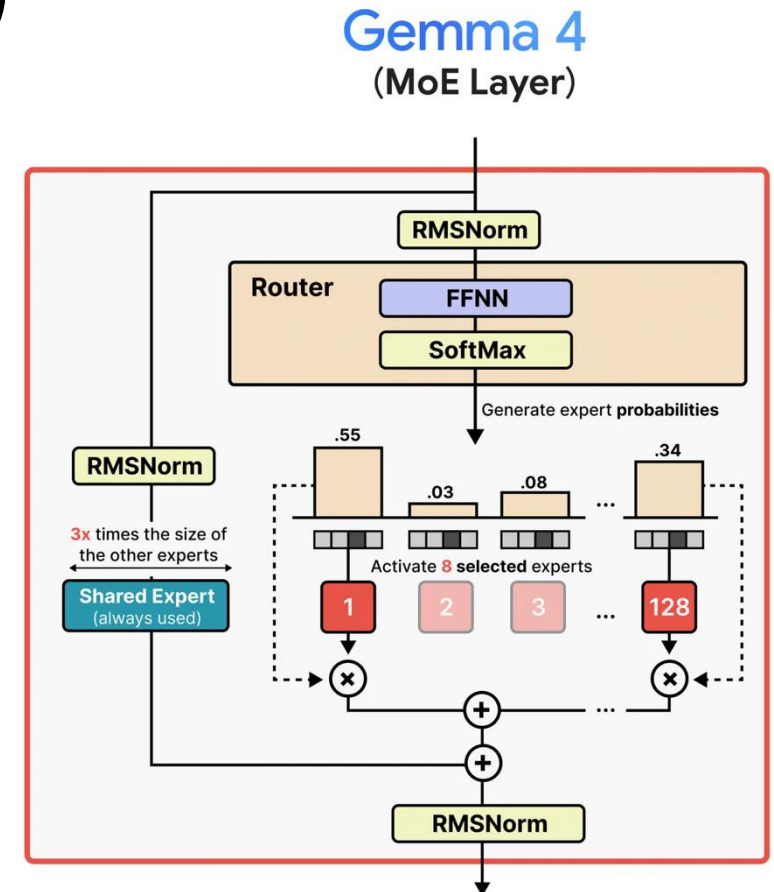


$$\text{Bias}_{\text{mse}}^2 + \text{Variance}_{\text{mse}} > \text{Variance}_{\text{prod}}$$

$$\text{Bias}_{\text{mse}}^2 + \text{Variance}_{\text{mse}} < \text{Variance}_{\text{prod}}$$

Mixture of Experts (MoE)

- A lightweight (0.36M/layer parameters) learned gating router computes a softmax probability distribution across all 128 experts and selects the Top 8 experts.
- The outputs of these 8 experts are then combined via a weighted sum using the router's confidence scores.



128 experts plus one shared expert,
with only 8 experts active per token

<https://botmonster.com/ai/gemma-4-26b-moe-8gb-vram-budget-hardware-setup-guide/>

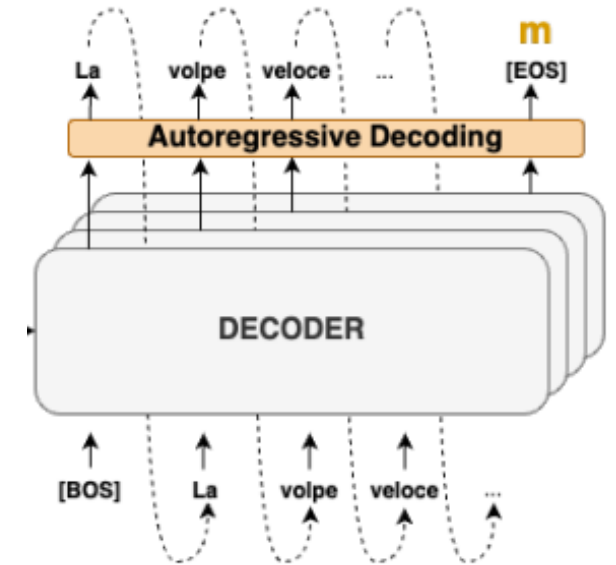
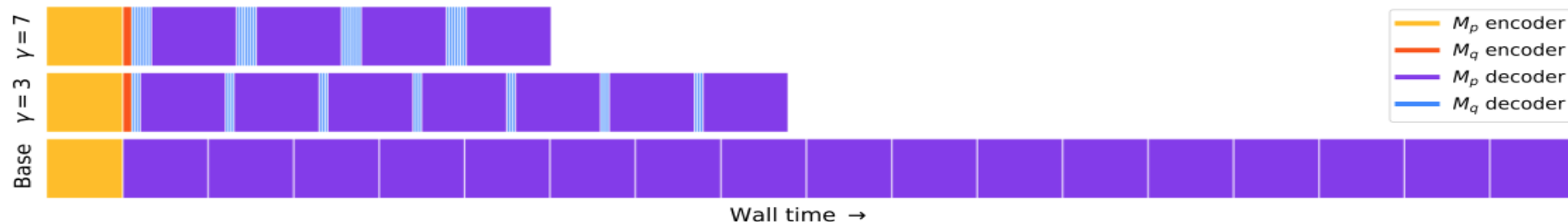
Speculative Decoding

- **Motivations**

- Traditional auto-regressive models need **sequential calls** for each token, causing **memory limits**.
- Some inference steps are harder, and some are easier

- **Speculative Decoding**

- M_q : small, efficient, and approximation model
 - Called **sequentially** γ times to generate tokens
- M_p : large target model
 - Evaluate the guesses and probabilities from M_q in **parallel**
 - Accept those that can lead to an identical distribution
 - Sample an additional token to fix the rejected one, or to add an additional one if all are accepted



Speculative Decoding

- The **green** tokens are the suggestions made by the small model
- The **red** tokens are the rejected suggestions by target model.
- The **blue** tokens are correction by target model.
- $\gamma = 5$

[START] japan ' s benchmark **bond** n
[START] japan ' s benchmark **nikkei** 22 **5**
[START] japan ' s benchmark **nikkei** 225 **index** rose 22 **6**
[START] japan ' s benchmark **nikkei** 225 **index** rose 226 . **69** **points**
[START] japan ' s benchmark **nikkei** 225 **index** rose 226 . 69 points , or **0** **1**
[START] japan ' s benchmark **nikkei** 225 **index** rose 226 . 69 points , or 1 . **5 percent** , to 10 , **9859**
[START] japan ' s benchmark **nikkei** 225 **index** rose 226 . 69 points , or 1 . 5 percent , to 10 , 989 . **79** **in**
[START] japan ' s benchmark **nikkei** 225 **index** rose 226 . 69 points , or 1 . 5 percent , to 10 , 989 . 79 in **tokyo** **late**
[START] japan ' s benchmark **nikkei** 225 **index** rose 226 . 69 points , or 1 . 5 percent , to 10 , 989 . 79 in late **morning trading** . [END]

Speculative Decoding

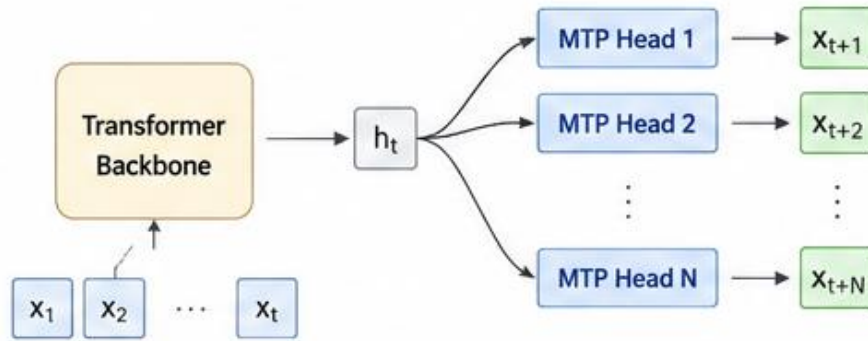
- α : The capability of a smaller model compared to the target model.
- Temp: sampling temperature
- By speculative inference, up to 3.4X speed up is achieved.

Table 2. Empirical results for speeding up inference from a T5-XXL 11B model.

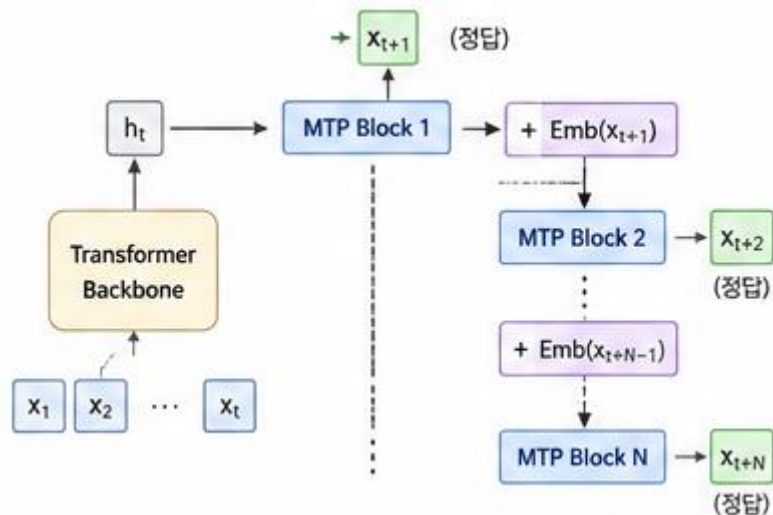
TASK	M_q	TEMP	γ	α	SPEED
ENDE	T5-SMALL ★	0	7	0.75	3.4X
ENDE	T5-BASE	0	7	0.8	2.8X
ENDE	T5-LARGE	0	7	0.82	1.7X
ENDE	T5-SMALL ★	1	7	0.62	2.6X
ENDE	T5-BASE	1	5	0.68	2.4X
ENDE	T5-LARGE	1	3	0.71	1.4X
CNNDM	T5-SMALL ★	0	5	0.65	3.1X
CNNDM	T5-BASE	0	5	0.73	3.0X
CNNDM	T5-LARGE	0	3	0.74	2.2X
CNNDM	T5-SMALL ★	1	5	0.53	2.3X
CNNDM	T5-BASE	1	3	0.55	2.2X
CNNDM	T5-LARGE	1	3	0.56	1.7X

Multi-Token Prediction (MTP)

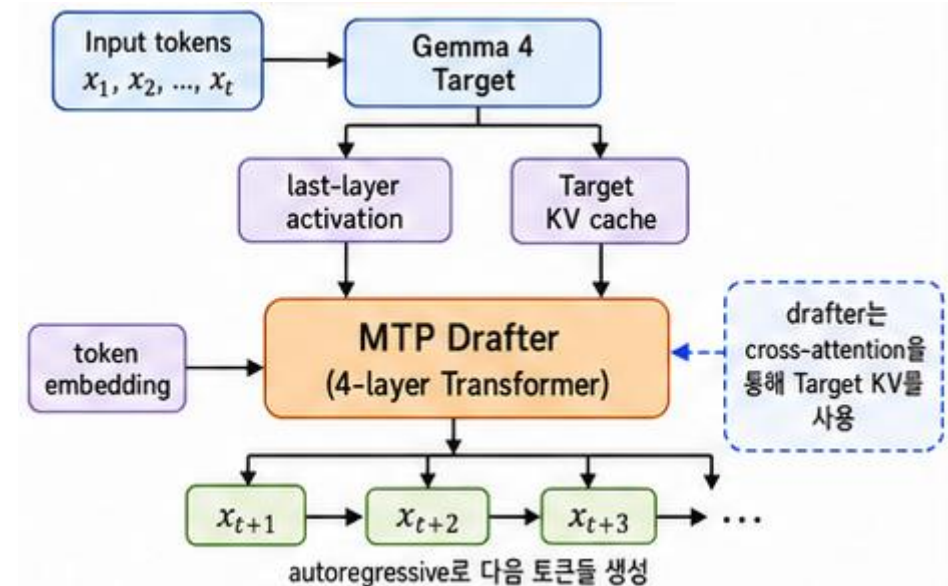
Drafter uses the hidden state of target model.



Independent parallel heads (Medusa, Step 3.5 Flash)



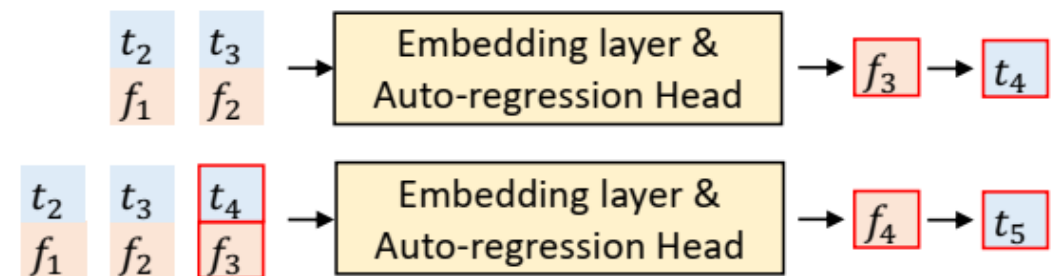
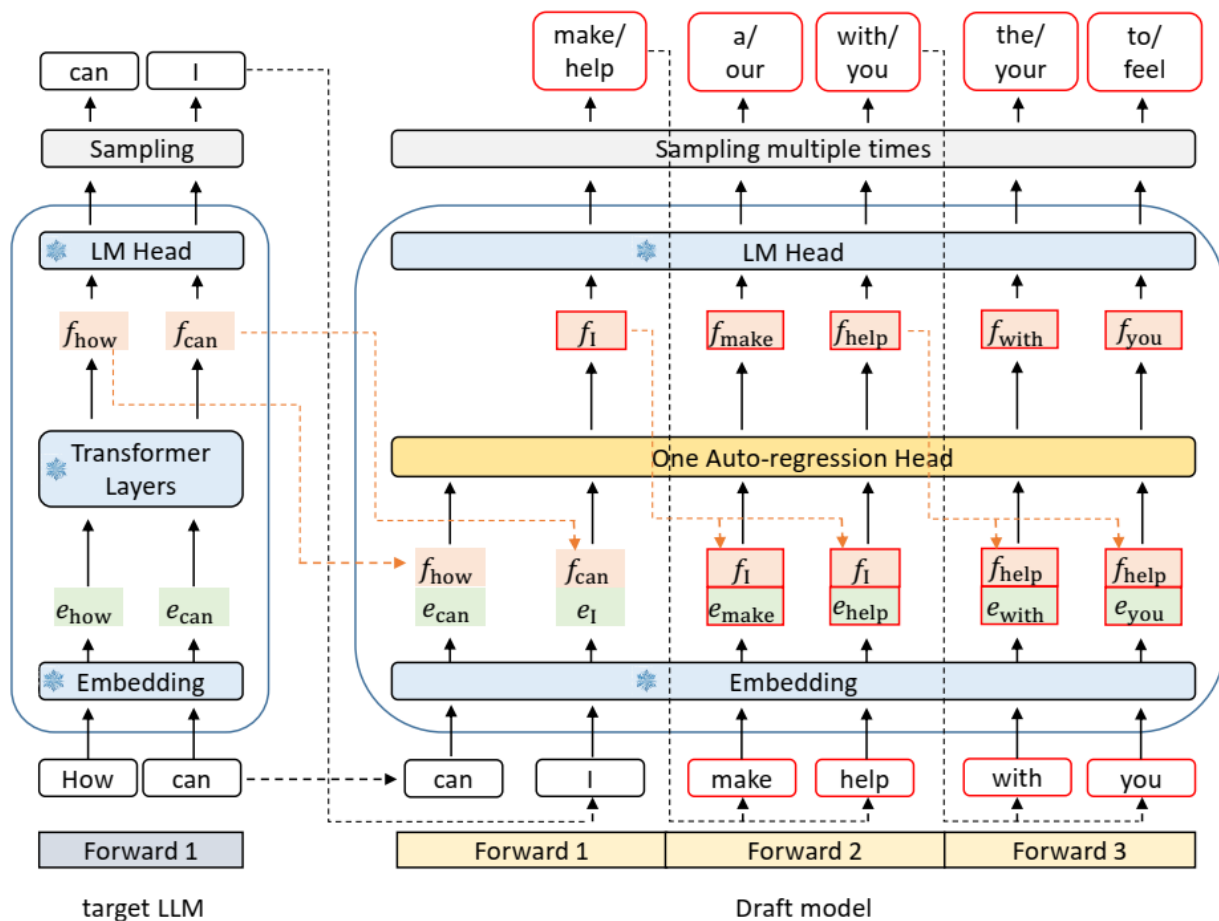
Cascaded (Sequential) MTP (DeepSeek-V3)
Compared to standard SDs, each MTP block is different.



MTP Drafter (Target-Coupled) (Gemma 4)

Speculative Depth	Mean Accepted Tokens / Step	Relative Token Yield / Step	Position 0 Acceptance	Position 1 Acceptance	Relative TPOT Speedup	Relative E2E Latency Reduction
no MTP (k=0, Baseline)	1.00 tok/step	Baseline (0%)	—	—	Baseline (0%)	Baseline (0%)
k = 1	~1.45 tok/step	+45%	~47%	—	7.0% faster	7% lower
k = 2 (Optimal)	~1.71 tok/step	+71%	~47%	~24%	14.0% faster	14% lower

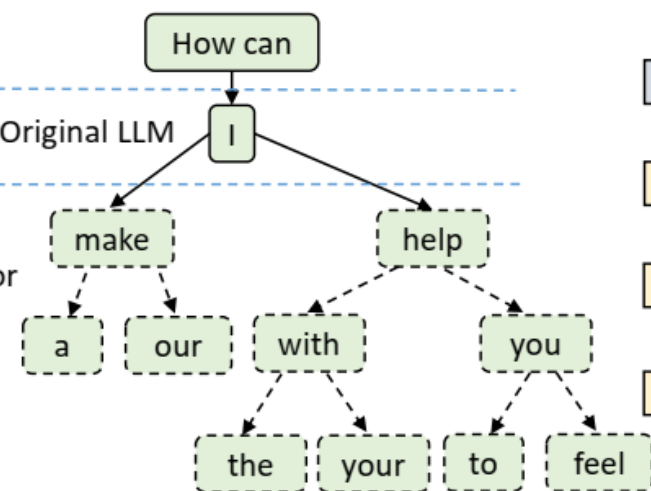
Eagle



Query

Sampling using Original LLM

Drafting using
FeatExtrapolator



Forward 1

Forward 1

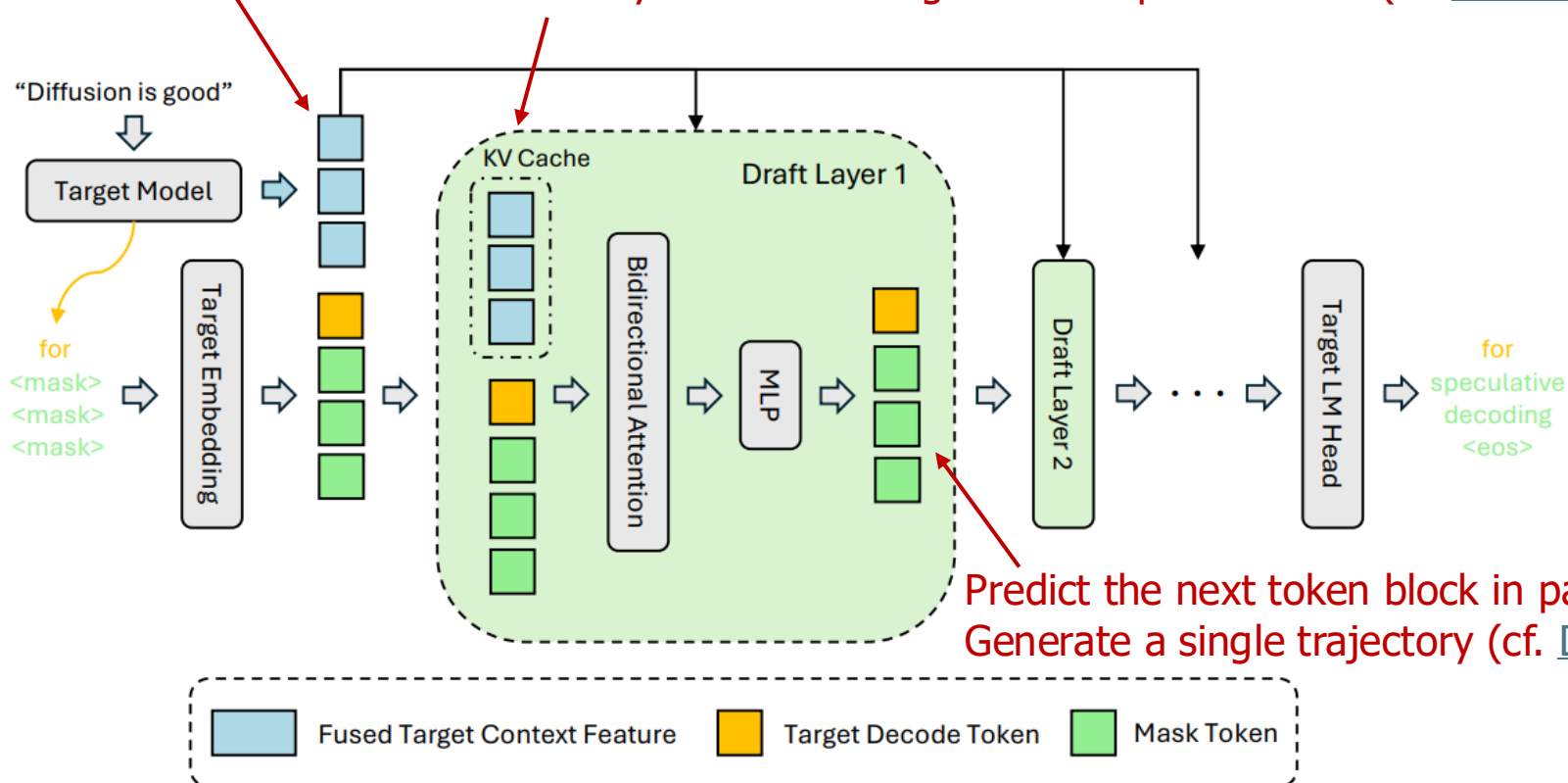
Forward 2

Forward 3

Diffusion-based SD - DFlash

Hidden states from a fixed set of m layers are extracted (*The Target Knows Best*)

Inject the fused target context feature (KV injection)
All draft layers share a single fused representation (cf. [DFlare](#))



Predict the next token block in parallel (Block-Level Diffusion)
Generate a single trajectory (cf. [DDTree](#) (Diffusion Draft Tree))

Figure 2. **DFlash Inference Design.** Hidden context features extracted from the target model are fused and injected into each draft layer's Key-Value cache to enable conditional speculation.

Diffusion-based SD - DFlash

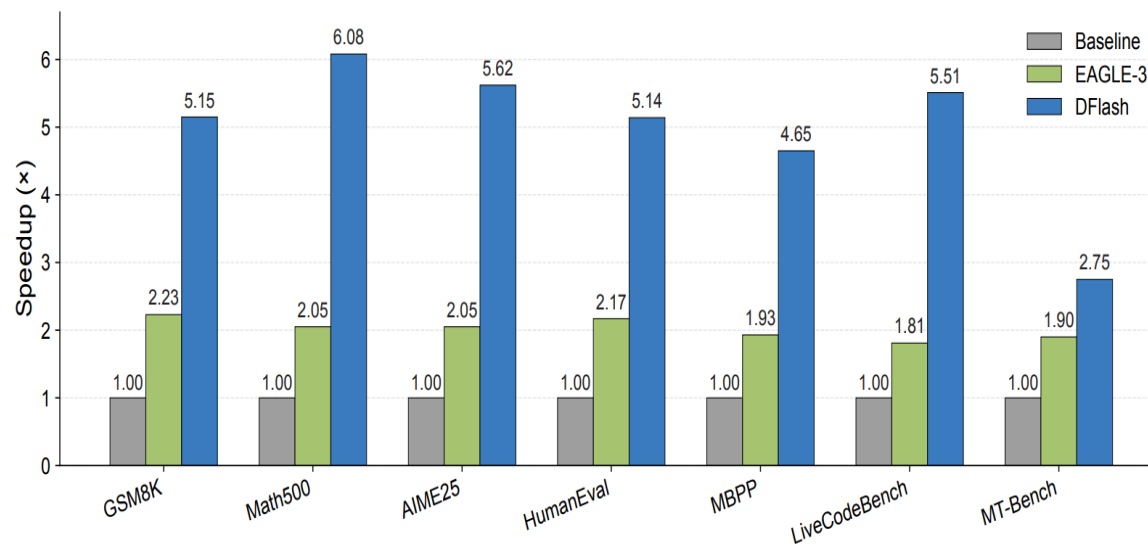


Figure 1. Speedup comparison between DFlash, EAGLE-3 against Autoregressive Decoding on Qwen3-8B (Yang et al., 2025) Transformers backend. Overall, DFlash achieves more than 2.5x higher speedup than EAGLE-3.

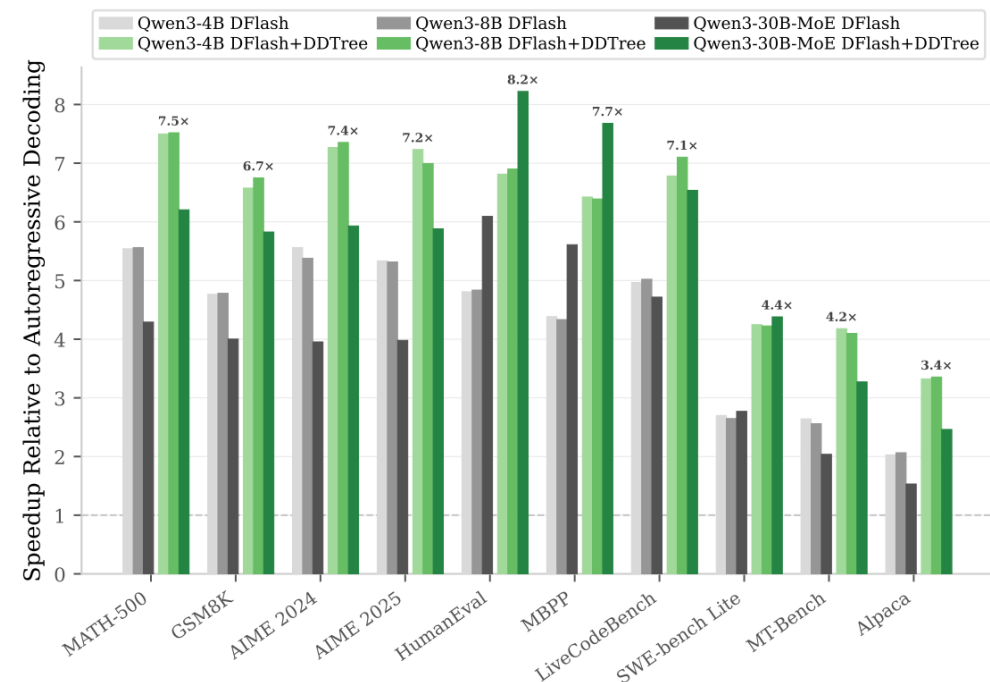


Figure 1: Speedups relative to autoregressive decoding at temperature 0.0 across datasets and target models. DDTree bars use the best tree-node budget for each dataset-model pair.

Diffusion-based SD - DSpark

- Suffix Decay in Parallel Drafting
 - Fully parallel drafters lack inter-token dependencies
 - Multi-modal collisions
 - Rapid acceptance decay at later positions
- DFlash-like parallel drafter: A heavy parallel backbone executes a single forward pass to generate base logits efficiently
- Tiny sequential head injects local transition information (prefix-dependent transition bias $B_k(x_0, x_{<k}, x_k)$)
- Confidence-based verification scheduler
 - Dynamic threshold based on GPU utilization

pos1: of (.55) / no (.40)
 pos2: course (0.40) / problem (0.45)
 → of problem

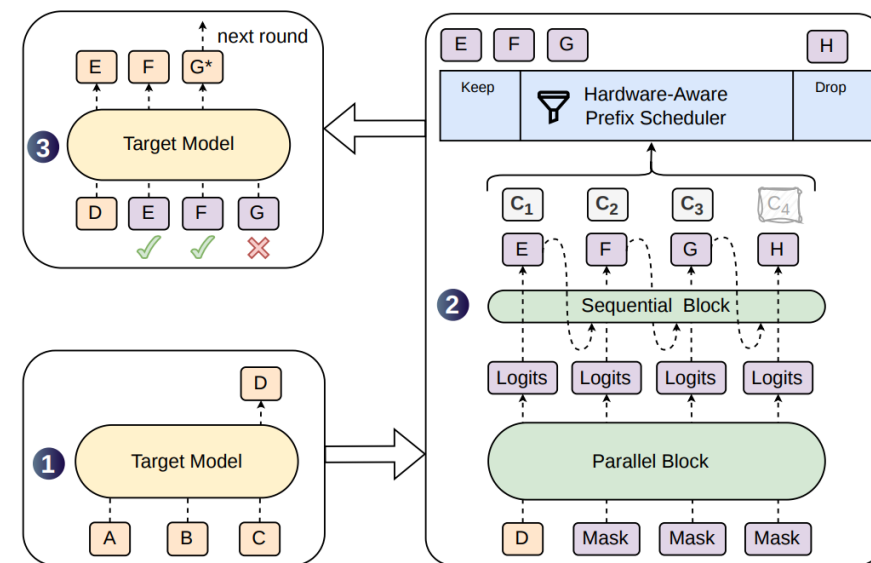


Figure 1 | The DSpark architecture and decoding cycle. Given prompt tokens $[ABC]$, the target model executes one step to generate the next token $[D]$, which serves as the anchor for the drafting phase. Using $[D]$ as the input, DSpark employs a heavy parallel backbone and a lightweight sequential head to generate draft tokens $[EFGH]$ along with their corresponding confidence scores $[c_1-c_4]$. The Hardware-Aware Prefix Scheduler then evaluates these scores to retain the prefix $[EFG]$ and drop the low-confidence token $[H]$. Finally, the target model verifies the scheduled prefix in parallel. As illustrated, $[E]$ and $[F]$ are accepted while $[G]$ is rejected, prompting the model to generate a corrected token $[G^*]$ to complete the current round.

pos1: **of (.55)** / no (.40)
 pos2: course (0.40+**0.14**) / problem (0.45-**0.09**)
 → of course

Diffusion-based SD - DSpark

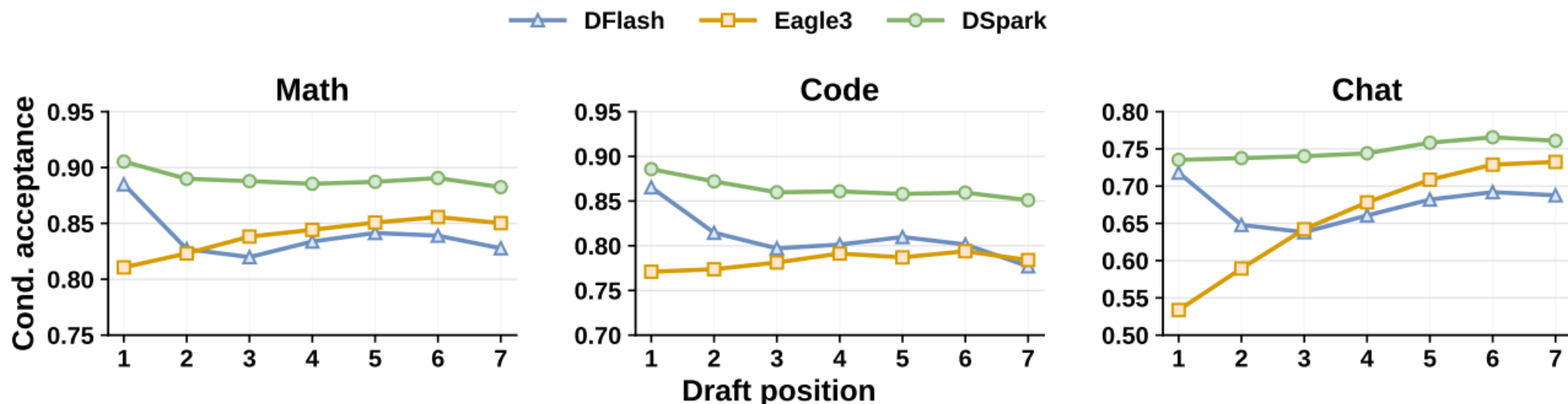
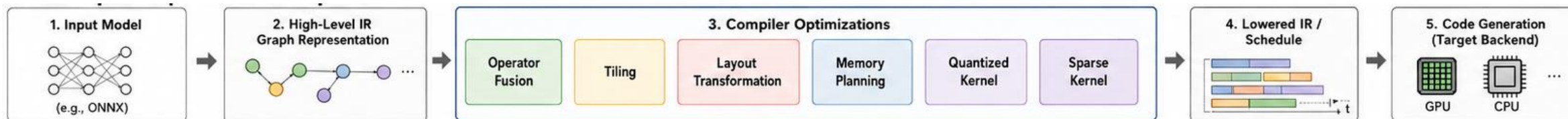


Figure 2 | **Position-wise conditional acceptance.** We report the empirical conditional acceptance rate for each draft position, averaged across benchmarks within each domain using the Qwen3-4B target model. Unlike standard prefix survival, this metric isolates the baseline predictive quality at position k by removing the penalty of previous rejections. Notice that the autoregressive drafter (Eagle3) remains stable or trends upward, while the parallel drafter (DFlash) suffers suffix decay.

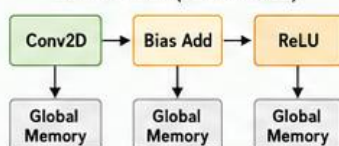
Compiler Optimization



1. Operator / Kernel Fusion

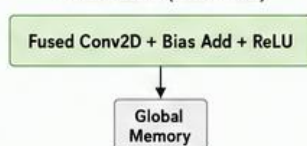
Multiple operators are merged into one fused kernel to reduce intermediate memory traffic and kernel launch overhead.

Before Fusion (Three Kernels)



- Intermediate results no longer need to be repeatedly written/read from memory.

After Fusion (One Kernel)

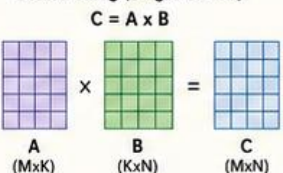


- Intermediate results no longer need to be repeatedly written/read from memory.
- Fewer kernel launches and lower overhead.

2. Tiling (Blocking)

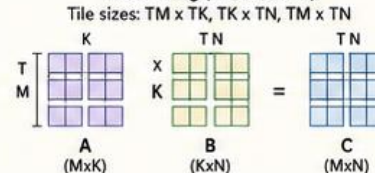
A large computation is split into smaller tiles to better use cache/shared memory and increase locality and parallelism.

Before Tiling (Single GEMM)



- Tiles fit in cache/shared memory.
- Improved data reuse.
- Reduced memory bandwidth demand.
- Higher compute efficiency.

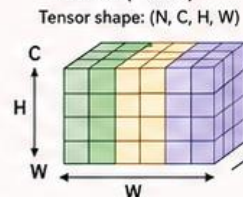
After Tiling (Tiled GEMM)



3. Layout Transformation

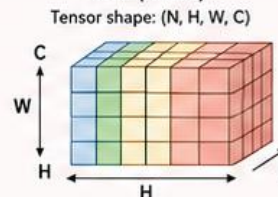
Tensor layouts are transformed into hardware-friendly forms to improve memory access patterns and vectorization.

Before (NCHW)



- More contiguous (coalesced) memory accesses.
- Better SIMD/vectorization and memory controller efficiency.

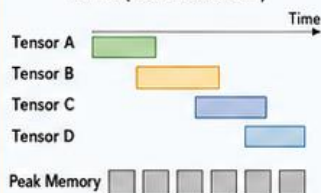
After (NHWC)



4. Memory Planning

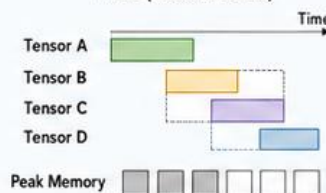
Tensor buffer lifetimes are analyzed so memory can be reused and peak memory usage minimized.

Before (Naive Allocation)



- Liveness analysis to determine when tensors are alive.
- Reuse buffers that are not live at the same time.
- Reduced peak memory footprint enables larger models/batches.

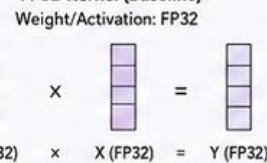
After (Planned Reuse)



5. Quantized Kernel

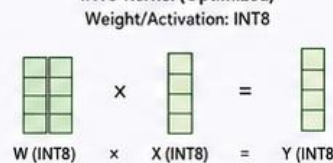
FP32 operations are converted to lower-precision kernels such as INT8 or INT4 to reduce memory and accelerate inference.

FP32 Kernel (Baseline)



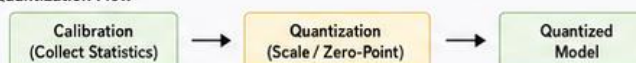
- Smaller memory footprint (~4x smaller than FP32 for INT8 weights).

INT8 Kernel (Optimized)



- Specialized low-precision hardware can execute faster.

Quantization Flow



6. Sparse Kernel

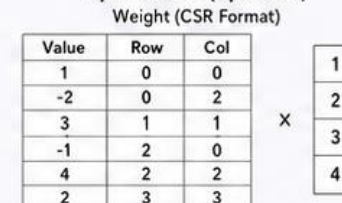
Sparse kernels exploit zeros in weights or activations to skip unnecessary computation and memory access.

Dense Kernel (Baseline)



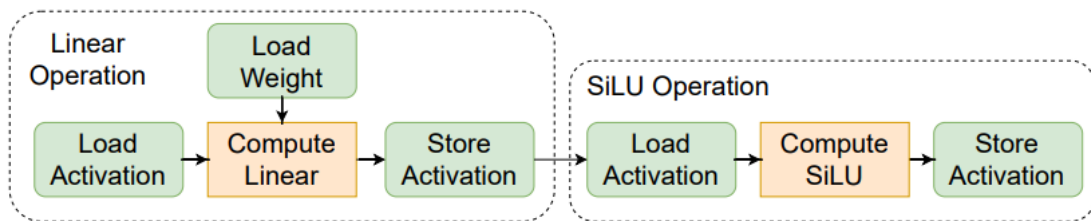
- Dense kernel computes all elements, including zeros.

Sparse Kernel (Optimized)

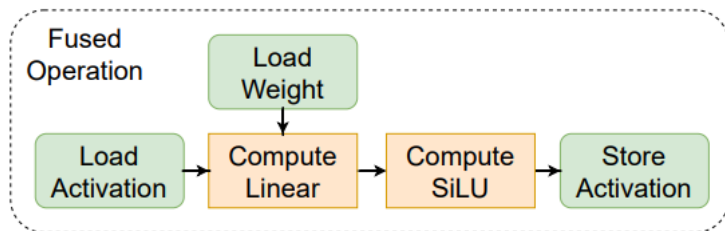


- Sparse kernel computes only nonzero elements.
- Reduced arithmetic and memory traffic can accelerate execution.

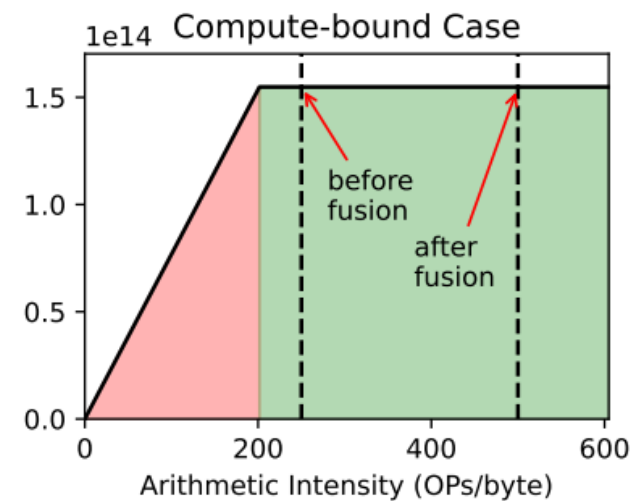
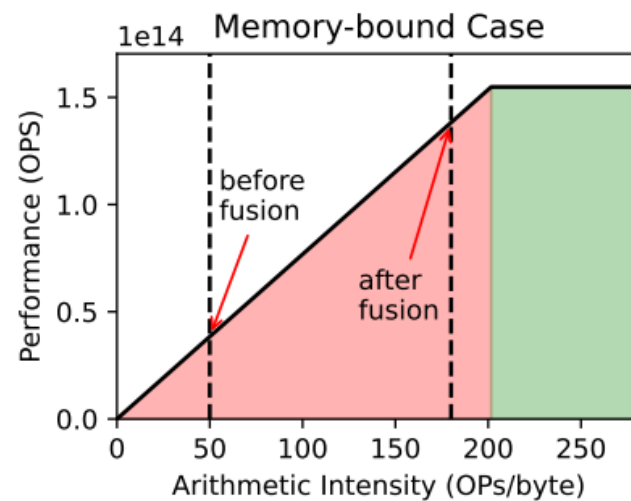
Operator/Kernel Fusion



(a) Linear operation and SiLU Operation



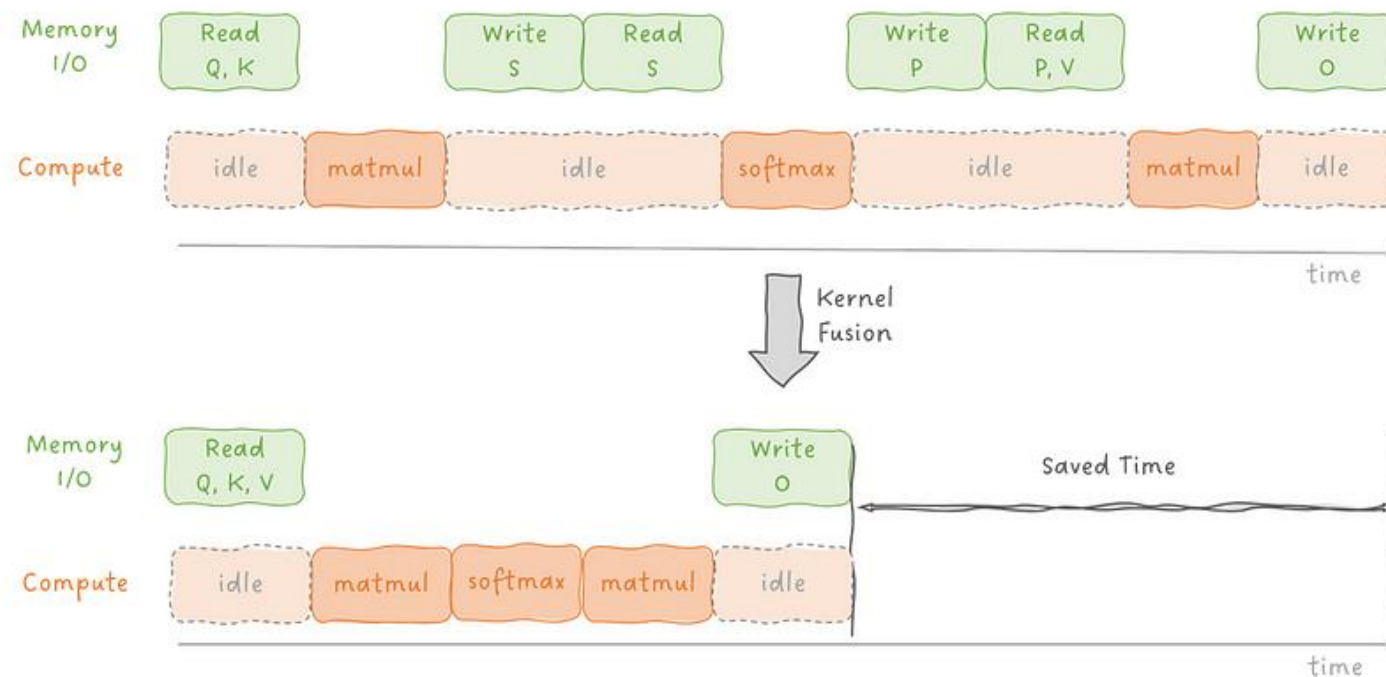
(b) Fused Operation



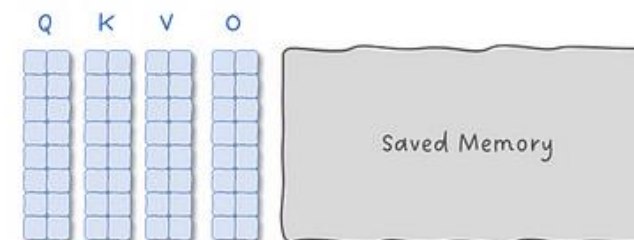
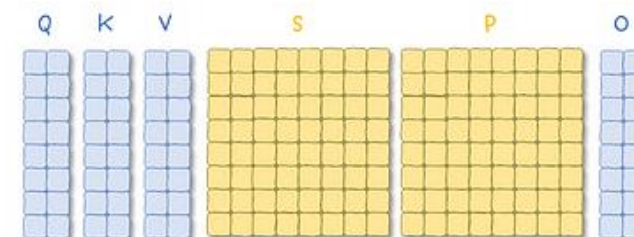
Operator Fusion

Simplified Attention
Equation

$$O = \text{softmax}(QK^T)V$$

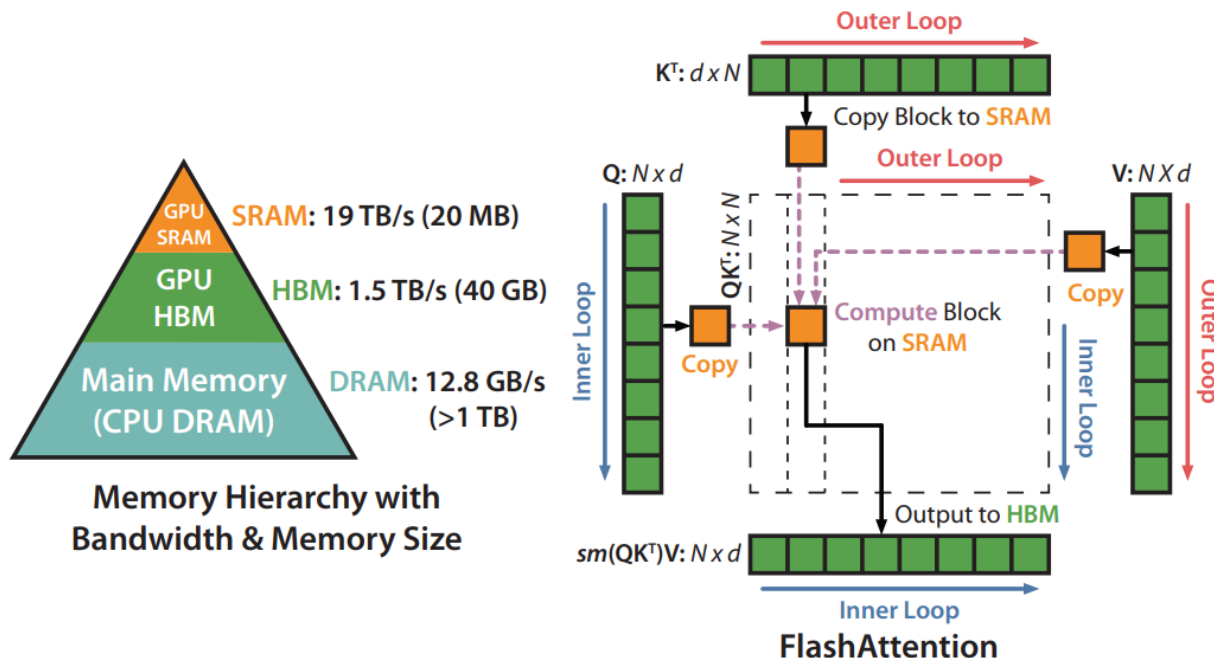


HBM
Memory



FlashAttention

- Avoid reading and writing the attention matrix to and from GPU global memory with fused kernel
- Computing the **row-wise softmax** need to access to whole input
- Softmax reduction: Tiling and Recomputation



$\mathbf{A}$	$\exp(\mathbf{A})$	$\exp(\mathbf{A}) \cdot \mathbf{1}^m$	$\frac{\exp(\mathbf{A})}{\exp(\mathbf{A}) \cdot \mathbf{1}^m}$																																				
<table><tr><td>1</td><td>0</td><td>$-\infty$</td></tr><tr><td>$-\infty$</td><td>1</td><td>$-\infty$</td></tr><tr><td>$-\infty$</td><td>$-\infty$</td><td>0</td></tr></table>	1	0	$-\infty$	$-\infty$	1	$-\infty$	$-\infty$	$-\infty$	0	<table><tr><td>2.7</td><td>1</td><td>0</td></tr><tr><td>0</td><td>2.7</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td></tr></table>	2.7	1	0	0	2.7	0	0	0	1	<table><tr><td>3.7</td><td></td><td></td></tr><tr><td>2.7</td><td></td><td></td></tr><tr><td>1</td><td></td><td></td></tr></table>	3.7			2.7			1			<table><tr><td>0.7</td><td>0.3</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td></tr></table>	0.7	0.3	0	0	1	0	0	0	1
1	0	$-\infty$																																					
$-\infty$	1	$-\infty$																																					
$-\infty$	$-\infty$	0																																					
2.7	1	0																																					
0	2.7	0																																					
0	0	1																																					
3.7																																							
2.7																																							
1																																							
0.7	0.3	0																																					
0	1	0																																					
0	0	1																																					
inputs (logits)	row-wise sum		inputs (softmax)																																				

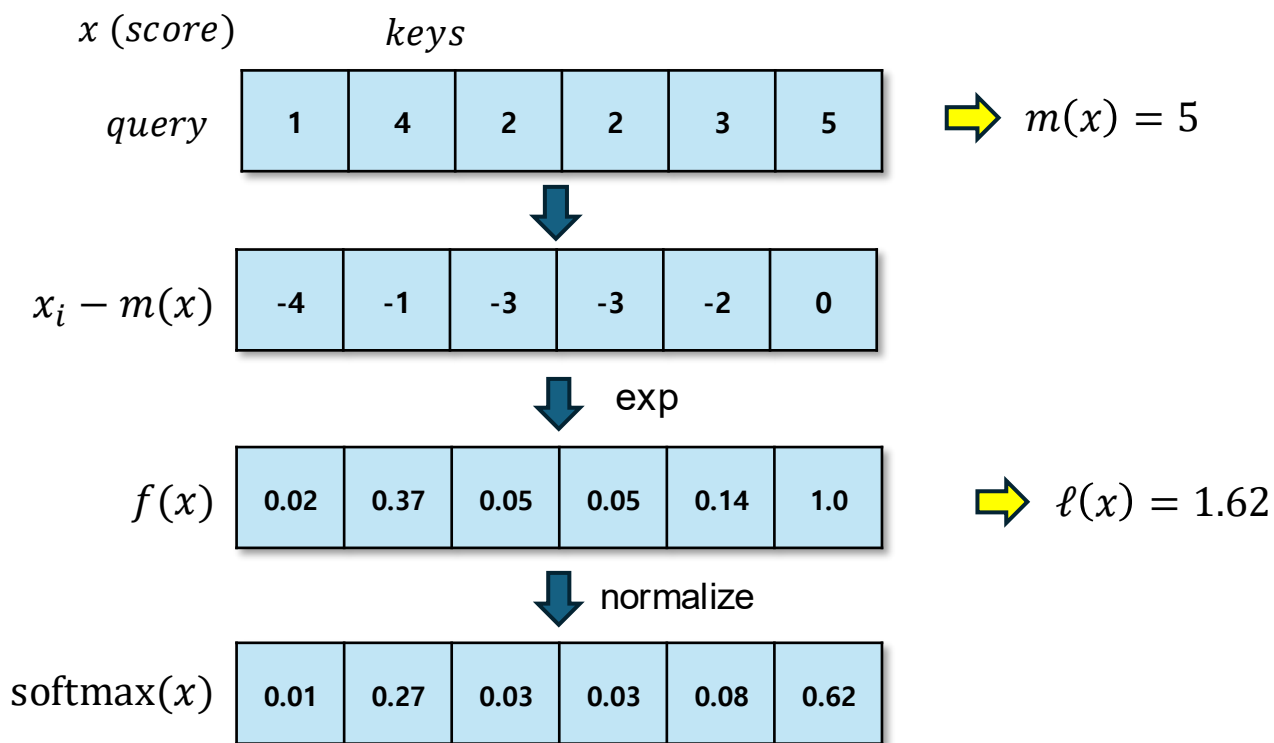
Image from <https://peterbloem.nl/blog/stable-softmax>

FlashAttention

- Standard Softmax vs Tiled Softmax

$$m(x) := \max_i x_i, \quad f(x) := [e^{x_1 - m(x)} \quad \dots \quad e^{x_B - m(x)}],$$

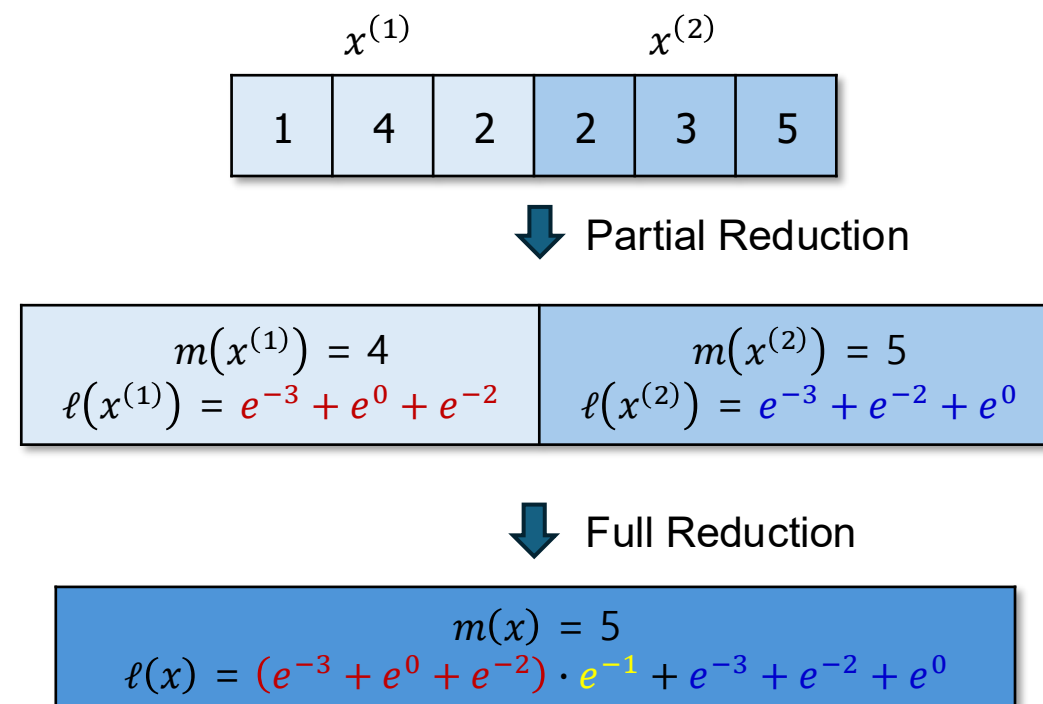
$$\ell(x) := \sum_i f(x)_i, \quad \text{softmax}(x) := \frac{f(x)}{\ell(x)}$$



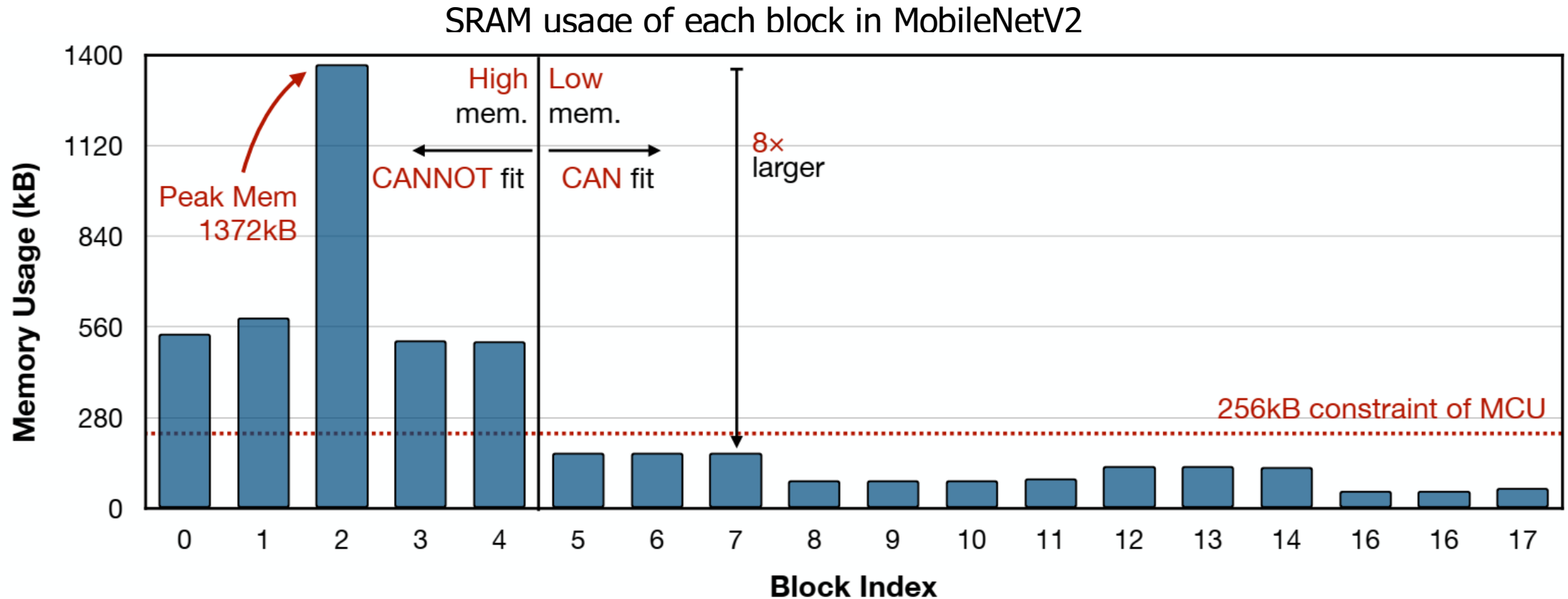
$$m(x) = m([x^{(1)} \ x^{(2)}]) = \max(m(x^{(1)}), m(x^{(2)})),$$

$$f(x) = [e^{m(x^{(1)}) - m(x)} f(x^{(1)}) \quad e^{m(x^{(2)}) - m(x)} f(x^{(2)})]$$

$$\ell(x) = \ell([x^{(1)} \ x^{(2)}]) = e^{m(x^{(1)}) - m(x)} \ell(x^{(1)}) + e^{m(x^{(2)}) - m(x)} \ell(x^{(2)})$$

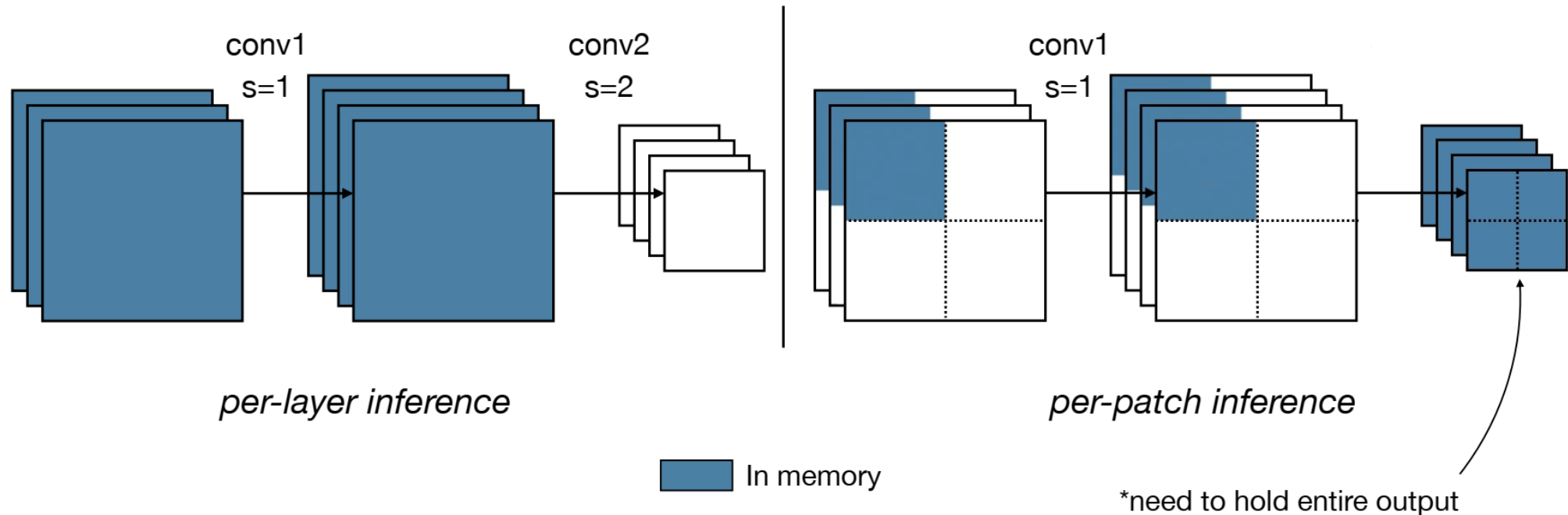


Patch-based Inference



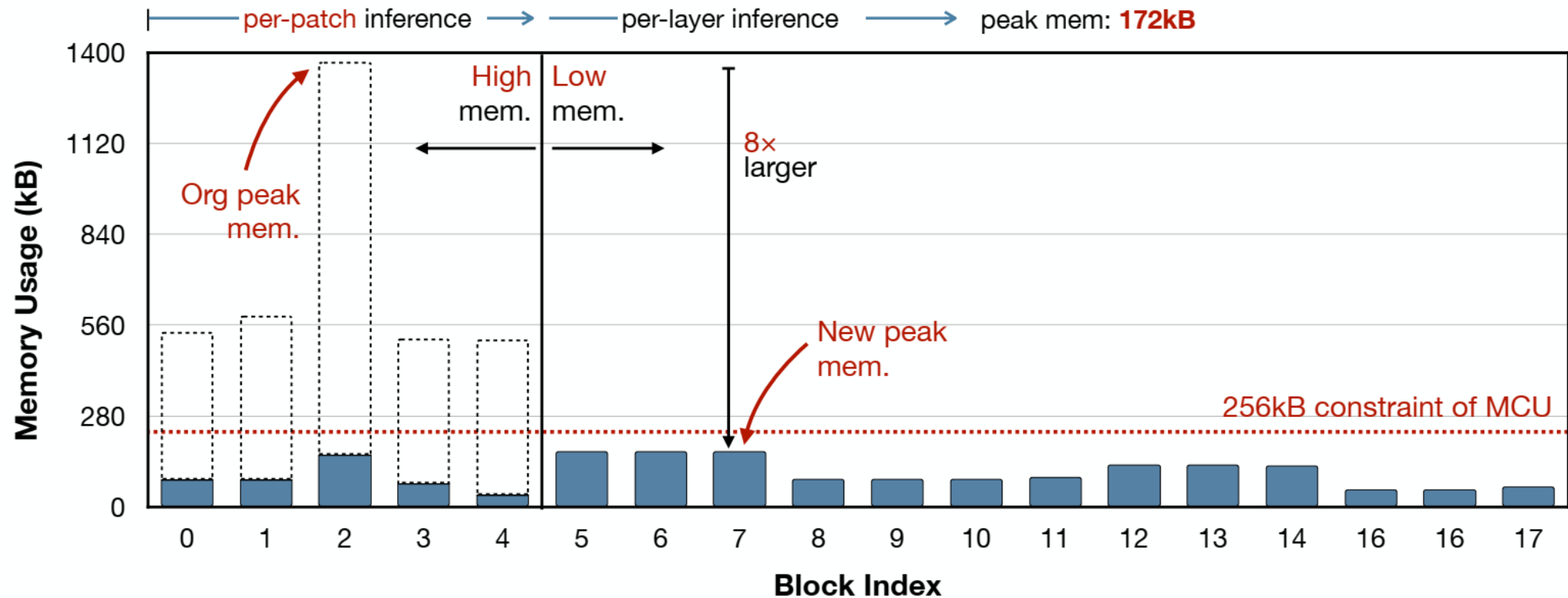
Patch-based Inference

- Saving Memory with Patch-based Inference
 - Break the memory bottleneck with patch-based inference



Patch-based Inference

- Saving Memory with Patch-based Inference
 - Memory saving for MobileNetV2



Patch-based Inference

- Problem: halo leads to repeated computation

